

ATLAS

Version 3.0

User Manual

Cohort definition and analytics on OMOP CDM data

Peter Hoffmann

First Edition — September 28, 2026

Preface

This is the user manual for ATLAS v3.0, the next version of the OHDSI ATLAS web application. It is a working manual: it describes what each area of the application does, how to use it, and the ATLAS-v3.0-specific behaviour that a researcher meets at the keyboard — the configuration panel, the version-history tabs, the cohort-builder controls, the plugin surface, the auto-save semantics — in the order a user encounters them.

This manual is *complementary* to *The Book of OHDSI* [20], not a replacement for it. The Book is the canonical reference for the methodology behind every ATLAS feature: the OMOP Common Data Model, the standardised vocabularies, phenotype development, characterization, comparative effectiveness, clinical and method validity. Where this manual touches one of those topics, it does so only enough to name the ATLAS controls a user is looking at, and points at the relevant Book chapter for the rest. Read them alongside each other.

Audience

The manual serves several readers:

- **Researchers and analysts** who design cohort definitions, run characterizations, pathways, or incidence rate analyses, and read the resulting reports.
- **Clinicians and domain experts** reviewing concept sets and cohort designs for clinical validity — they need enough of the interface vocabulary to comment usefully without becoming ATLAS operators.
- **Administrators** configuring data sources, managing roles, and supporting users in their organisation.
- **New OHDSI participants** for whom ATLAS v3.0 is their first contact with the OMOP Common Data Model and the OHDSI toolchain.

What this manual is, and is not

This is a *task-oriented* manual. Each chapter takes one feature, explains what it lets you do, and walks through the workflow. Where a screen requires deeper conceptual background — the difference between a standard concept and a source code, why a cohort is more than a list of people — the manual stops to explain rather than glossing over. Cross-references rather than re-explanations are used when a concept reappears in a later chapter.

This is not an installation handbook for first-time deployment; the appendix gives a working setup recipe but the canonical instructions live in the project's repository. It is

also not a WebAPI reference. The manual documents the user interface; if you need to call WebAPI directly — to script cohort generation, batch concept-set imports, or build a plugin against the same endpoints the UI uses — the appendix points to WebAPI’s live swagger UI, which is the authoritative endpoint reference. Per-section endpoint footnotes are deliberately not included.

Structure

The chapters follow the order in which most users meet the features:

- **Foundations** (Chapters 1–2) — what ATLAS v3.0 is for, and how to sign in and navigate.
- **Building blocks** (Chapters 3–5) — data sources, vocabulary search, and concept sets, the components every later workflow draws on.
- **Cohort definitions** (Chapter 6) — the longest single workflow in the application: entry events, inclusion criteria, exit, generation, reports, and the diagnostics-and-evaluation cycle the OHDSI community wraps around it.
- **Analyses** (Chapters 7–11) — characterizations, cohort pathways, incidence rates, patient profiles (including phenotype validation with PheValuator), and reusable feature analyses.
- **Operations** (Chapters 12–14) — versioning, administration, and the plugin system.
- **Appendix A** — a setup recipe for WebAPI v3.0 and the ATLAS v3.0 frontend, plus pointers to the live WebAPI reference and the wider OHDSI toolchain.

Conventions

Visible UI labels are bold (**Save**). Routes, configuration keys, and code identifiers appear in monospace (`/cohorts`). Menu paths use a sans-serif arrow form (Analysis → Characterizations). Screenshots are shown framed; until real captures replace them, slots are marked with a placeholder caption that names the figure to come. Subdued left-rule blocks distinguish notes, tips, cautions, and concept side-bars from the running prose.

A short cheat sheet of the OHDSI ecosystem (Achilles, HADES, PheValuator, Cohort-Diagnostics, Athena, Broadsea, Phenotype Library, Themis) is in the introduction’s “OHDSI toolchain at a glance” notebook. A glossary of OMOP / OHDSI terminology (cohort, concept, era, index date, standard concept, etc.) is at the back of the manual.

Acknowledgments

ATLAS v3.0 is the work of the OHDSI community continuing the long tradition of the original ATLAS. This manual draws on the OHDSI ATLAS wiki [21], *The Book of OHDSI* [20], and on the ATLAS v3.0 codebase itself. Any error in description belongs to the manual; the application owes its shape to its many contributors.

Contents

Preface	i
I Foundations	1
1 Introduction	2
1.1 What ATLAS v3.0 is for	2
1.2 What ATLAS v3.0 is not	5
1.3 How this manual is organised	5
1.4 Conventions used	6
2 Getting started	7
2.1 Before you log in	7
2.2 Signing in	7
2.2.1 The license agreement	9
2.2.2 Sessions and how they expire	9
2.3 The main navigation	9
2.4 Common list and editor conventions	11
2.4.1 Searching lists	11
2.4.2 Columns and sorting	11
2.4.3 Importing from a list	11
2.4.4 Why is Save disabled?	11
2.4.5 Who created and changed an asset	12
2.4.6 Leaving an editor with unsaved changes	12
2.5 Switching theme	12
2.6 Personal API keys	12
2.7 Links from ATLAS 2.x	13
2.8 Switching language	13
2.9 A first walkthrough	14
3 A first phenotype: end-to-end	15
3.1 The phenotype	15
3.2 Step 1 — Confirm the source is available	15
3.3 Step 2 — Build the metformin concept set	15
3.4 Step 3 — Build the diabetes concept set	16
3.5 Step 4 — Create the cohort definition	16
3.6 Step 5 — Add the inclusion rule	17

3.7	Step 6 — Set cohort exit	17
3.8	Step 7 — Generate	17
3.9	Step 8 — Read the inclusion-rule report	18
3.10	Step 9 — Eyeball some persons	18
3.11	Where to go from here	18
II	Building Blocks	19
4	Data sources	20
4.1	Picking a source	20
4.2	Reading a source report	21
4.2.1	Dashboard	21
4.2.2	Data Density	22
4.2.3	Person	22
4.2.4	Visit	22
4.2.5	Condition / Drug / Procedure / Measurement / Observation . .	22
4.2.6	Death	23
4.3	Comparing sources	23
4.4	Data quality	23
4.5	Patient cache and cohort reports on a source	24
5	Vocabulary search	25
5.1	Where vocabulary search lives	25
5.2	Running a search	26
5.2.1	What matches	26
5.2.2	Reading the result table	26
5.3	Concept details	27
5.3.1	Header	28
5.3.2	Stat cards	28
5.3.3	Concept attributes	28
5.3.4	Hierarchy mini-map	28
5.3.5	The full hierarchy dialog	28
5.3.6	Related concepts	29
5.3.7	Drilldown report	29
5.4	From search to set	30
5.5	Search strategies that work in practice	30
6	Concept sets	31
6.1	Why a list of concepts is not enough	31
6.2	Creating a concept set	31
6.3	The editor tabs	32
6.4	The Selected tab	32
6.4.1	Concepts the vocabulary does not contain	33

6.5	The Included tab	33
6.6	The Source Codes tab	34
6.7	Bulk-adding concepts	34
6.7.1	Paste IDs	34
6.7.2	Import by source code	34
6.7.3	Import JSON	35
6.7.4	Optimize	35
6.8	Tags	36
6.9	Recommend	37
6.10	Compare	38
6.11	Saving	39
6.12	Picking a concept set from another editor	39
6.13	The embedded concept-set editor	40
6.14	Starting from a phenotype library	40
6.15	Discovery	41
III	Cohort Definitions	43
7	Cohort definitions	44
7.1	Live preview	46
7.2	Listing and creating cohorts	46
7.2.1	Duplicating a cohort	47
7.3	The four logical pieces of a definition	47
7.4	Entry event (primary criteria)	48
7.4.1	Episode criteria	49
7.4.2	Custom era criteria	49
7.4.3	Cohort entry observation window	50
7.4.4	Three qualifying-event limits	50
7.5	Inclusion criteria	51
7.5.1	Rail and detail layout	51
7.5.2	Anatomy of a rule	52
7.5.3	Grouping criteria with AND / OR	52
7.5.4	The criterion inside a group	53
7.5.5	Reading the inclusion report	54
7.6	Cohort exit and censoring	56
7.6.1	Exit strategy	56
7.6.2	Censoring events	56
7.6.3	Era collapse and trimming	56
7.7	Concept sets used by this cohort	57
7.8	Generation	58
7.8.1	Cohort samples	59
7.9	Going further: external diagnostic tools	60
7.10	Saving, copying and exporting	61

7.10.1	Editing the cohort as JSON	62
7.10.2	Viewing the generated SQL	62
7.10.3	Tagging a cohort	63
7.10.4	Sharing a cohort	64
7.10.5	Auto-save and draft recovery	64
IV	Analyses	65
8	The Analysis Hub	66
8.1	Where the analyses live	66
8.2	Lists, editors, and results	67
8.3	A note on Profiles	67
9	Characterizations	69
9.1	What you can answer with a characterization	69
9.2	Anatomy of a characterization	70
9.2.1	Building a subgroup	70
9.3	Generating and viewing results	71
9.3.1	When a run fails or has no results	72
9.3.2	Result view modes	72
9.4	Export	73
9.5	Common pitfalls	73
9.6	Reproducing a characterization in R	75
10	Cohort pathways	76
10.1	When this analysis is the right tool	76
10.2	Setting up a pathway	76
10.3	Reading the result	77
11	Incidence rates	79
11.1	Setting up an incidence rate analysis	79
11.2	Reading the result	80
11.2.1	The Rate summary rail	81
12	Profiles	83
12.1	Finding a person	83
12.2	The timeline view	83
12.3	The rest of the profile page	84
12.3.1	The Demographics strip	84
12.3.2	The Observation periods band	85
12.3.3	The Highlights panel	85
12.4	Validating a phenotype using Profiles	85
12.5	Going further: PheValuator	87

13 Feature analyses	88
13.1 Built-in versus custom features	88
13.2 Finding a feature analysis	88
13.3 Creating a feature analysis	89
13.3.1 Prevalence Criteria	89
13.3.2 Distribution Criteria	89
13.3.3 Custom SQL	89
13.3.4 Concept sets and the toolbar	91
13.4 Reuse	91
 V Operations	 92
14 Versions	93
14.1 Where versions live	93
14.2 Previewing a version	93
14.3 Restoring a version	94
14.4 Pinning a published study	94
 15 Administration	 96
15.1 The configuration panel	96
15.2 Caches and TrexSQL	97
15.3 Managing data sources	98
15.4 Sharing an asset: Configure access	99
15.5 Roles and permissions	100
15.5.1 Built-in roles	100
15.5.2 Editing a role	100
15.5.3 Creating, importing and deleting a role	101
15.5.4 Permission reference	101
15.6 Jobs and long-running tasks	102
15.7 Audit information	102
 16 Plugins	 104
16.1 What plugins look like	104
16.2 Common plugin categories	104
16.3 Trust and isolation	105
16.4 The <code>plugins.json</code> manifest	105
16.5 Errors and troubleshooting	108
 17 Troubleshooting	 109
17.1 I cannot log in	109
17.2 I cannot see any data	110
17.3 Cohort generation does not produce what I expect	110
17.4 Concept set issues	111
17.5 Permissions: I cannot see or do something	112

17.6 Imports and analysis runs	113
17.7 Errors that show up in the browser console	113
17.8 Plugin problems	114
17.9 When the answer is not in this manual	114
A Setup guide: WebAPI and ATLAS v3.0	115
A.1 The pieces	115
A.2 Prerequisites	116
A.3 The Docker Compose path (recommended for evaluation)	116
A.4 Native install	118
A.4.1 1. Database	118
A.4.2 2. WebAPI v3.0	118
A.4.3 2a. Enable the TrexSQL query cache	118
A.4.4 3. Add data sources to WebAPI	120
A.4.5 4. ATLAS v3.0 frontend	120
A.4.6 Externalized runtime configuration	121
A.5 First-time login and creating a user	122
A.6 Common installation issues	122
A.7 Verifying a clean setup	123
A.8 Achilles: the missing prerequisite for descriptive reports	123
A.9 Related: Broadsea	123
A.10 Further reading and API reference	124
Glossary	125
Bibliography	127

PART I

Foundations

Introduction

ATLAS v3.0 is a web application for designing, generating, and reviewing patient cohorts on data that conforms to the OMOP Common Data Model (CDM) [4, 16, 29]. It is a modern reimplementaion of the long-standing OHDSI ATLAS tool. Functionally it covers the same ground as ATLAS 2.x — vocabulary search, concept sets, cohort definitions, characterizations, cohort pathways, incidence rates and patient profiles — built on top of the same WebAPI backend. The user interface has been rebuilt with Vue 3 [43] and Vuetify [8], and several workflows have been streamlined.

This manual is task-oriented. Each chapter takes one feature, explains what it lets you do, and walks through it in the order a user would meet it. If you are completely new to OHDSI, read this introduction and Chapter 2 first. If you already know ATLAS, you can jump straight to the chapter that matches the feature you need.

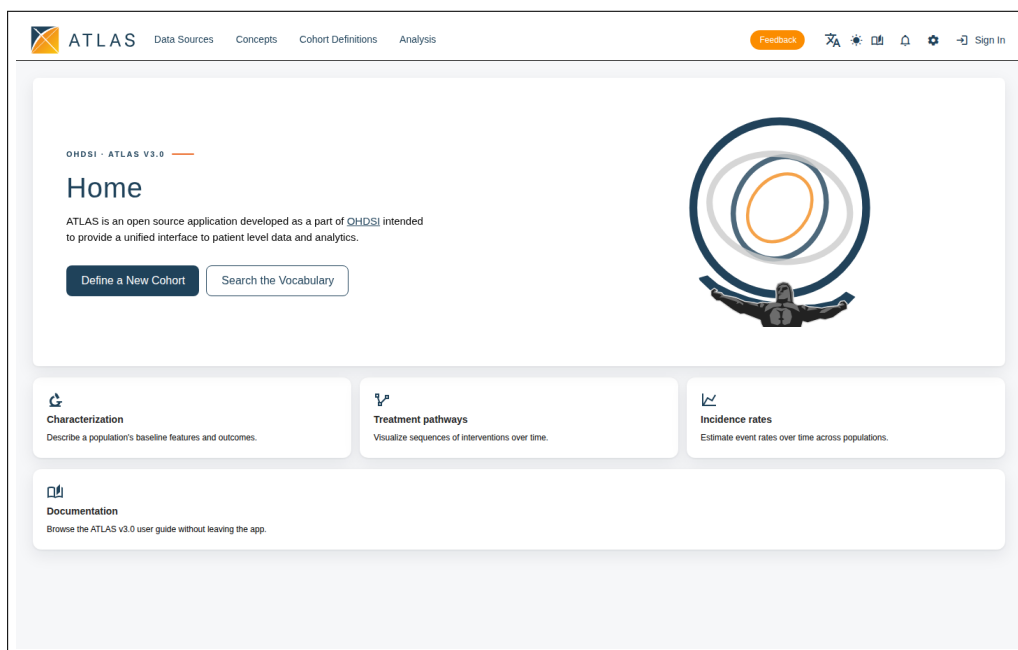


Figure 1.1. The ATLAS v3.0 home page. The top navigation lists the major application areas; the home card provides the two most-used entry points (**Define a New Cohort**, **Search the Vocabulary**) plus a row of analysis-area shortcuts.

1.1 What ATLAS v3.0 is for

Observational health data — claims, electronic health records, registries — answer different questions than clinical trials. They are large, messy, and the records were collected

for billing or care, not research. The OHDSI community addresses this with two pieces: a standardised data model (the OMOP CDM) and a library of methods that operate on that model. ATLAS v3.0 sits on top of both. It is the place where a researcher:

- explores which data sources are available and what they contain;
- searches the standardised vocabularies for the concepts that describe a clinical idea (a disease, a drug class, a procedure);
- collects those concepts into reusable *concept sets*;
- composes *cohort definitions* — formal, executable descriptions of a population such as "new users of metformin without a prior diagnosis of diabetes";
- generates the cohort against one or more data sources and reviews descriptive reports about it;
- runs additional analyses — characterizations, pathways, incidence rates — using cohorts as inputs.

Everything ATLAS v3.0 produces is a portable artefact. A cohort definition is stored as JSON and SQL and can be shared, version-controlled, and run again on a different network on a different CDM database to reproduce a finding. The OHDSI design ethos is to *execute analytics in place*: the patient-level data never leaves the hosting institution, and only aggregate results are pooled across the network [20].

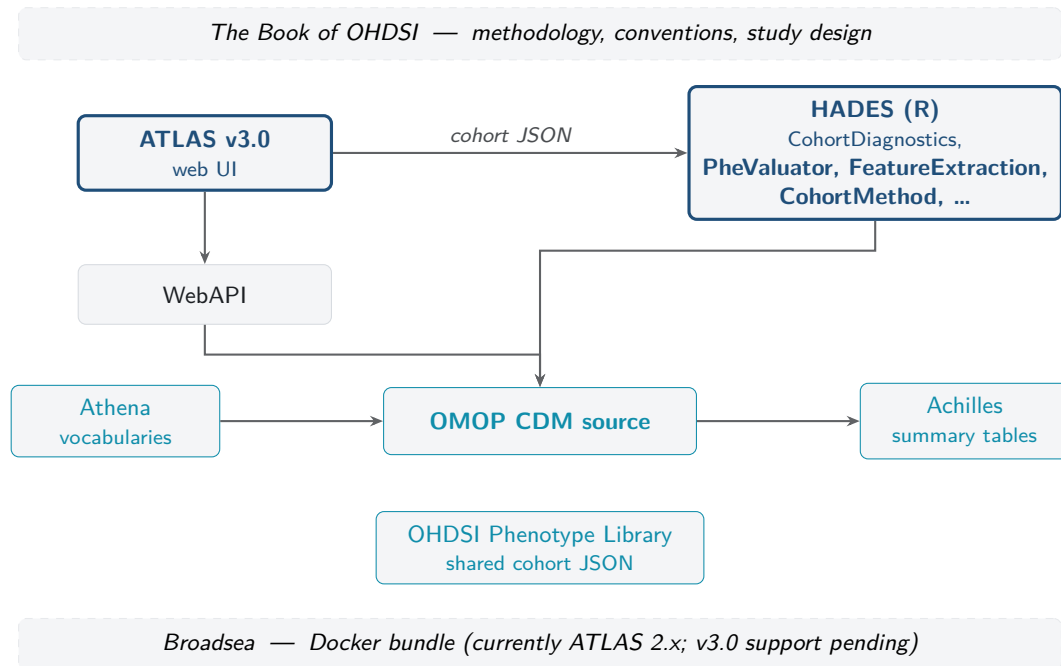
The cohort builder in ATLAS v3.0 is *rule-based*: a person is in the cohort if and only if they satisfy a set of explicit, hand-crafted criteria. Probabilistic phenotyping — where a model assigns each person a probability of meeting the phenotype — is a separate practice supported by the HADES PheValuator package (Chapter 12) [37].

The OMOP CDM is a person-centric relational schema. Every record about every person — a condition, a drug exposure, a measurement result — is stored in a small, predictable set of tables, and every clinical idea is represented by a *concept* drawn from the OHDSI standardised vocabularies. Because the schema and the vocabularies are the same everywhere, the same cohort definition you build in ATLAS v3.0 produces comparable results on a Korean claims database, a Dutch GP record system, and a US academic medical centre [41]. See *The Book of OHDSI*, chapter "OHDSI Analytics Tools" [20] for the longer treatment.

Note

The OHDSI toolchain at a glance. ATLAS is one piece of a larger OHDSI ecosystem. The names below are all referenced elsewhere in this manual; this is the cheat sheet:

ATLAS this web application; designs and runs cohort definitions and analyses.



How ATLAS v3.0 sits in the OHDSI ecosystem. Solid arrows are runtime data flows. Two further relationships are not drawn to keep the figure readable: ATLAS reads Achilles' summary tables when it renders descriptive reports, and ATLAS imports cohort JSON from the Phenotype Library.

WebAPI the Java/Spring backend ATLAS talks to. It compiles a cohort definition into SQL and executes it against a CDM source.

OMOP CDM the standardised relational schema all CDM sources conform to.

Athena the OHDSI vocabulary repository. The standardised vocabularies are released here.

Achilles an R package that profiles a CDM source and writes summary tables. ATLAS's descriptive reports read from those tables — if Achilles has not been run, the Data Sources page is blank.

HADES the OHDSI suite of R analytical packages (CohortMethod, FeatureExtraction, PatientLevelPrediction, and the two below).

CohortDiagnostics HADES package that runs deeper diagnostics on a cohort definition exported from ATLAS.

PheValuator HADES package that estimates phenotype sensitivity, specificity, and PPV against a probabilistic gold standard.

OHDSI Phenotype Library community-curated repository of vetted cohort definitions, each release DOI'd.

Themis the OHDSI conventions group; defines best practices for CDM and cohort design.

Broadsea a Docker bundle of the OHDSI stack (currently ATLAS 2.x; v3.0 support pending).

Book of OHDSI the canonical methodology reference for everything above. Cited throughout this manual.

1.2 What ATLAS v3.0 is not

ATLAS v3.0 is a *design and review* tool. It does not store the patient data itself, and it does not run advanced analytics in the browser. When you press **Generate**, ATLAS v3.0 sends the cohort definition to WebAPI, which compiles it to SQL and runs it against the chosen CDM database.

The Population-Level Estimation and Patient-Level Prediction [33] modules from older ATLAS versions are *not* part of ATLAS v3.0 and there is no concrete plan to re-implement them in this codebase. For comparative effectiveness and prediction work, use the OHDSI HADES Libraries [23] directly in R against the same CDM data, or fall back to the classic ATLAS instance your organisation maintains.

1.3 How this manual is organised

- **Chapter 2, Getting Started** — what you need before you log in, how to log in, a tour of the main navigation, conventions shared by every list and editor, the theme selector, personal API keys, and how old ATLAS 2.x links are handled.
- **Chapter 4, Data Sources** — picking a CDM database and reading its descriptive reports.
- **Chapter 5, Vocabulary Search** — searching the OMOP standardised vocabularies.
- **Chapter 6, Concept Sets** — building and sharing reusable collections of concepts.
- **Chapter 7, Cohort Definitions** — the longest chapter; covers entry events, inclusion rules, exit, censoring, generation, and reports.
- **Chapters 9–11** — the analytical modules: characterizations, pathways, incidence rates.
- **Chapter 12, Profiles** — exploring an individual patient timeline.
- **Chapter 13, Feature Analyses** — reusable covariate definitions consumed by characterizations.
- **Chapter 14, Versions** — saving, previewing and restoring previous versions of cohorts and concept sets.
- **Chapter 15, Administration** — roles and permissions, sharing individual assets with **Configure access**, managing data sources, and the patient cache.
- **Chapter 16, Plugins** — what plugins are and how they appear inside ATLAS v3.0.

1.4 Conventions used

- UI labels are bold: click **Save**, open the **Concept Sets** tab.
- Menu paths use chevrons: `Analysis >> Characterizations`.
- Routes and code are monospaced: `/cohorts`, the JSON key `conceptSet`.
- Side notes appear in coloured boxes:

Note

A short fact worth knowing but not essential to the procedure being described.

A faster way to do the same thing.

Something that can ruin your work or produce misleading results if you are not careful.

Throughout the manual, where a section maps onto a single WebAPI endpoint or store action, that information appears as a small footnote at the end of the section so power users know where the UI's data is coming from.

Getting started

This chapter assumes someone has already installed ATLAS v3.0 and pointed it at a WebAPI instance — installation is covered in the project's repository README. Here we look at the experience from a new user's perspective: what you need on day one, how to log in, and what each main area of the application is for.

2.1 Before you log in

You need three things:

1. A URL for the ATLAS v3.0 instance your organisation hosts. In Docker development this is usually `https://localhost`.
2. Credentials, or an identity provider (LDAP, OAuth, SAML, OpenID Connect) that the WebAPI is configured to accept.
3. At least one CDM database that WebAPI is connected to — without a data source, every analytical feature is read-only and most lists will be empty.

Note

Public OHDSI demonstration instances such as <https://atlas-demo.ohdsi.org> expose synthetic data. They are useful for trying ATLAS v3.0 risk-free but the population sizes there are not representative of the data you would use for a real study.

2.2 Signing in

Open the ATLAS v3.0 URL in a modern browser (Chromium-based browsers and Firefox are the test targets). At startup the app always calls `user/me`. When the WebAPI is configured for *anonymous access* that call returns an anonymous permission set, no login prompt appears, and you can browse whatever the anonymous role is allowed to see — the server still gates every action per-endpoint. The shipped Docker deployment instead turns authentication on through its runtime config (`userAuthenticationEnabled: true`), so a session is required before you act on WebAPI data.

When authentication is on, the login is a *modal dialog* rather than a separate page: the router guard opens it the first time you hit a gated action, for example clicking into **Cohort Definitions**.

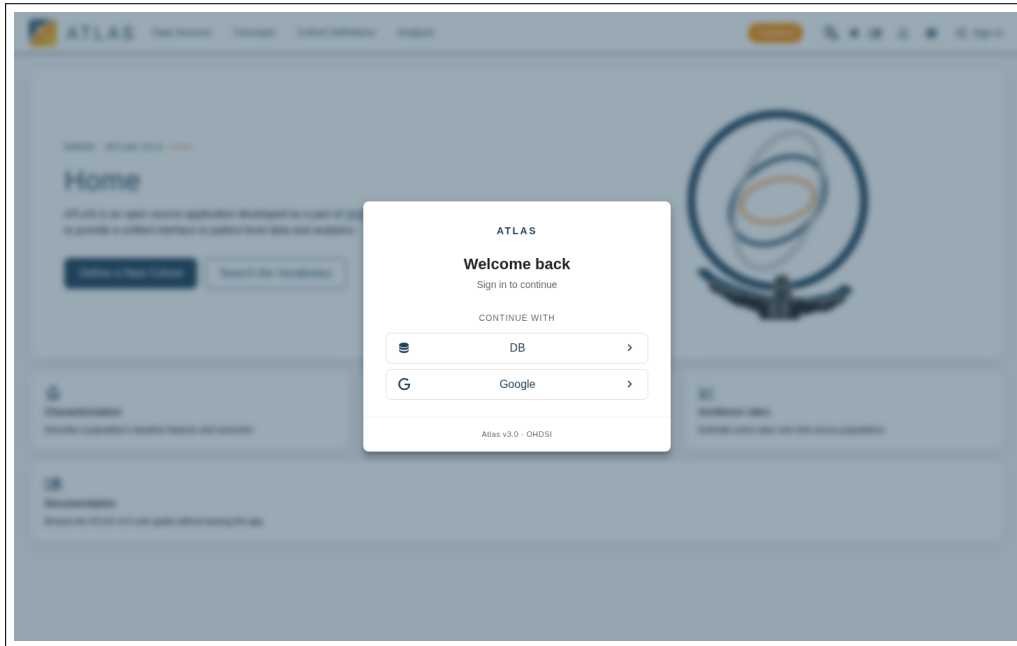


Figure 2.1. The login dialog (*Welcome back — Sign in to continue*). Under **Continue with** it lists one button per sign-in method the WebAPI offers — here the local database (**DB**) and **Google**.

The dialog asks WebAPI which sign-in methods it offers and lists one button per method under **Continue with** (if WebAPI returns none, the `authProviders` list from the runtime config is used instead). Picking a method that uses a username and password replaces the list with a credentials form; a **Back** link returns to the list when more than one method exists. What each method does depends on what the WebAPI is configured to accept:

Username and password A simple form. Used by LDAP-backed deployments and by the local user database.

OAuth providers One button per configured provider, e.g. Google. Clicking opens the provider’s consent screen and returns to ATLAS v3.0.

SAML / OpenID Connect Concrete supported single-sign-on flows. The provider redirects back to a dedicated callback route (`/saml/callback`, `/openid/callback`, or `/oauth/callback`) which finishes the sign-in and drops you back where you started. On `/oauth/callback` the provider hands back a one-time code, which ATLAS exchanges for a session token through WebAPI’s `/user/login/otc` endpoint; if the exchange fails, the WebAPI error is shown. Older hand-offs that put the token itself in the URL (`?token=...`) are no longer accepted.

After a successful sign-in your username appears in the top right. Clicking it opens the user menu: any entries contributed by plugins, then **API Keys** (Section 2.6) and **Sign Out**.

Note

Deep links. ATLAS v3.0 honours two query parameters when the app loads: `?cohortId=X` opens that cohort directly, and `?route=/path` navigates to an arbitrary in-app route. They are processed once, on the first navigation after boot — handy for sharing a link straight to a specific artefact. Links copied from an ATLAS 2.x instance also keep working; see Section 2.7.

2.2.1 The license agreement

On the first visit from a browser, ATLAS opens a **License Agreement** dialog asking you to accept the SNOMED International SNOMED CT and HemOnc license terms that govern the vocabulary content you are about to browse. **Accept** records the acceptance in the browser's local storage for 365 days, after which the dialog appears again. **Reject** shows a message that ATLAS cannot be used without accepting, and the dialog stays open. The acceptance is per browser, not per account: a new browser, a private window, or cleared site data brings the dialog back.

2.2.2 Sessions and how they expire

ATLAS v3.0 stores a JWT bearer token in the browser. The token has a fixed lifetime; a few minutes before it expires ATLAS v3.0 shows a dialog inviting you to extend the session. If you walk away from your machine the token quietly expires and the next gated action re-opens the login modal (or, on a deployment configured for anonymous access, silently drops you back to the anonymous permission set). Save before you step away: the only work ATLAS v3.0 recovers on its own is a *new*, never-saved cohort definition, whose draft it keeps in session storage and restores when you return to the new-cohort editor (Chapter 7).

If you keep getting logged out unexpectedly, check the system clock on the WebAPI server. A clock skew larger than the token's clock-tolerance makes every freshly issued token look already expired.

2.3 The main navigation

The top of the screen has a navigation bar with the major application areas, plus a configuration menu for users with administrative access. The left sidebar (where present) gives a chapter-by-chapter breakdown of the area you are currently in.

The top navigation bar contains four built-in entries plus a small control cluster on the right. From left to right:

Data Sources The list of connected CDM databases, with a descriptive report per source. Covered in Chapter 4.



Figure 2.2. The top navigation bar, here in an anonymous session. From left: the ATLAS logo, the four built-in entries, then the right-hand cluster — Feedback button, language selector, theme selector (sun icon), user manual, notifications, jobs panel (shown only with the `job:execution:get` permission), configuration cog, and **Sign In** (replaced by your username and the user menu once you are signed in).

Concepts Browse, search and edit reusable collections of concepts. Vocabulary search lives inside the same area, on a sibling tab. Covered in Chapter 5 and Chapter 6.

Cohort Definitions List, search, filter, edit and generate cohorts. Covered in Chapter 7.

Analysis A tabbed area combining [Analysis](#) > [Characterizations](#), [Analysis](#) > [Cohort Pathways](#), [Analysis](#) > [Incidence Rates](#) and [Analysis](#) > [Feature Analyses](#). Each is its own chapter (9–13).

The right-hand control cluster carries, from left to right:

Feedback Opens an external feedback form configured by the deployment.

Language selector See *Switching language* below.

Theme Light, dark, or system theme (Section 2.5).

User manual The book icon opens this manual inside the app (`/docs`).

Notifications The bell lists recent notifications, with an unread count.

Jobs A jobs icon that opens a drawer listing long-running server tasks (generations, cache builds). Shown only to users holding the `job:execution:get` permission.

Configuration cog Opens the configuration panel. Shown only when you hold at least one administrative permission or a plugin contributes an admin tab — if you cannot see it, ask your administrator (Chapter 15).

User menu / Sign In Your username and the user menu once signed in; a **Sign In** button otherwise.

Plugins your organisation has installed extend this navigation with additional entries (Chapter 16).

Note

Permissions affect what you see. Many actions in ATLAS v3.0 are gated by per-entity permissions: you may see a cohort or concept set in a list but find that **Save** or **Delete** is disabled, and the configuration cog is hidden entirely if you do not have any admin permission. Hover a disabled **Save** to see why (Section 2.4). ATLAS re-reads your permissions on every page change, so a role granted by an administrator takes effect without signing out and back in. Chapter 15 describes the role and permission model.

A few features live outside the top nav and are reached by other means:

Home Click the ATLAS logo on the far left to return to the landing page at any time.

Profiles The per-patient timeline view has no top-nav entry; open it by URL (`/profiles/:sourceKey/:personId`). Inside a cohort definition, clicking a person in a cohort sample shows the same profile inline in the report drawer (Chapter 12).

2.4 Common list and editor conventions

The list pages (cohorts, concept sets, and the four analysis types) and the editors behind them share a handful of behaviours. They are described once here and referred to from the later chapters.

2.4.1 Searching lists

The search box above a list splits what you type on spaces and keeps a row only if every term matches, in any order. `p1 schiz` therefore finds "[PL] ... schizophrenia", and `fib atrial` finds "Atrial fibrillation". Which fields are searched depends on the list: the concept-set and characterization lists match the name only; the cohort, pathway and incidence-rate lists match the name and the description; the feature-analysis list also matches the type and domain. In the concept tables' **Filter results** boxes (search results, **Included**, **Source Codes**), numeric ids and codes match whole or by prefix: `e11` finds `E11.9` but not `ME11`. The same any-order matching is used by concept-set and cohort pickers, the concept-set **Compare** tab, the hierarchy dialog, data-source prevalence tables, profile highlights, tag pickers, and the role and permission searches.

2.4.2 Columns and sorting

Every asset list shows an **Id** column and **Created** / **Updated** dates next to the author (**Created By** on the analysis lists). Lists open sorted by **Updated**, newest first, so what you touched last is at the top; an asset that was never modified uses its created date. The analysis lists show dates as relative time ("3 days ago") with the full date on hover. Click a column header to sort by it; sorting applies across all pages, not just the visible one. The **Tags** column is not sortable.

2.4.3 Importing from a list

The **Concept Sets**, **Characterizations**, **Cohort Pathways** and **Incidence Rates** lists carry an **Import** button (upload icon) next to **New**. Pick a `.json` file previously exported from ATLAS; the entity is created and opened in its editor. A file that is not valid JSON is refused with "*<file> is not valid JSON.*" and a failed creation shows the server's error message (or "*Import failed.*" when there is none). The cohort list has its own import dialog (Chapter 7); feature analyses have no import.

2.4.4 Why is Save disabled?

When an editor's **Save** button is disabled, hovering it shows the reason. ATLAS checks, in this order:

1. "You are previewing an earlier version. Return to the current version to save."
2. "You do not have permission to create a *<entity>*."
3. "You do not own this *<entity>* and have not been granted write access to it."
4. "Give the *<entity>* a name before saving."
5. "Resolve the validation errors before saving."

No tooltip appears when **Save** is disabled only because there is nothing to save or a save is already running. Write access to someone else's asset is granted with the **Configure access** lock button in the editor toolbar (Section 15.4).

2.4.5 Who created and changed an asset

Under the name and description, every editor shows an authorship line: "*Created by <name>, <date> · Updated by <name>, <date>*". The *Updated* half appears once the asset has been modified; when WebAPI records no user, the line reads just *Created <date>*.

2.4.6 Leaving an editor with unsaved changes

Navigating away from a cohort, concept set, characterization, or feature analysis with unsaved edits opens an **Unsaved changes** dialog: **Cancel** keeps you in the editor, **Discard changes** throws the edits away and leaves. Clicking outside the concept-set editor drawer counts as leaving.

2.5 Switching theme

The **Theme** button (a sun or moon icon, between the language selector and the user-manual icon) opens a menu with three choices:

Light The default on a first visit, unless the deployment configures a different default (Chapter 16).

Dark A high-contrast dark theme whose text and control colours meet WCAG AA contrast.

System Follows the operating system's light/dark setting and switches live when it changes.

The choice is stored per browser, so it follows you across sessions on the same machine but not to another device. Deployments can hide the selector entirely (`enableDarkMode: false`).

2.6 Personal API keys

Scripts and tools that call WebAPI directly can authenticate with a personal API key instead of your password. Open the user menu and pick **API Keys** (route `/profile/api-keys`). The page, titled **API Keys** ("*Manage personal API keys for programmatic access to WebAPI.*"), lists your keys with the columns **Name**, **Descrip-**

tion, **Key Identifier**, **Created**, **Expires**, **Last Used** and **Status** (**Active**, **Disabled** or **Expired**). With no keys yet it reads "You haven't created any API keys yet."

1. Click **New Key**. The **New API Key** dialog asks for a **Name** (required, e.g. "My laptop"), an optional **Description**, and an **Expiration**: **Never expires**, 30, 90 or 365 days, or **Custom...** with a **Number of days** (blank means never).
2. Click **Generate**. The dialog switches to **API Key Created** and shows the key once, with the warning "Copy this key now — you won't be able to see it again." Use **Copy**, store the key somewhere safe, and click **Done**.

Each row has two actions. **Revoke** stops the key working immediately but keeps it listed (status **Disabled**); **Delete** removes it permanently. Both ask for confirmation. How a script presents the key to WebAPI is documented in your WebAPI's swagger UI (see Appendix A).

2.7 Links from ATLAS 2.x

Bookmarks, wiki pages and papers often carry ATLAS 2.x links. ATLAS v3.0 recognises the old hash routes and redirects them; after the redirect the address bar shows the v3.0 form.

ATLAS 2.x link	Opens
<code>#/cohortdefinitions</code>	cohort list (<code>#/cohorts</code>)
<code>#/cohortdefinition/:id/...</code>	cohort editor
<code>#/cc/characterizations[:id]</code>	characterization list / editor
<code>#/cc/characterizations/:id/version/:v</code>	version preview
<code>#/cc/characterizations/:id/results/:exec</code>	characterization results
<code>#/cc/characterizations/:id/executions/:src</code>	editor, that source
<code>#/cc/feature-analyses[:id]</code>	feature-analysis list / editor
<code>#/pathways/:id/version/:v</code>	pathway version preview
<code>#/iranalysis[:id]</code>	incidence-rate list / editor
<code>#/iranalysis/:id/version/:v</code>	incidence-rate version preview
<code>#/concept/:id</code>	concept page (current source)
<code>#/search/:query</code>	vocabulary search (runs if 3+ characters)

2.8 Switching language

ATLAS v3.0 ships its translations *bundled* with the frontend — the JSON files under `src/locales/` (English, Korean, Russian, Chinese and Japanese). They are not fetched from WebAPI at runtime. Pick a language from the language selector in the header control cluster — the same control described under *The main navigation*, not the user menu; the interface re-renders without reloading the page. Keys a given locale does not translate fall back to bundled English. The text of cohort definitions and concept sets is stored verbatim — only the chrome around them is translated.

2.9 A first walkthrough

To get a feel for the application, the OHDSI community recommends this short loop on a demo dataset:

1. Open **Data Sources** and pick any source. Wait for the descriptive report to load and skim the demographics chart.
2. Open **Concepts**, switch to the **Search** tab and search the vocabulary for a familiar condition (e.g. "atrial fibrillation"). Tick **Descendants** in the **Add with** strip, add a couple of concepts — a new untitled concept set opens with them — then name the set and save.
3. Open **Cohort Definitions**, click **New Cohort**, set your concept set as the entry event, leave inclusion criteria empty, save, then expand the **Generation** section at the top of the builder and click **Generate** on the same data source's row.
4. When the row reaches **COMPLETED**, click its **Inclusion report** action — the report drawer shows the attrition funnel and per-rule breakdown.

That round-trip — vocabulary → concept set → cohort → report — is the core motion of ATLAS v3.0. The rest of the manual is variations on it.

A first phenotype: end-to-end

This chapter walks through one complete piece of work in ATLAS v3.0, from logging in to reading the cohort report. The reference chapters that follow describe each feature in depth; this one shows how they connect. If you are new to ATLAS, do this walkthrough on the public demo instance (<https://atlas-demo.ohdsi.org>) before tackling your own study.

Note

The demo instance ships a synthetic source called SYNPUF1K that is small enough to generate against in seconds. The exact counts you see may differ slightly from the figures below if the demo instance refreshes its sample, but the *shape* of the result will be the same.

3.1 The phenotype

Goal: *new users of metformin who do not have a prior diagnosis of diabetes recorded in the year before they start the drug.* This is a deliberately simple definition that touches every part of ATLAS: vocabulary search, two concept sets, an entry event with a washout, one inclusion rule using a zero-cardinality "absence" pattern, and a generation against the demo source.

3.2 Step 1 — Confirm the source is available

Open **Data Sources** from the top navigation. Pick SYNPUF1K from the source dropdown; the **Dashboard** report should render demographics and an observation-period chart. If the dashboard is empty, see Chapter 17.

3.3 Step 2 — Build the metformin concept set

1. Open **Concepts** from the top navigation; the **Concept Sets** list-tab is the default. Click **New Concept Set**; the editor opens as a drawer. Name it Metformin --- ingredient in the inline title at the top.
2. In the editor drawer, switch to the **Search** tab. Type metformin into the search hero and press Enter.

3. In the **Add with** strip above the results, tick **Descendants**; leave **Mapped** and **Exclude** off.
4. In the results table, sort or scan (or type ingredient into **Filter results**) to find the row whose name is exactly *metformin*, vocabulary RxNorm, type *Standard*, class *Ingredient* (concept ID 1503297). Click **Add** on that row. The editor switches to the **Selected** tab, where the row shows **Descendants** ticked. (If you forgot the strip, tick **Descendants** on the row here.)
5. Open the **Included** tab and skim the list — with descendants the set should cover branded and generic metformin products.
6. Click **Create** (or **Save**) on the drawer header to persist the set.

3.4 Step 3 — Build the diabetes concept set

The same flow with a different parent concept:

1. Click **New Concept Set**, name it *Diabetes mellitus*.
2. On the **Search** tab, search *diabetes mellitus*; tick **Descendants** in the **Add with** strip so all subtypes are covered, find the SNOMED standard concept whose name is exactly *Diabetes mellitus* (concept ID 201820), and click **Add**.
3. Check on the **Selected** tab that the row has **Descendants** ticked.
4. Click **Create**.

3.5 Step 4 — Create the cohort definition

1. Open **Cohort Definitions** from the top navigation. Click **New Cohort**. In the **Create a new cohort definition** dialog type *New users of metformin without prior diabetes* into **Name** and click **Create**; the empty builder opens. Click **Save** right away — a name is all a save needs, and the criteria can come later.
2. In step 1 of the editor (**Cohort Entry Events**), click **Add Criteria to Group** and pick **Drug Exposure** from the menu.
3. In the header of the new card ("a drug exposure of"), click **Select Concept Set**. The **Select Concept Set...** dialog opens (Section 6.12); search under **Import from repository** and pick *Metformin --- ingredient*. The button turns into a chip with the set's name.
4. Click the **Continuous observation: 0d Before · 0d After** chip at the top right of the entry-event group. In its popover set **Days before** to 365, leave **Days after** at 0, and close the popover.
5. Make sure the **Cohort Entry On** toggle in the step 1 header is on **Earliest** (the default for a new cohort), so that only each person's first qualifying drug exposure counts.

3.6 Step 5 — Add the inclusion rule

If the deployment has TrexSQL live preview and the demo source's patient cache is built, then after you add the first rule below pick SYNPUF1K in the source selector at the top of the inclusion-rule rail in step 2 (the rail appears once a rule exists; Section 7.1). The rail then shows running counts per rule as you edit — the "no prior diabetes" rule should drop the count noticeably when applied. If it does not, either the source has very few diabetes records or the concept set has not resolved correctly; treat the live counts as a sanity check.

1. In step 2 (**Inclusion Criteria**), click **Add rule**. Click the rule's header on the right and type `No prior diabetes diagnosis` into **Rule name**.
2. If the rule body shows an **Add criteria group** button, click it. Then click **Add criteria to group...** and pick **Condition Occurrence** from the menu.
3. Click **Select Concept Set** on the new card and pick `Diabetes mellitus`.
4. Click the temporal-window chip under the card ("event starts between ...") and choose the Quick Preset **Long-term baseline (–365 to 0 days)**.
5. Click the vertical cardinality label on the left of the card (it reads "at least 1"), choose **exactly** and set **Count** to 0. This is the no-negation pattern from Chapter 7: there is no NOT operator; absence is expressed as a zero-cardinality rule.

3.7 Step 6 — Set cohort exit

In step 3 (**Exit & Eras**), set the **Strategy** toggle to **Drug Exposure** so the person is in the cohort for as long as they keep getting metformin prescriptions. Click **Select Drug Concept Set** and pick `Metformin --- ingredient`, and leave **Gap Days** at 30 — the persistence window allowed between consecutive fills.

3.8 Step 7 — Generate

Click **Save** to persist the cohort, then expand the **Generation** section at the *top* of the builder (it starts collapsed). **Generate** and **Generate all** stay disabled while the cohort has unsaved changes or the design is invalid; hover them for the reason. In the data-source row for SYNPUF1K, click **Generate**; the row's status chip flips to **RUNNING** and updates as the job progresses. On the demo source the run completes in seconds and the row shows **COMPLETED** with a patient count.

Note

If the run ends as **FAILED**, or finishes with a blank patient count, or stays at **RUNNING** much longer than expected, the troubleshooting chapter (Chapter 17) covers the usual culprits: source not Achilles-processed, a patient cache that needs

to be built first, no rows because the concept set resolves to nothing on this source, or an attribute filter that excludes everyone.

3.9 Step 8 — Read the inclusion-rule report

When the SYNPUF1K row shows **COMPLETED**, click its **Inclusion report** action. The report drawer slides in from the right and shows the attrition funnel together with a per-rule satisfaction table:

- how many persons in the source had a qualifying drug-exposure entry event (this initial count already reflects the 365-day prior-observation requirement), and
- how many remained after the "no prior diabetes" rule was applied.

If any rule drops your numerator to zero, that is the rule to investigate first. If the diabetes-exclusion rule drops nobody, either there are no diabetes records before metformin in this source (unlikely) or the concept set resolved to nothing (check it in step 3).

3.10 Step 9 — Eyeball some persons

Close the inclusion-report drawer and click **Samples** on the same SYNPUF1K row. The samples panel opens in the drawer; click **New sample** to draw a random subset of the cohort (the sample header names its source, "Source: SYNPUF1K"), then click any person to open their profile (Chapter 12) inside the same drawer, with the membership window shaded on the timeline. The drawer's back-arrow returns to the samples list. Skim a few patients: do they look like new metformin users without a prior diabetes record?

3.11 Where to go from here

Variations on this same flow cover most of what ATLAS does:

- Add a second target cohort (e.g. new users of sulfonylureas) and run a *characterization* across both — Chapter 9.
- Define an outcome cohort (e.g. acute kidney injury) and run an *incidence rate* analysis with metformin as the target — Chapter 11.
- Define a small set of event cohorts (insulin, sulfonylurea, GLP-1 agonist, SGLT2 inhibitor) and run a *cohort pathway* analysis from the metformin target — Chapter 10.
- Open one of the cohort's persons in **Profiles** to see the actual records that triggered their cohort entry — Chapter 12.

The reference chapters that follow document each of these features in depth.

PART II

Building Blocks

Data sources

The **Data Sources** area lists every CDM database that the WebAPI is connected to and gives a descriptive view into each. It is the right place to start when you join a new project: before designing a cohort, you want to know what is in the data — how many persons, what condition mix, what time span, and how complete each domain is.

4.1 Picking a source

Open **Data Sources** from the top navigation. The page has two permanent controls:

- a *source picker* dropdown in the page header, showing every CDM database the WebAPI is connected to;
- a left-rail *report sidebar* listing the descriptive reports available for the selected source.

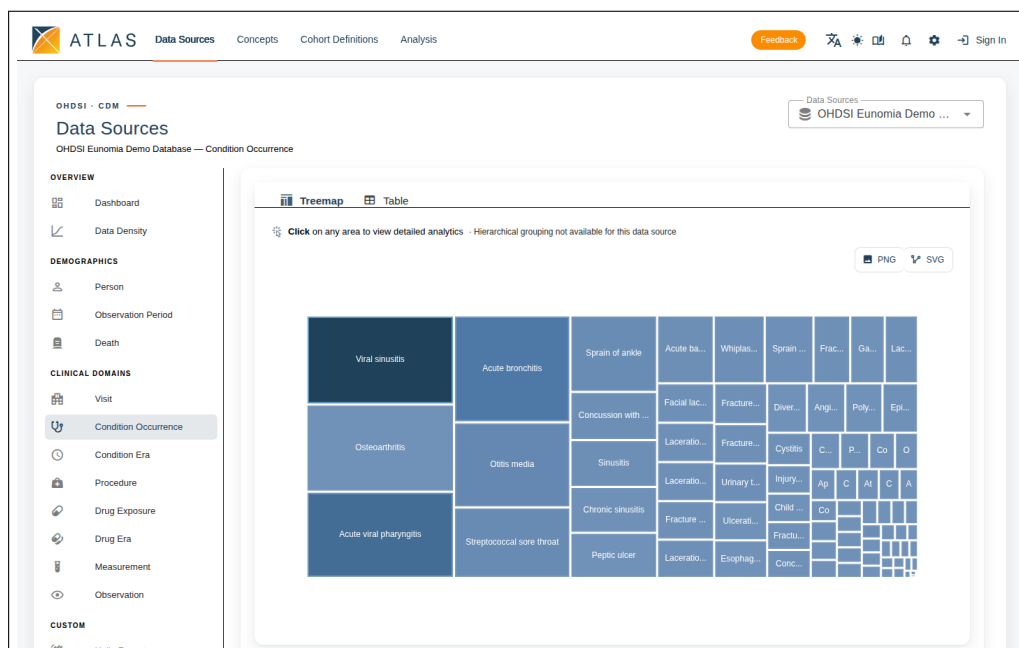


Figure 4.1. Data Sources page. The source-picker dropdown sits in the top-right; the left rail lists report types grouped by Overview, Demographics and Clinical Domains; the main panel renders the selected report.

The dropdown is populated from the WebAPI `/source/sources` endpoint; a source you cannot see may exist but be hidden from your role. Until a source is picked the sidebar is disabled and the main panel shows a "select a data source" hint.

Note

The "source key" — the short text identifier such as CCAE or SYNPUF1K — appears throughout the URL bar and the JSON of generated cohorts. When two analysts compare results, agreeing on the source key is the unambiguous way to know they are on the same database.

Descriptive reports on a source require that the OHDSI Achilles [9] routine has run against that source's results schema. A source that is configured in WebAPI but has not been Achilles-processed shows up in the list but renders empty report tabs. If your data source page looks bare on every tab, ask your administrator whether Achilles has been run — this is the single most common "set up but nothing shows" cause.

4.2 Reading a source report

Each source exposes the same family of descriptive reports, picked from the left sidebar. The thirteen built-in entries are grouped as:

Overview Dashboard, Data Density.

Demographics Person, Observation Period, Death.

Clinical Domains Visit, Condition Occurrence, Condition Era, Procedure, Drug Exposure, Drug Era, Measurement, Observation.

A report that has no Achilles results behind it on the chosen source still opens, but renders empty.

Plugins can add their own report entries to the sidebar (Chapter 16); they appear below the built-in groups under a heading chosen by the plugin (**Plugins** by default) and receive the selected source.

4.2.1 Dashboard

The default tab. It summarises the source at a glance: total persons, gender distribution, year-of-birth distribution, observation period density, and a cumulative observation chart. Use it as a sanity check. A source with millions of persons but only a couple of years of follow-up makes a different kind of study possible than a source with hundreds of thousands of persons followed for two decades.

Large counts throughout the dashboard charts — in both tooltips and axis labels — are rendered in compact SI form: 1,000 shows as 1.0k, 3,000,000 as 3.0M, scaling up to T, so a multi-million-person source stays readable at a glance. The cumulative observation curve is a percent-of-persons metric and scales to a full 100% on its y-axis. The age and year-of-birth histograms render cleanly with the same compact formatting.

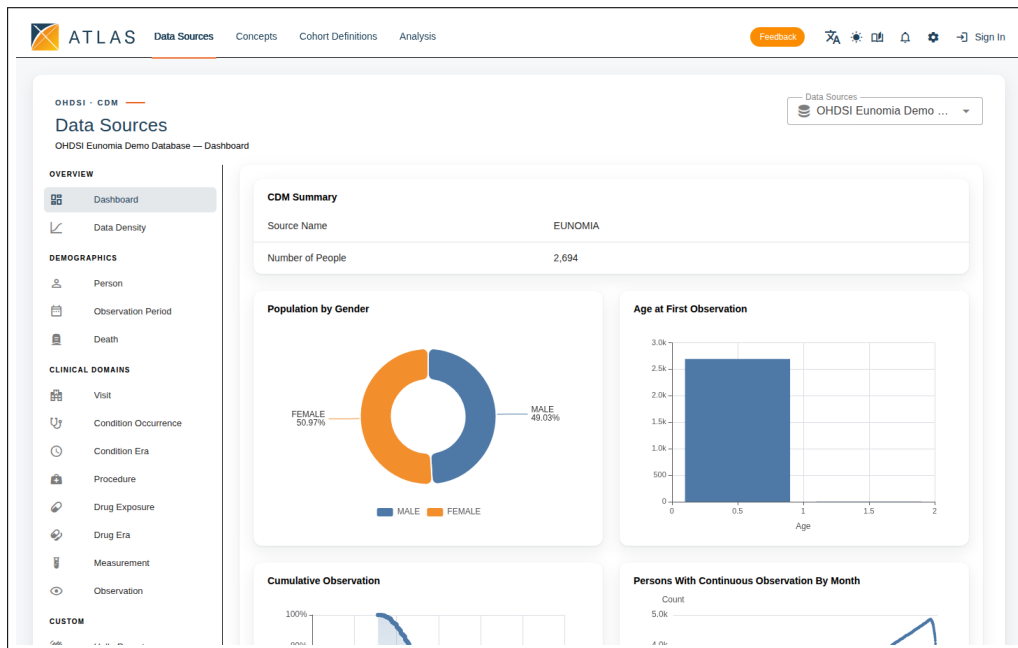


Figure 4.2. Source dashboard: CDM summary, gender split, age at first observation, and the cumulative observation curve.

4.2.2 Data Density

A breakdown of how many records exist per domain (condition, drug exposure, procedure, measurement, observation, visit, death) and how that volume changes over calendar time. Sparse rows here usually indicate either a data quality issue or that the source genuinely does not capture that domain.

4.2.3 Person

Demographics in more detail: age at first observation, race, ethnicity, where these are recorded.

4.2.4 Visit

Visit type and visit length distributions. A claims database is dominated by outpatient visits; an inpatient EHR will look very different here.

4.2.5 Condition / Drug / Procedure / Measurement / Observation

One tab per clinical domain, each showing the most prevalent concepts in that domain along with a per-concept drilldown. Scrolling these tabs is the fastest way to learn the "shape" of a source — for example, whether oncology is well represented, whether labs are coded with LOINC, whether inpatient procedures dominate over outpatient.

The default view is a *treemap*: a rectangle filled with nested boxes whose area is proportional to record count, coloured by parent vocabulary class. Hover any tile for the concept name and exact count. A toggle next to the treemap switches to a sortable, searchable table view with a concept-name filter — useful when you know the term you are looking for and want to find it directly rather than spotting it visually. The filter

matches every word you type, in any order, so `atrial fib` finds *Atrial fibrillation*; the table shows each concept's own name rather than its full hierarchy path.

Click any tile in the treemap or any row in the table to drill into the same per-concept report: prevalence by calendar month (how the concept's frequency moves over time), age and gender breakdown, and the distribution of age at first occurrence. The drill-down is the right view when you suspect a concept's counts changed because of a coding-system rollover rather than a true epidemiological shift.

4.2.6 Death

Counts of recorded deaths and their cause-of-death distribution where available. Many ambulatory data sources do not capture death well; that is something to know before designing a survival study.

4.3 Comparing sources

When you have access to several sources you can run the same analysis across all of them; the comparison is what gives observational evidence its strength. From the source list, your usual workflow is:

1. open the dashboard for each candidate source;
2. note the rough size and time span;
3. open the domain tab most relevant to your study (**Drug** for a drug exposure cohort, **Condition** for a disease cohort);
4. check that the concepts you plan to use actually appear with non-trivial counts.

If the prevalence of an entry concept is zero on a source, no cohort generation will ever produce non-empty rows there — even if the SQL runs without error. Catch this in the source report rather than after a long generation.

4.4 Data quality

ATLAS v3.0 itself is not a data-quality tool, but the same Achilles results that drive its descriptive reports also flag the most common data-density anomalies: implausible dates, sudden coding-rate changes, records outside observation periods, and other signals that something upstream of the CDM has shifted. Watch for sharp discontinuities on the temporal trend charts; they usually indicate an ETL change rather than a true epidemiological event.

For a deeper, systematic view, OHDSI ships the *Data Quality Dashboard* (DQD) [14] as a sister tool to ATLAS. DQD runs over 1,500 checks organised by the Kahn data-quality framework [7] (Conformance, Completeness, Plausibility) and produces a per-source dashboard. DQD is not part of ATLAS v3.0; it runs in R against the same CDM and writes its output where your administrator hosts it.

4.5 Patient cache and cohort reports on a source

Some descriptive reports and per-source cohort reports rely on a TrexSQL patient cache being built in advance. Cache management is not exposed on the source page itself; administrators trigger and monitor cache rebuilds from the configuration panel's **Cache** tab (Chapter 15). If a report depends on a missing cache, the report panel itself surfaces the appropriate hint.

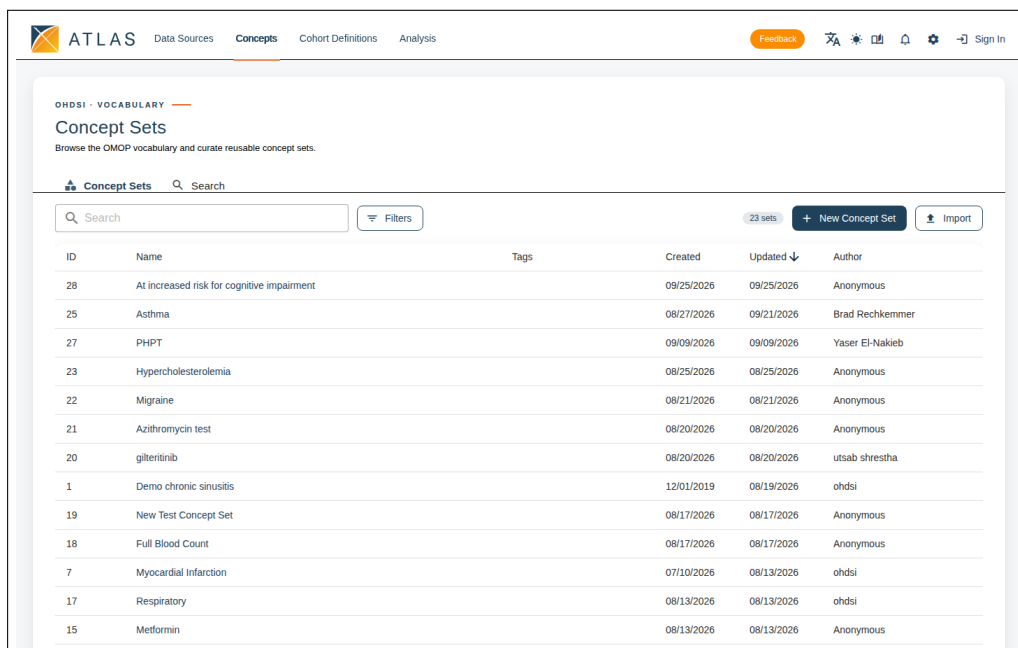
The same source detail page is reused later when you open a generated cohort and ask for its reports — see Chapter 7.

Vocabulary search

Every clinical idea expressed in ATLAS v3.0 — a disease, a drug, a procedure — is represented by one or more *concepts* drawn from the OHDSI standardised vocabularies. Vocabulary search is the way you find those concepts. It is also the entry point to building concept sets (Chapter 6).

5.1 Where vocabulary search lives

ATLAS v3.0 surfaces vocabulary search on the **Concepts** page, which has two tabs: **Concept Sets** (the default — a list of saved concept sets) and **Search**. The Search tab is the standalone search experience; the same search is also embedded inline inside the concept-set editor and wherever else a workflow asks you to pick a concept (for example, when you set an entry event in the cohort builder).



ID	Name	Tags	Created	Updated ↓	Author
28	At increased risk for cognitive impairment		09/25/2026	09/25/2026	Anonymous
25	Asthma		08/27/2026	09/21/2026	Brad Rechkemmer
27	PHPT		09/09/2026	09/09/2026	Yaser El-Nakieb
23	Hypercholesterolemia		08/25/2026	08/25/2026	Anonymous
22	Migraine		08/21/2026	08/21/2026	Anonymous
21	Azithromycin test		08/20/2026	08/20/2026	Anonymous
20	gilteritinib		08/20/2026	08/20/2026	utsab shrestha
1	Demo chronic sinusitis		12/01/2019	08/19/2026	ohdsi
19	New Test Concept Set		08/17/2026	08/17/2026	Anonymous
18	Full Blood Count		08/17/2026	08/17/2026	Anonymous
7	Myocardial Infarction		07/10/2026	08/13/2026	ohdsi
17	Respiratory		08/13/2026	08/13/2026	ohdsi
15	Metformin		08/13/2026	08/13/2026	Anonymous

Figure 5.1. Concepts page on the **Concept Sets** tab: the saved-set list with its search box, **Filters**, **New Concept Set** and **Import** buttons, and the Id / Name / Tags / Created / Updated / Author columns (all but **Tags** sortable). The sibling **Search** tab hosts the standalone vocabulary search.

Note

Links from ATLAS 2.x keep working: `#/search/⟨query⟩` opens this tab and runs the query (when it has three or more characters), and `#/concept/⟨id⟩` opens the concept on the current vocabulary source (Section 2.7).

5.2 Running a search

Switch to the **Search** tab. Type a free-text query — a name, a synonym, a code — of at least three characters into the search hero and press **Enter**. The results table fills with matching concepts.

5.2.1 What matches

The search runs against concept names, synonyms, and source codes (ICD-9, ICD-10, SNOMED, RxNorm, LOINC, NDC, and so on, depending on which vocabularies your WebAPI has loaded). A query like `atrial fibrillation` matches the standard SNOMED concept and its descendants; a query like `I48` matches the ICD-10 code and the standard concept it maps to.

Note

The OHDSI vocabulary lifecycle — standard vs. non-standard concepts, what each **Invalid Reason** flag (D, U, R) means, the **Maps to** and **Maps to value** relationships, the role of vocabularies like SNOMED, ICD-10, RxNorm and LOINC — is documented in *The Book of OHDSI*, chapter 5 "Standardized Vocabularies" [20]. This chapter only covers what the ATLAS UI shows.

5.2.2 Reading the result table

The result table has these columns: a selection checkbox, a per-row **Add** button, **Concept Id**, **Concept Code**, **Concept Name**, **Vocabulary**, **Type** (the standard-concept flag), **Domain**, **Class**, **Validity**, and four count columns — **Record Count**, **Descendant Record Count**, **Person Count** and **Descendant Person Count** (elsewhere abbreviated RC, DRC, PC and DPC). All columns are sortable, and sorting applies across every page of results, not just the visible one.

To narrow a noisy search, two controls sit above the table:

Filter results a free-text box that filters the loaded results by id, name, code, domain, vocabulary or class. Like every search box in ATLAS it matches all the words you type, in any order (Section 2.4).

Filters a menu button whose badge counts the active filters (including a non-empty **Filter results** box). It opens multi-select facets — Vocabulary, Domain, Standard, Class and Validity — applied client-side to the results already loaded. Each active facet value appears as a removable chip, and **Clear All Selections** drops them in one click.

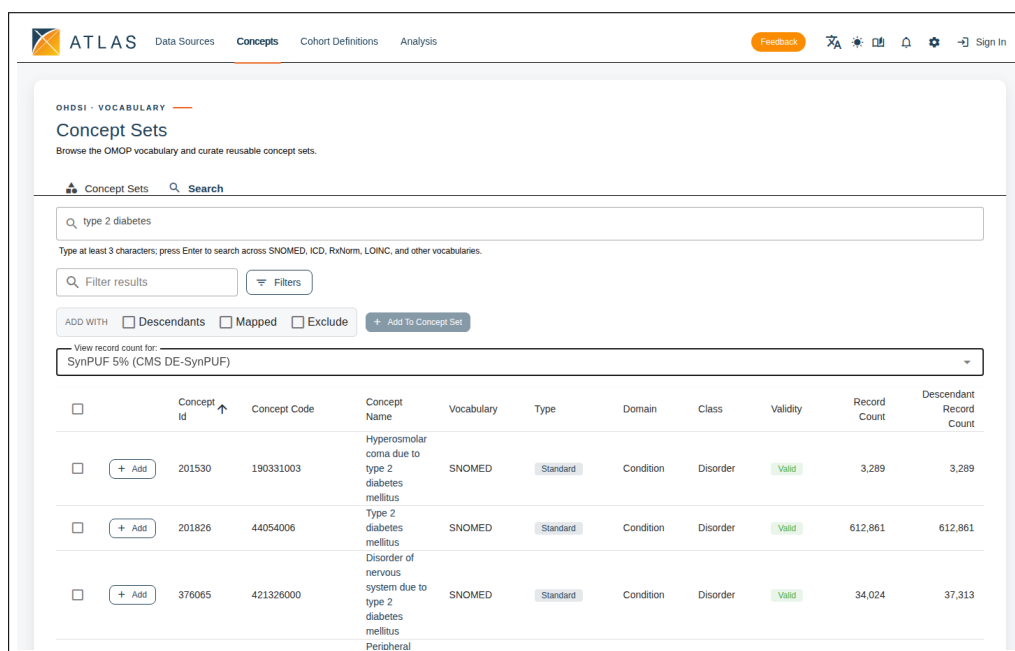


Figure 5.2. The **Search** tab after a query. Under the search box sit the **Filter results** box and the **Filters** menu, the **Add with** strip (**Descendants** / **Mapped** / **Exclude** plus **Add To Concept Set**) and the **View record count for:** source selector; the table has a checkbox and an **Add** button on every row.

Combine the filters with column sorting — often on **Descendant Record Count** to surface the most-used concepts in the source — to home in on the concept you want.

The count columns come from the source picked in the **View record count for:** selector above the table. It lists every source with a Results daimon and defaults to the current vocabulary source; changing it refreshes the counts in place without re-running the search. If the counts stay at zero, the chosen source has no Achilles results for those concepts — pick another source. A load failure shows "Failed to load record counts for the selected source."

The concept name in every row is a real link to the concept's full page (`/concept/<source>/<id>`), so middle-click and "copy link" work. Inside the concept-set editor the same click opens the details over the editor's tabs, with a back arrow to return — on the **Selected**, **Included** and **Source Codes** tabs alike. The characterization prevalence tables open them in a slide-in drawer.

5.3 Concept details

Concept Details is available as a full-page route at `/concept/<source>/<id>` — where the name links on the **Search** tab go, and what you copy-paste when you want to share a concept with a colleague — and, depending on where you clicked, as a slide-in drawer or an in-editor panel with the same content.

5.3.1 Header

The sticky header carries the concept name plus chips for the standard-concept flag (**Standard**, **Classification**, or **Non-standard**), vocabulary, concept ID, domain and concept class. Two action buttons sit on the right:

Add to Concept Set adds the concept to the currently-edited concept set, the same operation as the per-row **Add** action in the search table. It is always enabled: if no concept set is open, ATLAS auto-creates a new untitled set, opens the editor, and shows a snackbar reading "Added "..." → New Concept Set".

Copy ID writes the concept ID to the clipboard.

5.3.2 Stat cards

Below the header, one stat card per configured data source shows record count, distinct person count, and descendant record count (formatted with K / M suffixes). This is the cross-source view the search-result table cannot give you: the same concept can be common in one source and absent in another.

5.3.3 Concept attributes

The **Concept Details** card lists the full attribute set — concept ID, code, domain, vocabulary, concept class, standard concept flag, invalid reason. This is the canonical place to read off a concept's metadata; the attributes that fit in the search-results table are a subset of this list.

5.3.4 Hierarchy mini-map

The hierarchy card shows up to three parents and six children of the concept, drawn as a tree, with a "... *N* more descendants" line when the list is truncated. **View full hierarchy** → opens the full hierarchy dialog (Section 5.3.5).

5.3.5 The full hierarchy dialog

The dialog is titled **Hierarchy** · ⟨concept⟩ and opens with a count line such as "8 ancestors · 18 descendants". Its toolbar has:

- a **Filter by name or code...** box;
- **Class:** any, **Domain:** any and **Vocabulary:** any selects;
- a **Tree / Flat** toggle. **Tree** nests descendants under their parents (click the triangle to expand a node); **Flat** lists every descendant in one table. While the view switches, a "Rebuilding the list..." progress indicator is shown.

The table columns are **Concept Name**, **Code**, **Class**, **Domain**, **Vocabulary**, **RC** and **DRC**. An **Ancestors** block above the concept lists its parents with a "distance *N*" label; **Show *N* more ancestors** expands it to the full ancestry and **Show direct parents only** collapses it again.

When a concept set is open in the editor, the dialog doubles as a bulk editor: each row gets a checkbox, the **Add with** strip appears (Section 5.4), and rows already in the set carry **in set**, **excluded** or **+desc** badges. This is the quickest way to exclude a handful of unwanted descendants: tick them, tick **Exclude**, and add. Concepts already in the set with the same options are skipped, and the snackbar says so ("Added N concepts, skipped M already in the set").

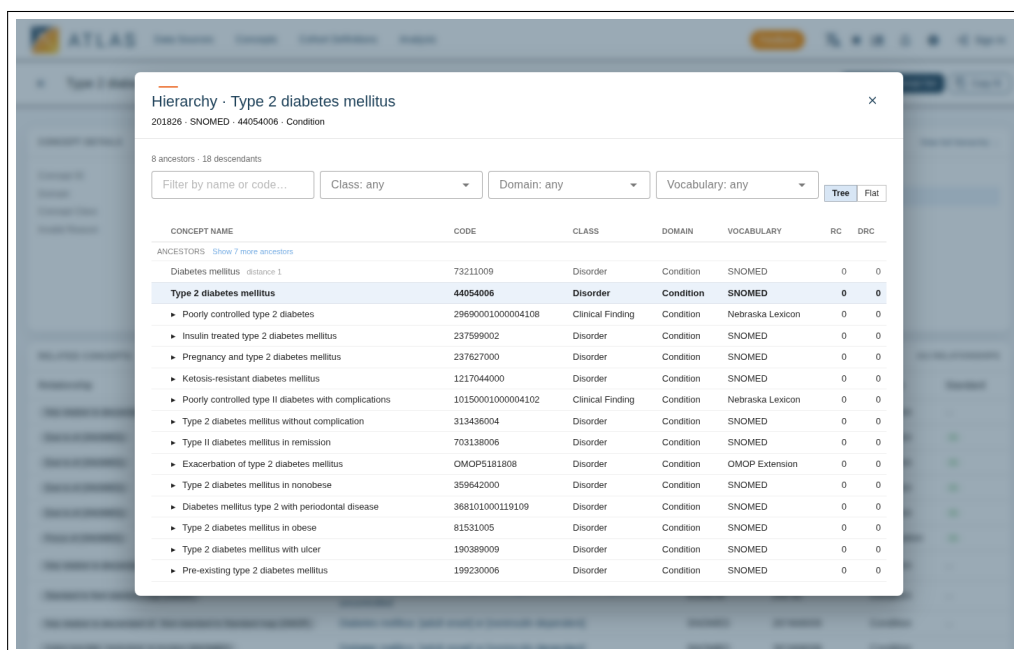


Figure 5.3. The full hierarchy dialog for *Type 2 diabetes mellitus* in **Tree** mode: filter box, Class / Domain / Vocabulary selects, the **Ancestors** block with **Show 7 more ancestors**, the concept itself highlighted, and its expandable children with RC / DRC counts.

Non-standard concepts have no hierarchy; the dialog says "No hierarchy found for non-standard concepts." If a node's children fail to load, the row shows "Could not load children" with a **Retry** action.

5.3.6 Related concepts

The **Related concepts** table lists relationship rows (*Maps to*, *Subsumes*, *Has answer*, etc.) with the related concept's ID, name, vocabulary and standard flag. Clicking a related concept name opens its details in the same drawer, replacing the current concept; the back-arrow returns to the previous one. This is how you walk a relationship graph without leaving the panel.

5.3.7 Drilldown report

The **Drilldown Report** card shows the concept's prevalence on a chosen data source. A source dropdown sits at the top of the card; two tabs underneath select what to plot:

Age at first occurrence a boxplot of the age distribution at the patient's first record of the concept.

Calendar month a line chart of prevalence per 1000 people by calendar month.

The card hides itself for concepts in domains with no drilldown data (e.g. concept-class concepts that never appear as records).

5.4 From search to set

The flags that decide how a concept enters a concept set — **Descendants**, **Mapped** and **Exclude** — are chosen *before* you add, in the **Add with** strip above the table:

- Tick the flags you want in the strip.
- Either click the per-row **Add** button (it flips to **Remove** once the concept is in the set), or tick several rows — the header checkbox selects every concept — and click **Add To Concept Set** to add them all in one go.

Both paths use the strip's flags. A snackbar confirms each add ("Added "..." → <set>" or "Added *N* concepts → <set>"). Adding is always enabled: when no concept set is open, ATLAS creates a new untitled set and opens the editor. Flags can still be changed per row afterwards in the editor's **Selected** tab (Chapter 6).

Note

The exact set of vocabularies you can search is whatever your WebAPI was loaded with from *Athena* [10], the OHDSI vocabulary repository. Two analysts on the same ATLAS v3.0 release but with different vocabulary snapshots can see slightly different concept IDs; record the snapshot date along with your study.

5.5 Search strategies that work in practice

- Start broad, then narrow. Search for the disease by name, pick the highest-level standard concept that still represents what you mean, and rely on *include descendants* rather than listing every variant by hand.
- Cross-check against codes. If you know the ICD-10 chapter for your phenotype, search by code as well; the standard concept you arrive at should be the same.
- Use the source-specific record counts. Point **View record count for:** at your target source: a concept with zero records there is unlikely to drive cohort entry, regardless of how well it represents the disease in the abstract.

Concept sets

A *concept set* is a named, reusable collection of concepts together with rules for how each concept participates. Concept sets are the building blocks of cohort definitions and most analyses; if you find yourself listing the same concepts in two cohorts, you should probably be referencing the same concept set in both.

6.1 Why a list of concepts is not enough

Two ideas separate a concept set from a flat list:

Descendants A standard concept usually has descendants in the OHDSI hierarchy. *Type 2 diabetes mellitus* (SNOMED) has many specific subtypes underneath it. With **Include descendants** ticked on the parent, every subtype is included automatically and stays current as the vocabulary evolves.

Mapped source codes Real patient records are often coded in source vocabularies (ICD-10, RxNorm, NDC). With **Include mapped** ticked, the set captures records that map up to the chosen standard concept even if they were originally recorded under a source code.

A concept set therefore expands at runtime into a — usually larger — list of concept IDs that the cohort SQL actually uses.

6.2 Creating a concept set

1. Open **Concepts** from the top navigation. The **Concept Sets** tab is the default; you land directly on the list.
2. Click **New Concept Set**. The editor opens as a drawer covering almost the whole window, with a vertical **Concept set editor** rail on its left edge.
3. Give the set a descriptive name in the inline-edit title at the top of the drawer — research groups frequently adopt a convention such as [Phenotype] [author] [version].
4. A new set opens on the **Search** tab. Use the vocabulary search (Chapter 5) to find your starting concepts; tick the flags you want in the **Add with** strip and click **Add** on each row, or tick several rows and click **Add To Concept Set** (Section 5.4).
5. When the first concept lands in the empty set, the editor switches to the **Selected** tab so you can see the result; later adds keep you on **Search**. Adjust per-row flags on **Selected**.

To start from a file instead, click **Import** next to **New Concept Set** on the list page and pick a concept-set JSON file. ATLAS creates a new set — named after the file's name field, else the file name, else "Imported concept set" — and opens it in the editor. (The editor's own **Import JSON** button, below, instead merges into the set that is already open.) A file that is not JSON or not a concept-set expression is refused with an error message.

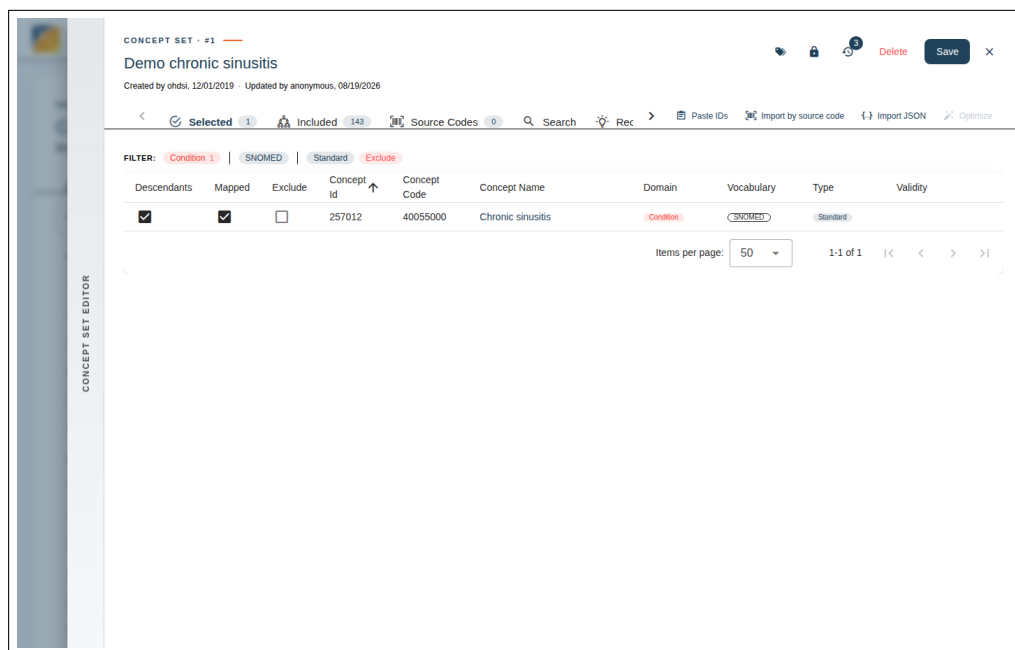


Figure 6.1. The concept-set editor drawer on the **Selected** tab. The header carries the eyebrow "Concept Set · #1", the name, the *Created by ... · Updated by ...* line, and the **Tags**, **Configure** access (lock), **Versions**, **Delete** and **Save** controls; below are the six tabs, the **Paste IDs** / **Import by source code** / **Import JSON** / **Optimize** buttons, the facet filter strip and the per-row Descendants / Mapped / Exclude flags.

6.3 The editor tabs

The editor drawer has six tabs, in order: **Selected**, **Included**, **Source Codes**, **Search**, **Recommend**, and **Compare**; the first three show a count chip in the tab header. A new set opens on the **Search** tab so you can start adding concepts immediately; an existing set opens on **Selected**, where its concepts and per-row flags are shown.

The **Search** tab is the vocabulary search of Chapter 5 — **Filter results** box, **Filters** menu, row checkboxes and the **Add with** strip — minus the **View record count for:** selector, which only the standalone search page has.

6.4 The Selected tab

The **Selected** tab lists every concept you have explicitly added to the set, with three per-row toggles and a remove action:

Descendants Whether the set also covers concepts below this one in the OHDSI hierarchy.

Mapped Whether non-standard source codes that map up to this concept count as members.

Exclude When on, members covered by this row are subtracted from the set, even if another row would otherwise include them.

Membership itself is implicit: every row in the table is part of the set unless its **Exclude** flag is on. To take a concept out of the set entirely, use the row's remove action rather than an Include toggle.

These toggles compose. A common shape is "include the parent disease with descendants, then add an exclude row for one specific subtype".

The same concept may appear on more than one row as long as the rows carry different flags. For example, concept X with **Descendants** plus concept X with **Exclude** reads "X's descendants, but not X itself". Remove and the flag toggles act on the individual row. ATLAS refuses a true duplicate ("Concept already exists in this set with the same options") and a flag change that would make two rows identical ("Another entry for this concept already uses those options"). The **Selected** tab also has a filter strip at the top of the table to narrow the displayed rows by domain, vocabulary, standard-concept flag, or exclusion status.

The concept-name cell on each row is a link to **Concept Details** (Chapter 5, Section 5.3); use it to inspect attributes, the hierarchy mini-map, and per-source prevalence without leaving the editor.

6.4.1 Concepts the vocabulary does not contain

A set can reference concept IDs that the current vocabulary source does not have — typically a set written against a newer or different vocabulary release. ATLAS keeps such a set usable instead of failing to open it. The missing rows stay in the set, but their name cell reads *Not in the selected vocabulary* (with a warning icon) and their other columns are blank, and a warning above the table, **Some concepts are not in the selected vocabulary**, names the source and lists the missing IDs. The IDs are kept when you save; to see their details, pick a vocabulary source that contains them in the configuration panel (Chapter 15). While any concept is missing, **Optimize** is disabled (Section 6.7.4).

6.5 The Included tab

The **Selected** tab shows what you *added*; the **Included** tab shows what the set actually *resolves to* — the parents you selected plus every descendant and mapped concept the toggles pull in, minus any excluded rows. The columns are Concept Id, Concept Code, Concept Name, Domain, Vocabulary, Type and Validity (no record counts; use the **Search** results, the **Concept Details** stat cards or the hierarchy dialog for those). When the set is empty the table shows an empty state reading "Add concepts on the Selected tab to see them here".

The resolved list can be long, so the tab has the same **Filter results** box and **Filters** menu as the search table, with Vocabulary, Domain, Standard and Validity facets, a working pager and multi-column sorting. When the filters leave nothing, the table says "No concepts match the active filters." with a **Clear filters** action.

Rows here are selectable too. Tick the concepts you do not want, and use the **Add with** strip — where **Exclude** is ticked by default on this tab — to add them back to the expression as exclude rows. This is the fastest way to prune a descendant subtree that swept in more than you intended.

This is the list the cohort SQL ultimately uses, so it is the right place to sanity-check the set before you point a cohort at it.

6.6 The Source Codes tab

The **Source Codes** tab lists the non-standard *source* codes that map to the set's included concepts — the ICD-10, Read, NDC or other source-vocabulary codes a record might carry before it is mapped up to a standard concept. The table has columns Concept Id, Concept Code, Concept Name, Class, Domain, and Vocabulary, with a **Filter results** box and Class / Domain / Vocabulary facets. Reviewing it is the practical way to see which real-world codes the set will actually match in an ETL'd database. As on **Included**, rows can be ticked and added to the expression through the **Add with** strip.

6.7 Bulk-adding concepts

The right side of the editor's tab strip carries three import buttons — **Paste IDs**, **Import by source code**, and **Import JSON** — for adding many concepts at once instead of one search result at a time, followed by **Optimize**, which tidies the rows you already have (Section 6.7.4).

6.7.1 Paste IDs

Paste IDs opens the **Paste concept IDs** dialog where you paste a list of concept IDs (one per line, or space- or comma-separated); **Resolve** looks them up in the vocabulary and reports a matched / unmatched summary before you confirm the add. This is the fastest way to import a code list from a spreadsheet or a phenotype paper.

6.7.2 Import by source code

Import by source code accepts *source* codes rather than concept IDs — for example ICD-10 E11.9. Click **Resolve**: ATLAS looks each code up, lists what it found under *Resolved*: and what it did not under *Not found*:, and **Add *N*** adds the resolved concepts to the set. If you correct a bad entry and resolve again, the **Add** button re-enables. Use it when your starting list is expressed in a source vocabulary rather than in OMOP concept IDs.

6.7.3 Import JSON

Import JSON opens the **Import concept set JSON** dialog. It accepts a concept-set expression of the form `{"items": [...]}` — the same shape ATLAS uses to serialise a set. **Parse** checks the text, then **Add** *N* merges the items into the open set, each together with its per-row flags (descendants, mapped, exclude), so this round-trips a set exported from elsewhere while preserving how each concept participates.

6.7.4 Optimize

Optimize, the last button in the same row, asks WebAPI for the smallest expression that resolves to exactly the same concepts in the current vocabulary — for example, it drops a row that another row's **Descendants** flag already covers. It opens the **Concept Set Optimization** dialog, which first shows *"Attempting to find an optimal definition for this concept set..."* and then one of three results:

Already optimal. *"The current concept set definition is fully optimized."* Nothing to do; close the dialog.

A shorter definition. Two tables: **Optimized Concept Set** (the rows that remain, with their descendants and exclude flags) and **Concepts Removed** (*"removed since the optimized version already includes them"*), each with a count chip.

An error. The WebAPI message, with **Retry**.

With a shorter definition you choose how to keep it:

- **Overwrite Current Concept Set** replaces the open set's rows with the optimized ones and switches to **Selected**. Nothing is written until you **Save**.
- **Create New Concept Set** asks for a name (pre-filled as *"⟨name⟩ - OPTIMIZED"*) and **Save** creates it as a separate library set right away; the set you opened is not changed. This option needs permission to create concept sets and is not offered in the embedded editor (Section 6.13).

Optimize is disabled, with the reason on hover, when the set has fewer than two concepts (*"Add at least two concepts to optimize the concept set."*), when some of its concepts are missing from the vocabulary (*"Some concepts are not in the selected vocabulary, so the concept set cannot be optimized."*), and while you preview an older version. The optimizer does not carry the **Mapped** flag over: mapped concepts it resolved come back as explicit rows, so check **Included** before you save.

A concept set that resolves to thousands of included concepts is rarely what you wanted. Either narrow the parent concept, turn off descendants, or add specific exclusions until the resolved list reads sensibly.

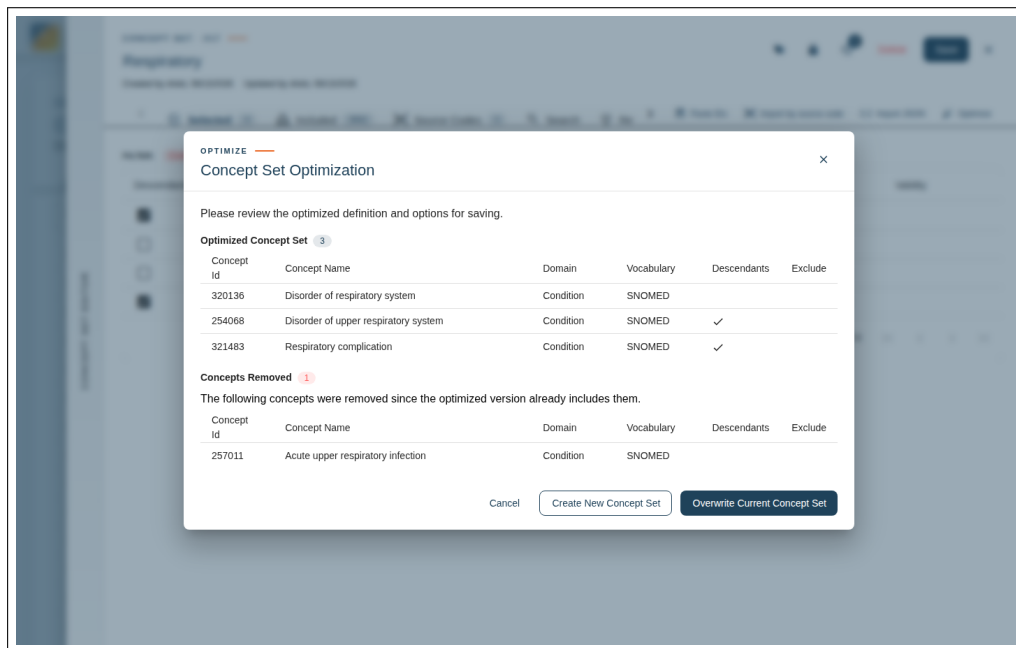


Figure 6.2. Concept Set Optimization for a four-concept set: the **Optimized Concept Set** keeps three rows (two now with **Descendants**), **Concepts Removed** lists the one row the optimized version already covers, and the footer offers **Cancel**, **Create New Concept Set** and **Overwrite Current Concept Set**.

Before saving a concept set you intend to publish, sit with a domain-aware reviewer in front of the **Selected** and **Included** tabs, with the record counts from the hierarchy dialog (Section 5.3.5) to hand. This is the catch step for two common mistakes: (i) a standard concept whose mapped source codes are clinically broader than your phenotype intends (the up-hill mapping problem), and (ii) a descendant subtree that sweeps in obsolete or rarely-used codes a reviewer would have flagged. The complementary check — *orphan codes*, source codes a clinician would have included but the set does not capture — is what the HADES CohortDiagnostics package automates [28, 30].

6.8 Tags

The editor header carries a **Tags** button with a count badge. Clicking it opens a tag-selection dialog where you attach one or more tags to the set; the tags are saved together with the concept set. On the **Concept Sets** list they render as coloured chips in the **Tags** column, and they are one of the facets you can filter the list by (Section 6.15). Tags are the lightweight way to group related sets — by study, by domain, by review status — without baking that grouping into the set name.

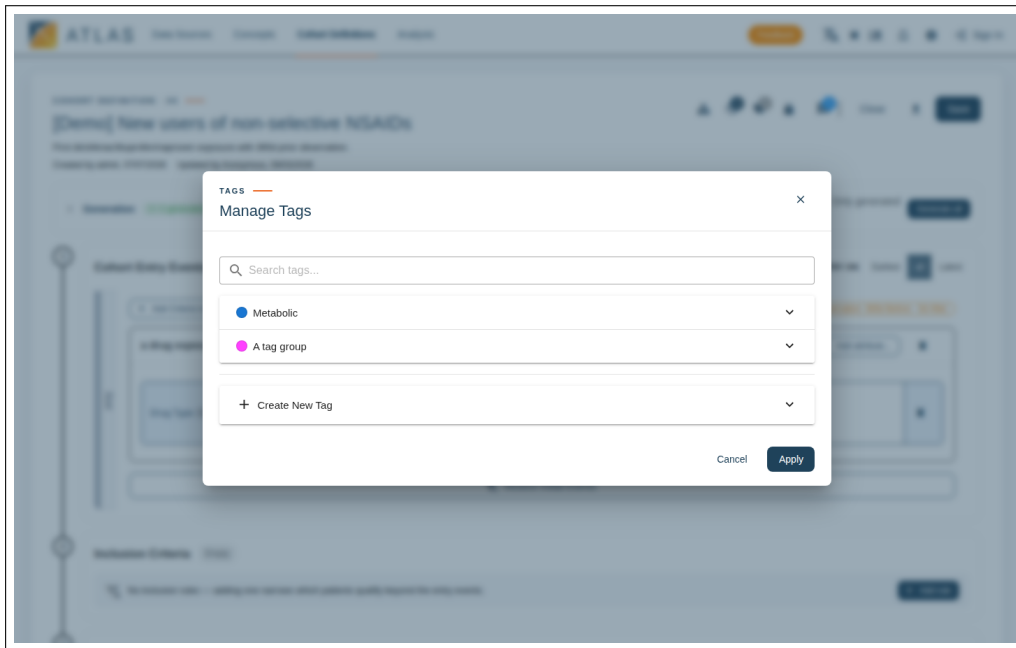


Figure 6.3. The shared **Manage Tags** dialog (shown here opened from a cohort; concept sets use the same dialog): search box, one expandable panel per tag group, and a **Create New Tag** panel, with **Cancel** and **Apply**.

6.9 Recommend

The **Recommend** tab uses PHOEBE 2.0 to suggest additional concepts that are commonly co-included with the ones you have already added. Recommendations need seed concepts to work from, so add at least one concept on the **Selected** tab before opening it; with an empty set there is nothing to recommend against. PHOEBE relies on co-occurrence patterns drawn from a large network of OMOP databases, so a concept it suggests is one that other phenotype authors have routinely paired with yours. Review each suggestion before accepting it — the recommender does not know your phenotype’s intent, only the company it tends to keep in published code lists.

The results table shows a “*N* recommendations” count and is selectable: tick the rows you want, choose flags in the **Add with** strip, and click **Add To Concept Set** to add them in one action. A **Refresh** action recomputes the recommendations after you have changed the set’s concepts. With an empty set the tab says “Add concepts to the set first — recommendations are based on what is already included.”

When the WebAPI’s vocabulary schema does not include the PHOEBE 2.0 tables, the tab shows the banner “Recommendations are not available. The PHOEBE 2.0 vocabulary tables are required to generate recommendations.” with a link, **How to install the PHOEBE 2.0 tables**, to the setup instructions on the OHDSI forums. Moving to a source that has the tables clears the banner.

6.10 Compare

The **Compare** tab compares the concept set you are editing (**Concept Set 1**) against another saved concept set (**Concept Set 2**). The workflow is:

1. Click **Choose** on the toolbar; pick the second set from the picker dialog.
2. Click **Compare Concept Sets**. ATLAS resolves both sets, computes the intersection and the two differences, and renders a Venn diagram alongside a result table.
3. A segmented toggle above the results chooses the comparison level: **Expression concepts** compares the concepts as explicitly listed in each set's expression, **Included concepts** compares the fully-resolved expansions (descendants and mapped codes included), and **Source concepts** compares the source codes each set matches. The levels can differ markedly when either set leans on **Include descendants**.
4. The result table has a **Match** column that tags every concept as *Both* when it appears in both sets, or with the *name of the set* that uniquely contains it, followed by Concept Id, Concept Code, Concept Name, Domain, Vocabulary and Class (Source Code and Source Vocabulary in **Source concepts** mode). The CSV export additionally carries Standard and Invalid Reason.
5. Narrow the rows with the **Search table...** box (all words, any order) and the **Filters** menu, which offers Match (*Both* or either set's name), Vocabulary, Domain, Standard, Class and Validity facets. The Venn diagram keeps showing the full comparison.
6. **Export** on the toolbar writes the *filtered* rows to a CSV file for review or attachment to a phenotype submission; it is disabled when the filters leave no rows.

Use the comparison to:

- review what changed between two phenotype iterations before retiring the older one — the rows tagged with the older set's name are what it captured that the new one no longer does;
- check whether two phenotypes that should be disjoint (e.g. a positive and a negative control) actually are — the Venn diagram should have an empty intersection;
- migrate from a peer-curated library set to a local one without losing terms accidentally.

A worked iteration: you have set v1 published in a study, set v2 in development. Open v2, switch to **Compare**, choose v1 as **Concept Set 2**. The rows tagged with v1's name are the terms v2 dropped — defend each one in the change log. The rows tagged with v2's name are the new terms in v2 — justify each. The *Both*

rows are the unchanged backbone. Do this review before you point any cohort at v2.

6.11 Saving

Save (or **Create** on a new set) writes the current concept set to WebAPI. The first save creates a permanent ID; later saves create new versions (Chapter 14). When **Save** is disabled, hover it: the tooltip gives the reason (no permission, no write access to someone else's set, no name, or you are previewing an earlier version; Section 2.4). **Delete** removes the set. The rest of the header:

- a *Created by ... · Updated by ...* line under the name, showing who created and last changed the set;
- **Tags** (see above);
- **Configure access** (lock icon), available once the set has been saved, to grant other roles read or write access (Section 15.4);
- **Versions**, which opens the version history (Chapter 14);
- the close button. Closing the drawer — or clicking outside it — with unsaved changes opens an **Unsaved changes** dialog with **Cancel** and **Discard changes**.

Include Mapped pulls in records whose *source* code maps to your standard concept, even when the mapping is "up-hill" to a broader parent than the source code itself denotes. For precision-sensitive phenotypes, leave it off and rely on standard concepts the source database was ETL'd to use directly. The mapping mechanics are covered in *The Book of OHDSI*, chapter 5 [20].

6.12 Picking a concept set from another editor

Wherever a design needs a concept set — a cohort criterion's **Select Concept Set** button, a characterization subgroup, an incidence-rate stratify rule, a feature-analysis criterion — ATLAS opens the same centred **Select Concept Set...** dialog. It has three parts:

New Concept Set starts an empty set inside the current design and opens it in the embedded editor (next section).

In this definition lists the sets already embedded in the design you are editing, with their IDs; click one to use it.

Import from repository searches the saved concept-set library (all words, any order); picking one copies it into the design.

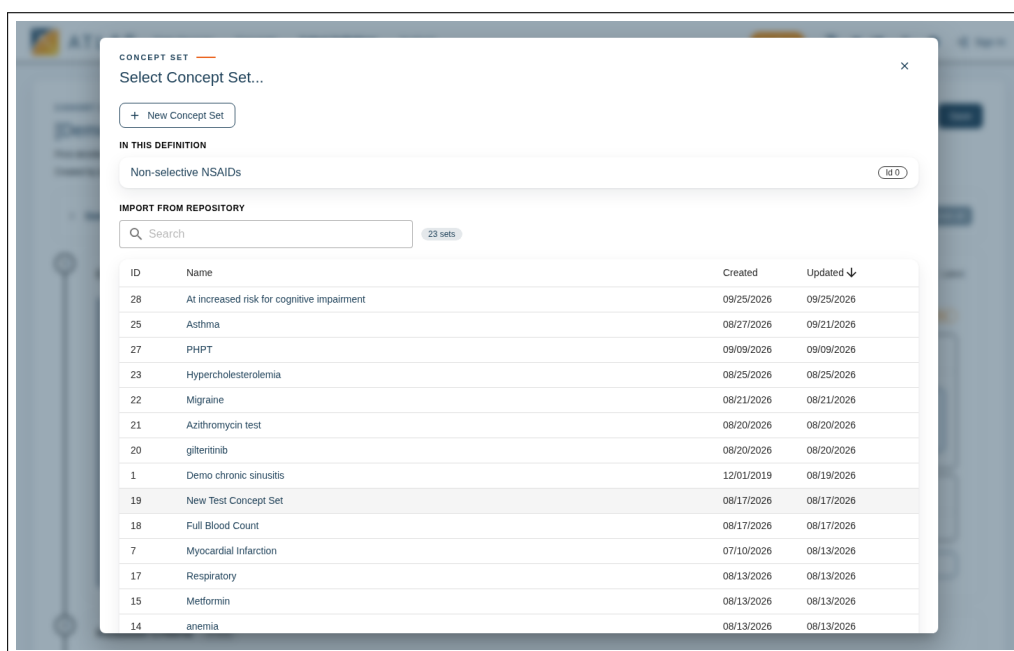


Figure 6.4. The **Select Concept Set...** dialog opened from a cohort criterion: **New Concept Set** at the top, the one set already embedded in this cohort under **In this definition**, and the searchable, sortable library under **Import from repository**.

6.13 The embedded concept-set editor

A cohort or analysis stores its own copy of every concept set it uses. When you edit one of those copies — by clicking the concept set chip on a criterion, the pencil in a design’s concept-set list, or **New Concept Set** in the picker — the editor opens as the same side drawer described above, but in *embedded mode*:

- the tabs and bulk-add buttons are the same, and **Optimize** can overwrite the copy (but not create a new library set);
- there are no **Tags**, **Configure access**, **Versions** or **Delete** controls, because the copy is not a library object;
- the header buttons are **Cancel**, **Apply** and a close (X) button. **Apply** writes the edited copy back into the design (you still need to save the cohort or analysis); **Cancel** or X leaves without applying, asking first in the **Unsaved changes** dialog if you edited the copy.

Editing the embedded copy never changes the library set it came from, and vice versa.

6.14 Starting from a phenotype library

Before authoring a concept set from scratch, check whether the phenotype already exists in a community library. Several peer-curated libraries can be searched first:

- *The OHDSI Phenotype Library* [17] holds peer-reviewable cohort definitions in OMOP-CDM ATLAS JSON. Each entry has a stable identifier and each release of the library carries its own DOI for citation.

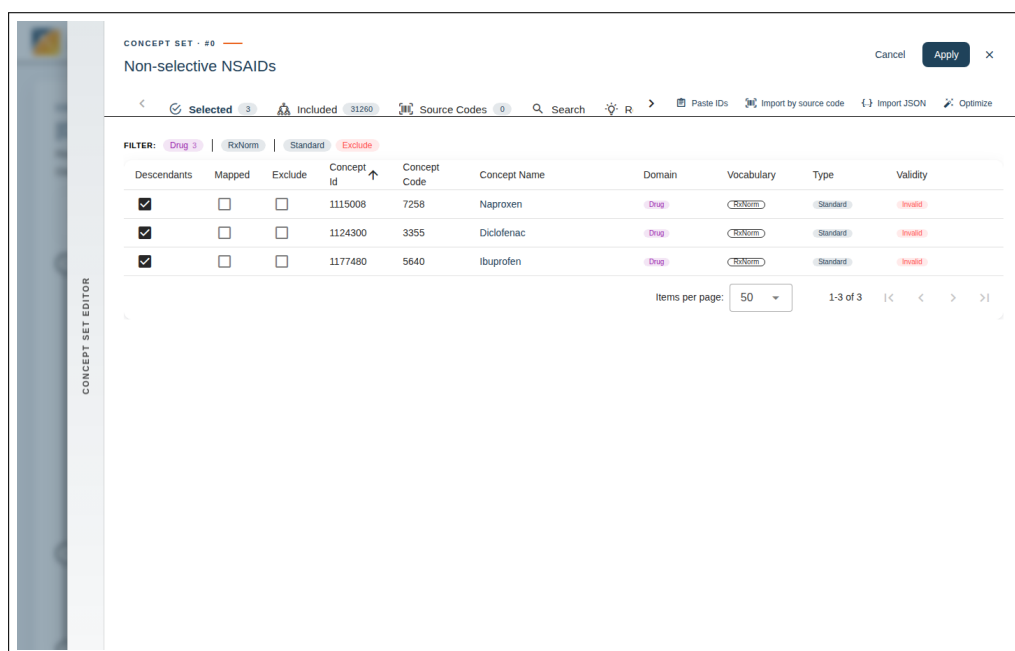


Figure 6.5. The embedded concept-set editor, opened from a cohort's concept-set list. The header shows **Cancel** and **Apply** instead of the library controls; the tabs and the bulk-add buttons are the same as in the library editor.

- *The HDR UK Phenotype Library* [3] indexes more than a thousand phenotype algorithms expressed as clinical code lists — ICD, SNOMED, Read — across cardiology, oncology, COVID-19 and other areas, contributed by groups across UK research; it is API-accessible and ships an R client.
- *EHDEN* [2] (the European Health Data & Evidence Network) curates phenotypes used in pharmacoepidemiology studies on its federated 100M-patient OMOP estate, and produces training material on phenotype definition, characterisation, and evaluation that complements the OHDSI tools.

A concept set or cohort imported from any of these is almost always cheaper to adapt than to author — and visibly more defensible at peer review — than starting with a blank expression.

When you reuse a definition from the OHDSI Phenotype Library, record both the cohort identifier and the library release version (each release has its own DOI). Cohort definitions accepted into a release are immutable for that release, but a definition can be marked deprecated [D] or in error [E] in a later one. Pinning the release insulates your study from those later changes.

6.15 Discovery

Concept sets accumulate quickly. The list page on the **Concept Sets** tab carries a free-text search at the top that matches the set name — every word you type must

appear, in any order (Section 2.4) — and next to it a **Filters** menu offering **Author**, **Tags**, and **Created / Modified** date-range facets. Each active facet shows below the bar as a removable chip, so you can stack, say, an author and a tag and see at a glance what is narrowing the list. The table columns are Id, Name, Tags (rendered as coloured chips), Created, Updated, and Author. The list opens sorted by **Updated**, newest first, so the set you touched last is on top; click any other header except Tags to re-sort (sorting spans all pages). Adopt a naming convention — e.g. [Phenotype] [author] [version] — and tag sets consistently so both the search and the facets stay useful as the library grows.

PART III

Cohort Definitions

Cohort definitions

A *cohort* is a set of persons who satisfy one or more criteria for a duration of time. A *cohort definition* is the formal, machine-executable description of those criteria. This chapter is the longest in the manual because the cohort builder is where you spend most of your time in ATLAS v3.0.

The cohort builder edits the OHDSI CIRCE cohort expression directly — what you see on screen is the same object that is saved, exported as JSON, and compiled to SQL. The page is a single column: a header with the cohort's name and toolbar, a collapsible **Generation** section for running the cohort against connected sources, and then three numbered steps matching the logical pieces of a definition — **1 Cohort Entry Events**, **2 Inclusion Criteria**, and **3 Exit & Eras** (exit strategy, censoring events, and era collapse).

Note

A cohort definition built in ATLAS is one stage of a longer phenotype-development workflow that includes diagnostics, evaluation, and submission to a community library. The methodology is documented in *The Book of OHDSI* [20] (chapters 10, 16, 18) and is out of scope for this manual; we focus on the ATLAS controls.

ATLAS's cohort model lets one person belong to the same cohort during *multiple non-overlapping time periods* — a person who initiated metformin in 2014 and again in 2019 has two cohort entries. They are never in the cohort twice simultaneously. This matters when you read reports: the "cohort entries" count is greater than or equal to the "persons" count, and the difference tells you how often re-entry happens in your data.

If you came from SAS or R, you are looking for NOT EXISTS. ATLAS does not have it. Every "absence" constraint — "no prior exposure to drug X", "no diagnosis of Y" — is expressed as an inclusion rule with cardinality *exactly 0* in the relevant temporal window. This pattern recurs throughout the chapter; the cohort builder itself has no negation operator.

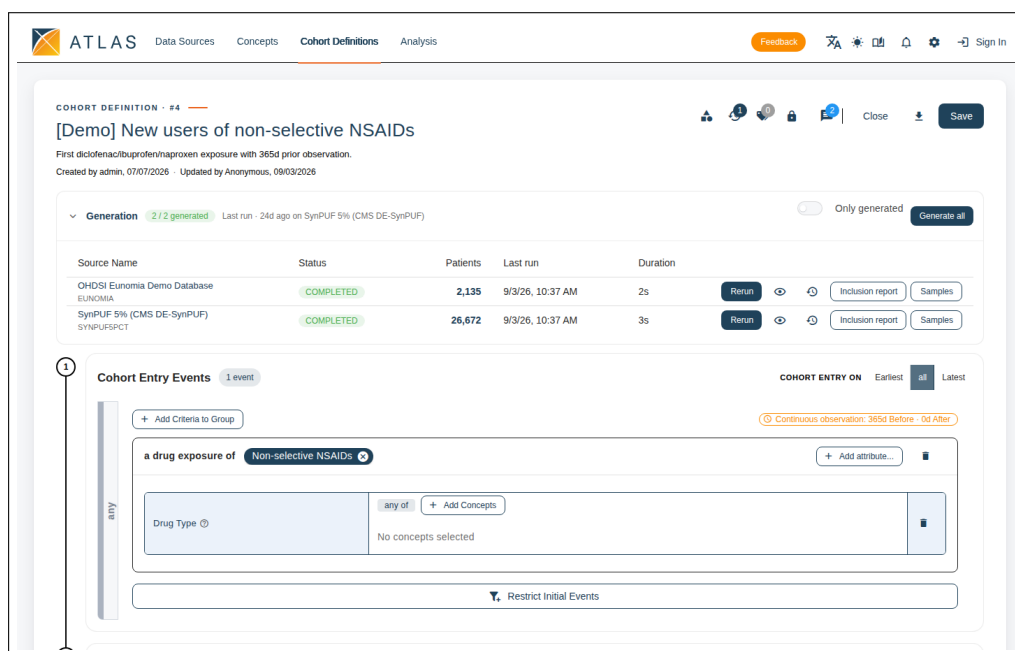


Figure 7.1. The cohort builder. The header carries the eyebrow "Cohort definition · #id", the editable name and description, and the authorship line ("Created by ... · Updated by ..."). The collapsible **Generation** section sits above the numbered steps — here expanded to show one row per data source. The toolbar icons open the cohort's concept sets, versions, tags, access settings and validation messages; **Close**, **Export** (download icon), and **Save** follow.

The editor header

The header toolbar carries, from left to right:

Concept Sets (shapes icon) — opens the cohort's **Concept Sets** dialog (Section 7.7). Its badge counts concept sets the definition no longer uses.

Versions (history icon) — the version history (Chapter 14).

Tag Management (tag icon) — Section 7.10.3.

Configure access (lock icon) — grant roles read or write access to this cohort (Section 15.4). Disabled until the cohort has been saved, and while you preview an earlier version.

Validation messages (speech-bubble icon) — the badge counts warnings from the CIRCE checker; click it to read them.

Close / Cancel — reads **Close** when the cohort has no unsaved changes and **Cancel** when it does. Leaving a dirty editor, by either button or by navigating away, raises the **Unsaved changes** dialog (**Cancel / Discard changes**).

Export (download icon) and **Save** — Section 7.10.

Under the title, an authorship line reads "Created by ⟨user⟩, ⟨date⟩ · Updated by ⟨user⟩, ⟨date⟩"; the *Updated* half appears once the cohort has been modified. The same line appears in every ATLAS editor (Section 2.4).

7.1 Live preview

When the deployment has TrexSQL enabled and a per-source patient cache built (Chapter 15, "Caches and TrexSQL"), the cohort builder shows running patient counts as you edit — no **Generate** round-trip required. There is no single "patient count" bar at the top of the builder; counts appear *inline*, one per inclusion rule, in the inclusion-rule rail described in Section 7.5.

The data source the live preview runs against is chosen from an unlabelled source selector (database icon) in the header of the inclusion-rule rail, directly under **Inclusion rules** and the **Add rule** button. Next to it a status chip reports the state of that source's patient cache:

Ready. The cache is up to date; live counts are current.

Stale. The cache exists but is older than the underlying data. Live counts still display but with a stale-tone chip; rebuild the cache for accuracy.

Building. The cache is being built. Live counts appear once the build finishes.

Error. The last cache build failed. Live counts are unavailable for this source until it is rebuilt.

Not Built. The source has no cache. Ask an administrator to build it from the configuration panel's **Cache** tab; live counts are unavailable until then.

The source picked here drives the rail's entry-event count, the per-rule counts, and the final qualifying-cohort figure.

A rule that drops the live count from 80% to 0% the moment you add it is almost always a misconfigured concept set or an inverted cardinality, not a real epidemiological signal. Treat the inline counts as your in-place sanity check: they catch mistakes that would otherwise only surface after a full **Generate**.

Note

On deployments where TrexSQL is not enabled the selector is not shown at all and the rail carries no counts. Where TrexSQL is on but the chosen source has no cache yet, the chip reads **Not Built** and counts stay empty until an administrator builds it. Proceed with **Generate** as the only feedback loop.

7.2 Listing and creating cohorts

Open **Cohort Definitions** from the navigation. The list page supports two views — tile and table — toggled in the top-right. The table has the columns **Id**, **Name**, **Tags**, **Author**, **Created**, and **Updated**; click a header (**Id**, **Name**, **Author**, **Created**, **Updated**) to sort, and the sort spans every page, not just the visible one. The default order is **Updated**, newest first, so what you were last working on is at the top. The search box

matches every word you type, in any order, against name and description — "nsaid new" finds "[Demo] New users of non-selective NSAIDs" (Section 2.4).

On the search row, right-aligned, sit a count chip ("*N* cohorts") and two buttons:

- **New Cohort** first asks for a name in the **Create a new cohort definition** dialog (**Create** stays disabled until you type one), then opens an empty definition.
- **Import** opens the **Import cohort definition** dialog: a **Name**, an **Upload JSON file** field, and an **Expression JSON** box you can paste into instead. The expression is validated before anything is saved.

Clicking a row or card opens that cohort in the editor.

Import is *strict*. ATLAS v3.0 no longer runs imported JSON through a converter; the expression must be a valid CIRCE cohort expression. A field CIRCE does not define is refused with its path and, where possible, a hint — for example *"Expression JSON is not a valid cohort expression: InclusionRules[0].title: unrecognized field (did you mean "name"?)"*. Nothing is saved until the JSON validates. Fix the offending field in the file (or in the dialog) and import again.

7.2.1 Duplicating a cohort

Every cohort card (and every row in the table view) carries a **Duplicate** action. It copies the full definition under the name "*<name> (copy)*" — and, when that name already exists, auto-increments to "*(copy 2)*", "*(copy 3)*", and so on — then opens the new copy directly in the builder. WebAPI has no server-side copy endpoint for cohorts, so ATLAS builds the duplicate client-side from the fetched definition; the copy is an independent cohort with its own ID and version history.

7.3 The four logical pieces of a definition

Every cohort definition answers four questions, in this order:

1. **Who enters the cohort, and on what date?** — entry event (also called primary criteria, or initial event).
2. **Of those entrants, who do we keep?** — inclusion criteria with cardinality and temporal logic.
3. **When does the cohort end for each person?** — exit strategy.
4. **What ends membership early?** — censoring events.

The rest of this chapter walks each in turn.

7.4 Entry event (primary criteria)

The entry event defines what kind of clinical record makes a person a candidate for the cohort, and which date of that record becomes the person's *index date*. In step 1, **Cohort Entry Events**, click **Add Criteria to Group** and pick a criterion type:

- Condition Era, Condition Occurrence
- Custom Era (Section 7.4.2)
- Death
- Device Exposure
- Dose Era, Drug Era, Drug Exposure
- Episode (Section 7.4.1)
- Measurement
- Observation, Observation Period
- Payer Plan Period
- Procedure Occurrence
- Specimen
- Visit Detail, Visit Occurrence

Demographic criteria are not offered as entry events; they are available inside inclusion-rule groups (Section 7.5.3). An empty step reads *"No primary criteria yet. Click "Add Criteria to Group" to start."* When you have several entry criteria, a vertical **any** label runs down the left edge: a person enters on *any* of them (logical OR).

Each criterion is a card whose header reads as a sentence — "a drug exposure of", "a condition occurrence of", "an episode of" — followed by:

- a **Select Concept Set** button. Once a set is chosen it becomes a closable chip showing the set's name; click the chip to edit the set, or its x to clear it. The picker is described in Section 6.12.
- an **Add attribute...** menu listing the filters available for that domain — for a condition occurrence, age, gender, condition type, condition status, visit type, **Condition Source Concept**, date adjustment, **Nested Criteria** and more; for a drug exposure, dose, quantity, days supply, refills, route, stop reason and so on. Attributes appear on the card only once you add them, each with its own delete button. Concept-typed attributes (drug type, visit type, ...) carry an **any of / not any of** toggle.
- a delete icon that removes the criterion.

Note

A criterion with no concept set matches its *entire* domain — “a drug exposure of” with no set means “any drug exposure”. That is occasionally what you want (e.g. “any visit”) but more often a forgotten step.

Figure 7.2. Step 1, Cohort Entry Events. The **Cohort Entry On** toggle on the right (**Earliest** / **all** / **Latest**) is the qualifying-events limit for entry; the amber chip (“Continuous observation: 365d Before · 0d After”) is the look-back / follow-up requirement. The criterion card reads “a drug exposure of [Non-selective NSAIDs]” with one added attribute (Drug Type), and **Restrict Initial Events** sits at the bottom of the group.

7.4.1 Episode criteria

Episode (card text “an episode of”) selects records from the OMOP CDM v5.4 EPISODE table — abstracted disease phases, treatment regimens and lines of therapy, as populated by oncology and similar ETLs. Its **Add attribute...** menu offers **First Episode**, **Age**, **Gender Concept Set**, **Start Date**, **End Date**, **Episode Number**, **Episode Object Concept Set**, **Episode Type Concept Set**, **Date Adjustment**, and **Nested Criteria**. Only sources whose ETL populates EPISODE return rows.

7.4.2 Custom era criteria

Custom Era builds eras on the fly from other criteria: the card reads “a custom era using [*N* days] gap of:”, where the chip sets the gap (persistence window) in days. Below it:

- a **Criteria List** block with **Add Criteria...**, which accepts any criterion type (a drug exposure, a procedure, ...). Records matched by those criteria are collapsed into eras whenever they are no further apart than the gap. Empty state: “*No criteria yet.*”
- an **Additional Attributes** block with **Add Attribute...: First Era**, **Date Adjustment**, **Start Date**, **End Date**, **Age at Start**, **Gender Concept Set**, **Duration**, and **Nested Criteria**. Empty state: “*No additional attributes yet.*”

Use a custom era when the pre-built DRUG_ERA / CONDITION_ERA tables do not match your persistence rule — for example, treatment episodes of an injectable with a 90-day gap, or eras spanning several ingredients defined by your own criteria.

Figure 7.3. Entry events with an Episode criterion ("an episode of", awaiting a concept set) and a Custom Era criterion ("a custom era using 0 days gap of:") with its empty **Criteria List** and **Additional Attributes** blocks.

7.4.3 Cohort entry observation window

The required continuous observation around the index date is set from the amber chip in the entry-event toolbar, which reads **Continuous observation: N d Before · M d After**. Click it to open the **Continuous observation** popover with two fields:

- **Days before** controls washout. A value of 365 means "the person must have been observable in the data for at least a year before they entered the cohort", which is the usual way to require a clean look-back period.
- **Days after** ensures a minimum follow-up.

Encode the required look-back as **Days before** on the entry event itself, not as an inclusion rule. Doing it as an inclusion rule biases downstream incidence and characterization denominators; the OHDSI Themis convention is the entry-event form [20, 25].

7.4.4 Three qualifying-event limits

ATLAS exposes three independent limits that decide *which* qualifying events count, each as an **Earliest** / **all** / **Latest** toggle:

Cohort Entry On sits in the header of step 1 and applies to the entry events themselves. **Earliest** produces a *new-user* or *incident* cohort (only the person's first qualifying

record); **all** lets every qualifying record create a cohort entry; **Latest** keeps only the most recent.

Limit Restricted Events to appears below the optional restriction group, which you create with the full-width **Restrict Initial Events** button at the bottom of step 1. That group further restricts entry events *before* the inclusion rules run; the toggle decides which surviving entry is kept.

Limit Included Events to sits below the inclusion area (shown once at least one rule exists) and decides whether the cohort keeps the first, every, or last entry per person that passes the inclusion rules. With **Earliest**, ATLAS keeps only the earliest qualifying entry — usually what new-user designs want.

all plus generous descendants on a chronic-disease concept set is a fast way to multiply rows by an order of magnitude. Pick the limit on each section that matches the question.

7.5 Inclusion criteria

Inclusion criteria are applied after entry events. Each rule is itself a small cohort criterion — “at least one diagnosis of X in the 365 days before index”, “no exposure to Y any time before index” — and the person must satisfy every rule.

Note

Each inclusion rule you add raises specificity at the cost of sensitivity — the trade-off is the same as in any phenotype algorithm. Inspect the inclusion-rule report and a sample of profiles (Chapter 12) to see which side of that trade you have landed on. The methodology is covered in *The Book of OHDSI* [20], chapter 10.

7.5.1 Rail and detail layout

With no rules, step 2 shows “No inclusion rules — adding one narrows which patients qualify beyond the entry events.” and an **Add rule** button. Once a rule exists the section becomes a master–detail layout. The left pane is an *inclusion-rule rail*: an **Inclusion rules (N)** header with **Add rule**, the live-preview source selector (Section 7.1), and one row per rule. Each row shows the rule’s number and name and a summary of its content (“1 group · 2 criteria”). Click a row to load the rule on the right; **Add rule** appends a new, empty rule and selects it. With no rule selected the right pane reads “Select a rule on the left to edit it.”

When live preview is on, the rail also carries running counts: each rule row shows the persons surviving that step and the cumulative percentage of entry events still retained. The row’s colour compares it with the previous step: green when 80% or more of the previous step’s persons survive, amber at 40% or more, otherwise red. While the counts

recompute — debounced shortly after your last edit — all rule rows pulse. If the statistics request fails, an error appears under the rail.

Rules can be reordered by drag-and-drop using the handle at the left of each row. Order matters for the attrition story even though it does not change the final cohort: the per-rule count is the count after that rule and every previous rule has been applied.

The right pane's header shows the selected rule's name and description (placeholders *"Unnamed Rule"* and *"No description"*). Click the header to open a popover with **Rule name** and **Description** fields; the trash icon on the right deletes the rule. A new rule starts with one empty **all** group; if you delete that group, an **Add criteria group** button brings a new one back.

7.5.2 Anatomy of a rule

1. **Criterion type.** Pick what kind of event the rule is about from **Add criteria to group...** (condition, drug, procedure, measurement, observation, visit, device, era, episode, demographic, etc.).
2. **Concept set.** **Select Concept Set** on the card header references one of the cohort's concept sets or creates a new one (Section 6.12).
3. **Attribute filters.** Same domain-specific **Add attribute...** list as for the entry event.
4. **Cardinality.** Click the vertical label down the left edge of the criterion (it reads e.g. **at least 1**; tooltip *"Select logic type"*). The popover offers **exactly**, **at least**, or **at most** plus a **Count**, and a chip that toggles between **Using all events** and **Using distinct events**. Distinct counting adds a **Distinct by** select — **Start Date**, **Standard Concept**, or **Visit** — which maps to CIRCE's `Occurrence.CountColumn`.
5. **Temporal window.** The chip under the criterion states the window relative to index, e.g. *"event starts between 30 days Before and 0 days After index start"*. Click it to edit (Section 7.5.4).

7.5.3 Grouping criteria with AND / OR

Each inclusion rule holds a *criteria group*. Its toolbar has two buttons: **Add criteria to group...** adds another criterion (the menu includes every entry-event type plus **Demographic Criteria**), and **Add Nested Group** inserts a sub-group with its own combinator. An empty group reads *"No events in group. Add events to build criteria logic."*

The combinator that decides how the criteria combine is *not* a dropdown at the top of the rule: it is a clickable vertical label running down the left edge of the group card (tooltip *"Click to change match type"*). Clicking it opens a small popover with four options:

all (logical AND) — the person must satisfy every criterion in the group.

any (logical OR) — the person must satisfy at least one.

at least N — the person must satisfy any N of the criteria; a **Count** field sets N .

at most N — the person must satisfy no more than N of the criteria.

Nested groups are how complex phenotypes like "diabetes diagnosis AND (insulin use OR HbA1c ≥ 6.5)" are expressed. Build the nesting carefully — two criteria at the same level combine differently from one criterion plus a sub-group.

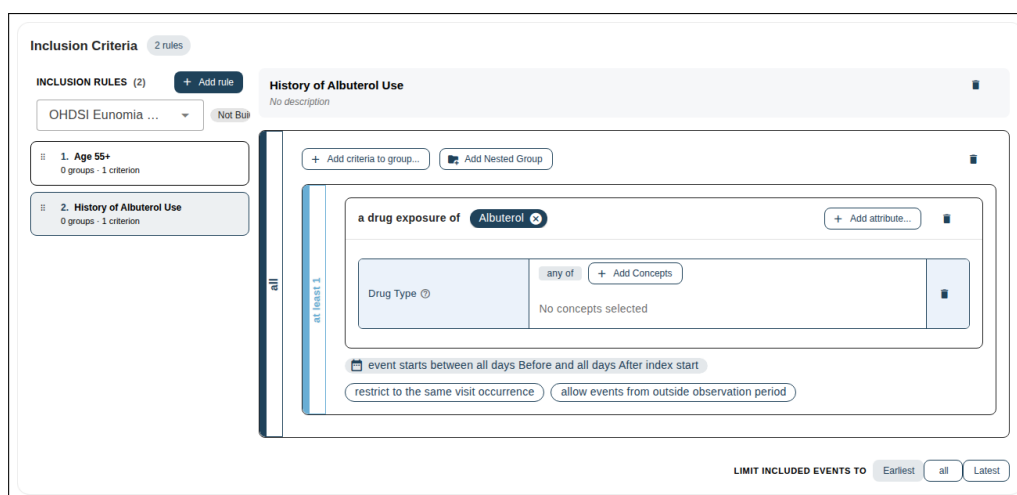


Figure 7.4. Inclusion Criteria with two rules. The rail on the left lists the rules with their content summary under the live-preview source selector (here **Not Built**); the selected rule "History of Albuterol Use" holds an **all** group containing "a drug exposure of [Albuterol]" with the **at least 1** cardinality label, a temporal-window chip, and the two flag chips. **Limit Included Events to** sits at the bottom.

A phenotype that requires a confirming second event after index introduces *immortal time* between the first and second records. Either index on the second event in ATLAS (so the immortal window precedes index) or keep the confirming window short. The bias is documented in Suissa [36] and Swerdel et al. [39].

7.5.4 The criterion inside a group

Entry events use a bare criterion card: they are the anchor, so they have no cardinality and no temporal window. Every criterion placed *inside a group* — in the **Restrict Initial Events** group, in an inclusion rule, or in a nested group — is wrapped with three extra controls:

- the *cardinality label* down the left edge (**at least 1**, **exactly 0**, ...), described above;
- the *temporal-window chip*. Clicking it opens a popover with a **Quick Presets** list — **Short-term baseline**, **Medium-term baseline** and **Long-term baseline** (−30, −180 and −365 to 0 days), **All time prior to index**, **On index date**, **Acute follow-up**, **90-day follow-up** and **1-year follow-up** (0 to 30, 90 and 365

days after), **All time after index**, and an “event overlaps with index” preset — and a row reading [Event starts|Event ends] between N [Before|After] and M [Before|After] [Index starts|Index ends]. **Add Temporal Window** adds a second window on the event’s other end (e.g. “and event ends before index ends”);

- two *flag chips*, **restrict to the same visit occurrence** and **allow events from outside observation period**; click a chip to toggle it.

The same wrapper is used in feature-analysis criteria (Chapter 13), characterization subgroups, and incidence-rate stratify rules, so learning it once covers them all.

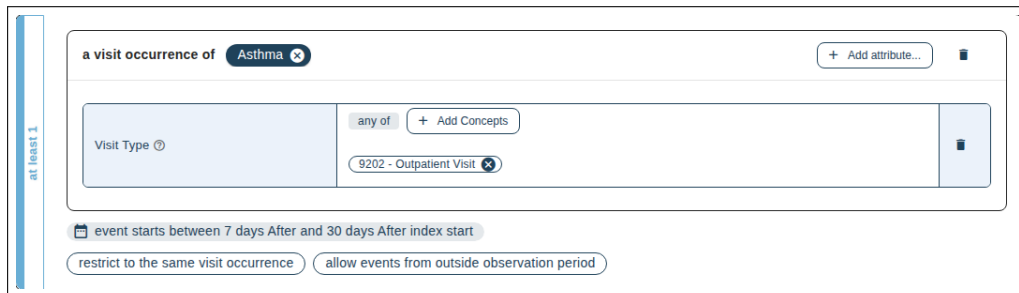


Figure 7.5. A grouped criterion — here the **Restrict Initial Events** criterion of an asthma cohort: the **at least 1** cardinality label on the left, the criterion card (a visit occurrence of *Asthma* with a **Visit Type** attribute), the temporal-window chip (“event starts between 7 days After and 30 days After index start”), and the two flag chips.

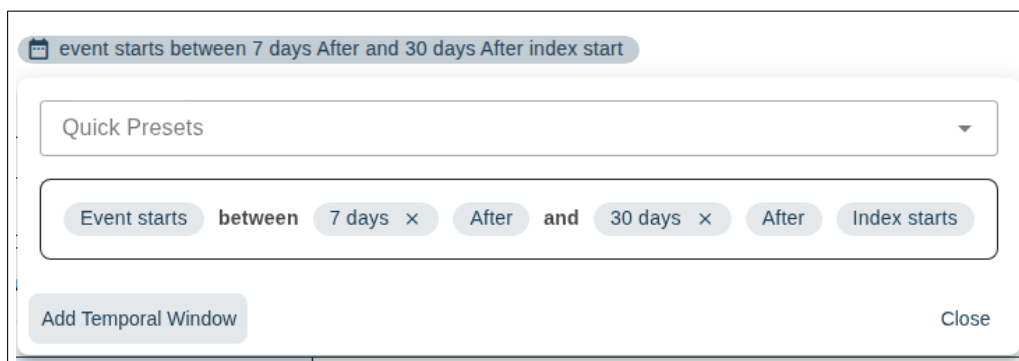


Figure 7.6. The temporal-window popover opened from a criterion’s window chip: the **Quick Presets** list, the editable window row, **Add Temporal Window** for a second (end-date) window, and **Close**.

7.5.5 Reading the inclusion report

After generation, the **Inclusion report** action on a row of the generation table (Section 7.8) opens the **Inclusion rule report** in a side drawer. The report has four parts, all keyed to the same data:

Mode tabs. **By Person** (default) counts each person once across all matching cohort entries; **By Event** counts every qualifying entry separately. The two views agree exactly when the cohort has **Cohort Entry On** set to **Earliest**; they diverge as soon as a person can have multiple entries.

Summary stats. A row of stat cards: **Match rate**, **Matches** (the final cohort size), **Lost** (entry events removed by the rules), and **Total events**.

Attrition. The **Attrition** panel is a per-rule horizontal funnel chart with one bar per inclusion rule, sized to the percentage of entry events surviving that rule. Bars colour-grade from green through amber to red as retention drops; each bar carries the absolute count and percentage, with counts shown in full with thousands separators. A footer summarises the journey: *"Initial: N patients → Final: M patients (X.X % retained)"* with the retention figure tinted by severity. The panel header offers a **Download CSV** action (the file is written as `attrition.csv`) and a **Save as PNG** export of the chart.

Per-rule satisfaction. A table with one row per rule, showing how many entry events each rule kept or dropped. The **Cumulative remaining** column is the funnel's numbers in tabular form — how many entry events survive after this rule and every previous rule have been applied.

Population breakdown. A treemap of the rule-satisfaction signature: each tile is a unique combination of which rules a person passed and which they failed, sized to the number of people with that signature. Hover for the bitfield. The treemap surfaces the dominant failure pattern at a glance — if one tile dominates, that combination is the one to investigate.

Note

When a generation recorded no inclusion statistics for the chosen mode — typically because the cohort has no inclusion rules — the drawer shows an information alert instead of a zero-filled report: *"This generation recorded no inclusion statistics for this mode, so there is no attrition to report. The cohort itself may still contain records; see the generation panel for its record count."* An empty report therefore does *not* mean the cohort matched nobody; check the **Patients** column in the Generation table.

When counts surprise you, read the funnel as a sequence of suspects:

- A rule that drops to near-zero on its own usually means the concept set resolved to nothing on the source — check the concept set's **Included** tab and re-run on a source with known coverage.
- A rule that drops nobody is either trivially satisfied (the temporal window is too wide, the cardinality is too lax) or redundant with an earlier rule. Either way it is not pulling weight.
- A rule that drops disproportionately many people compared with a peer source signals an ETL or vocabulary-mapping difference — the same rule against the same JSON should drop comparable fractions. The HADES CohortDiagnostics package formalises this cross-source comparison with attrition tables side by side [30].

7.6 Cohort exit and censoring

Step 3, **Exit & Eras**, holds the exit strategy, the censoring events, and the **Cohort Eras** settings.

7.6.1 Exit strategy

Exit defines the per-person end date. The **Strategy** toggle offers three choices:

Continuous Observation. The person leaves the cohort when their continuous observation period ends. No further configuration. The most common choice for prevalent or chronic-disease cohorts.

Fixed Duration. A fixed number of days from a date of the qualifying event: pick the **Date Field (Start Date or End Date)** and the **Offset (days)**. Useful for time-to-event analyses with a hard horizon.

Drug Exposure. End of continuous exposure to a drug concept set, for drug-utilisation cohorts. Choose the set with **Select Drug Concept Set**, then set **Gap Days** (the persistence window between consecutive exposures still considered the same era; default 30), **Offset (days)** (days added after the last exposure ends), and optionally **Days Supply Override**. If days supply is missing from the records, ATLAS assumes one day per exposure.

7.6.2 Censoring events

Censoring events end the person's membership early when an outcome makes continued follow-up nonsensical — "Cohort members exit early when any of these events occur." Click **Add Censoring Event** and pick a criterion type; common choices include death, end of insurance enrolment, and onset of an outcome you want to exclude. The menu currently lists the types by their raw CIRCE names (`ConditionOccurrence`, `DrugExposure`, ...) and does not offer Custom Era or Episode.

7.6.3 Era collapse and trimming

For cohorts where the same person can enter and re-enter, the **Cohort Eras** block reads **Specify era collapse gap size: [N] days**. With a 30-day gap, two cohort entries 20 days apart are merged into one continuous era; with a zero gap, every entry stays separate.

The **add trimming options...** link next to it reveals two date pickers, **Left censor cohort start dates to** and **Right censor cohort end dates to** (placeholder "*No Censoring*"). They clip every cohort era to a calendar window — useful when a study period is fixed or when a source's data is unreliable before or after a known date.

The 30-day persistence window is the OHDSI convention for drug eras because it has historically performed well for drug–outcome estimation, not because it is biologically motivated. For long-acting injectables and depot formulations, 90 days

or more is common; for acute treatments where a real treatment break is clinically meaningful within days, shorter values matter. Treat the window as a study parameter: report it in the manuscript and, on sensitive analyses, run the cohort with ± 30 days around your choice as a sensitivity check.

Figure 7.7. Exit & Eras. The **Strategy** toggle picks **Continuous Observation**, **Fixed Duration**, or **Drug Exposure**; **Censoring Events** end membership early; the **Cohort Eras** block sets the era collapse gap and, via **add trimming options...**, the left and right censor dates.

7.7 Concept sets used by this cohort

The **Concept Sets** icon in the cohort toolbar opens a dialog listing every concept set embedded in the definition, with the columns **Id**, **Name**, **Concepts** (the number of expression rows), **Status** (an **Unused** chip for sets no criterion references), and **Actions**. The icon's badge counts the unused sets, a reminder to tidy them up before sharing.

- **Edit** (pencil) opens the set in the *embedded* concept-set editor, a side drawer with the same **Selected** / **Included** / **Source Codes** / **Search** / **Recommend** / **Compare** tabs as the library editor but with **Cancel** / **Apply** instead of **Save** (Section 6.13). **Apply** writes the change into this cohort only.
- **Delete** removes the set. If criteria still reference it, ATLAS asks first: *"Deleting "<name>" will clear it from those criteria, and a criterion with no concept set matches its entire domain. N criteria still use it."*

New sets are usually created from a criterion's **Select Concept Set** button, which opens the concept-set picker (Section 6.12): **New Concept Set**, the sets already **In this definition**, or **Import from repository** to copy one from the shared library. If a library set references concepts the current vocabulary source lacks, it is still imported — a warning notification (*"Concept set "<name>" has concepts that are not in the selected vocabulary"*) lists the IDs that are kept in the definition but cannot be displayed (Section 6.4.1).

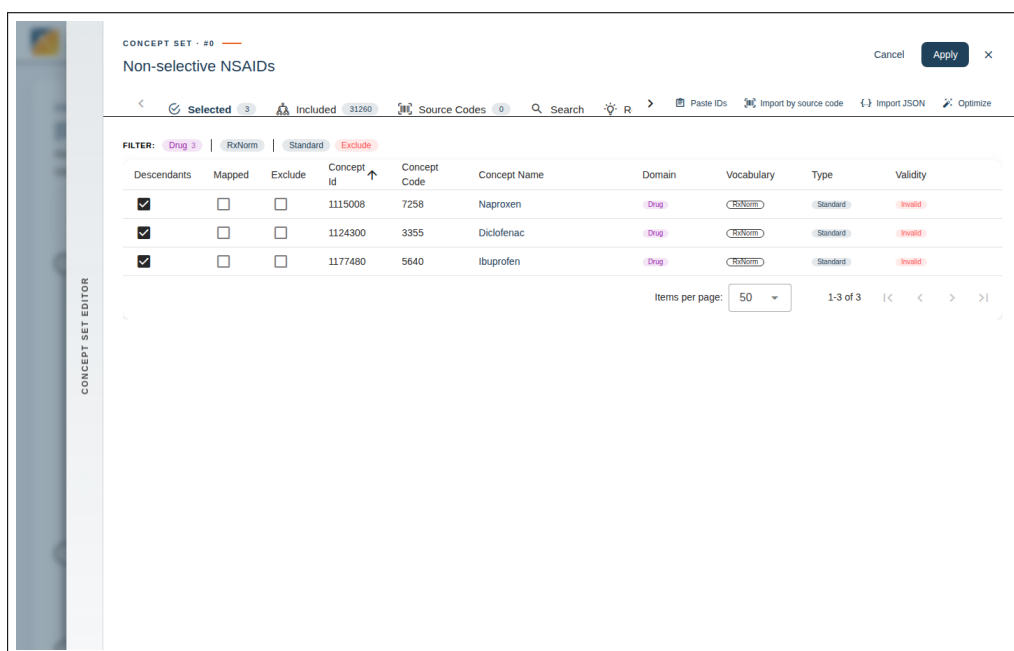


Figure 7.8. The embedded concept-set editor, opened with **Edit** from the cohort's Concept Sets dialog. The eyebrow reads "Concept set · #0" (the set's id within this cohort); **Cancel** and **Apply** replace the library editor's Save.

Note

A cohort definition stores its own *copy* of every concept set it references. This copy travels with the cohort JSON when you export it, so a downstream user can reproduce the cohort even without access to your concept-set library. The cost is that updating a concept set in the library does *not* retroactively update the embedded copy on every cohort that uses it — re-importing the library version into the cohort is a deliberate, explicit action.

7.8 Generation

Generation runs the definition's SQL against a chosen source. The cohort builder carries a collapsible **Generation** section between the header and step 1, collapsed by default — there is no toolbar **Generate** button or separate side panel.

The section header summarises status at a glance:

- a status chip reading **Save cohort to generate, No generations yet**, or *X / Y generated* (green once at least one source has completed and none has failed or is still running, amber while a run is pending or in progress, red when any run has failed);
- a **Last run** · *⟨relative time⟩ on ⟨source⟩* meta line for the most recent successful run;
- an **Only generated** toggle that filters the source table down to sources the cohort has already been generated against, hiding the rest;

- a **Generate all** primary button on the right that runs the cohort against every source you have write access on, skipping sources whose runs are already pending or running.

Generate all and every row's run button are disabled while the cohort has unsaved changes or its design is invalid; hover for the reason (*"Save changes before generate"* or *"Design is not valid"*). Save first, then generate — the server always runs the saved definition.

The body is a single table with one row per connected source and the columns **Source Name**, **Status**, **Patients**, **Last run**, and **Duration**, followed by the row actions:

Generate / Rerun starts a new generation for that source (the button reads **Generate** before the first run and **Rerun** afterwards). While a run is pending or in progress the row's status chip animates and the button turns into **Cancel**. A run the backend reports as failed shows a red **FAILED** chip.

Previous runs (history icon) opens the **Previous runs** dialog, listing every run for that source with **Date**, **Status**, and **Duration**. WebAPI keeps results only for the most recent completed run, so **View results** is enabled for that run alone; older completed runs say *"Superseded: only the most recent run keeps its results"*, and runs that did not complete say *"Results are available only for completed runs"*. Use the dialog to see when and how long runs took, not to compare their counts.

Inclusion report opens the inclusion-rule report in a side drawer (Section 7.5.5). Disabled until the row's last run has completed.

Samples opens the cohort samples panel for that source in the same drawer (Section 7.8.1). Also disabled until the run has completed.

Both **Inclusion report** and **Samples** reflect their state in the URL (`?report=inclusion&source={key}` or `?report=samples&source={key}`), so a link to a particular cohort with a report open is shareable.

Note

A row whose source you do not have write access on shows a disabled run button with the message *"You do not have permission to run jobs against this source"*. The row's read-only data — last-run status, patient count, history — remains visible.

7.8.1 Cohort samples

The **Samples** panel is the fastest path from a generated cohort to individual patient review. Click **New sample** at the top of the panel to open the sample-creation dialog; specify:

Sample name a descriptive label, e.g. review-2025-01-women-over-65.

Number of persons how many to draw, up to the per-source maximum.

Gender (optional) one or more of **Male**, **Female**, **Other** / **non-binary**.

Age (optional) a comparator (*less than, greater than, between, etc.*) plus a value or range.

ATLAS runs a server-side random draw constrained by the filters and lists the resulting persons. Selecting a sample loads its detail block underneath, headed **Source:** `<source name>` and "*N* persons · created `<date>`" so you always know which database the patients come from. Clicking a person opens their **Profile** *in the same drawer*, with a back-arrow in the drawer header that returns to the samples list. Two icon buttons on each sample, **Refresh sample** and **Delete sample**, re-roll it with a new draw or remove it.

Use a sample of 20–50 patients as the lightweight half of phenotype validation: skim each profile, note any obvious mismatches, and adjust the cohort definition. This is much cheaper than a full PheValuator run (Chapter 12, "Going further") and catches the gross errors first.

Note

Generation is the only step that actually queries patient data, and on large sources it can take a while. It is also the first place a flaw in the definition — a concept set that returns nothing, a temporal window that excludes everyone — becomes visible. The inclusion-rule report (Section 7.5.5) is your first stop when counts surprise you.

7.9 Going further: external diagnostic tools

The reports built into the Generation section are deliberately a quick view — attrition by rule, plus a sample. For the deeper diagnostics expected of a sharable phenotype, two external HADES R packages consume the JSON ATLAS exports:

CohortDiagnostics [28, 30] produces cross-source attrition, index-event breakdown, source-code review, orphan codes, incidence by age / sex / year, cohort overlap, and time distributions. Fast, runs without a gold standard.

PheValuator (Chapter 12, Section 12.5) estimates sensitivity, specificity, and PPV against a probabilistic gold standard.

Both are recommended evidence for OHDSI Phenotype Library submissions [26]. The methodology behind them lives in *The Book of OHDSI* [20], chapters 16 and 18.

For *negative-control outcomes* — the supplementary cohorts many OHDSI designs use to detect residual bias — ATLAS is just where you build them. Tag them `negative-control`; the methodology and recommended counts are covered in the Book of OHDSI's Method Validity chapter.

7.10 Saving, copying and exporting

Save the cohort with a meaningful name as soon as you start editing, even before any criteria are filled in. The first **Save** creates a permanent ID and unlocks version history, access control and generation; subsequent edits then accumulate as versions you can roll back to, rather than as one giant undo stack you can lose.

- **Save** writes the cohort to WebAPI. Repeated saves create new versions (Chapter 14). A cohort needs only a name to be saved — an incomplete definition with no entry events can be kept as a draft. A small dot next to **Save** (tooltip “*Unsaved changes*”) marks a dirty editor. When **Save** is disabled, hover it for the reason: previewing an earlier version, no permission to create cohorts, no write access to this cohort, a missing name, or validation errors (Section 2.4).

- The **Export** action (download icon) opens a menu with four items:

JSON — *Download as file* writes the definition’s JSON to disk.

Copy to clipboard — *JSON* copies the same JSON for pasting elsewhere.

View / Edit — *Overwrite the cohort from JSON* opens the **Cohort JSON** dialog (below).

SQL — *View and download the generated SQL* opens the **Cohort SQL** dialog (below).

The exported JSON can be re-imported into this ATLAS instance or another.

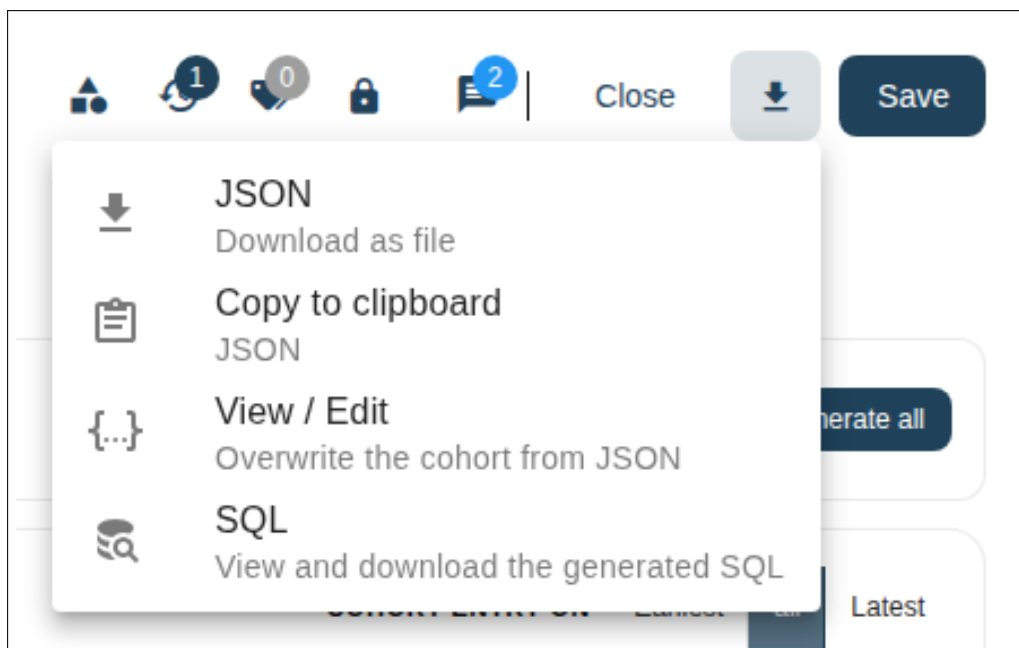


Figure 7.9. The cohort **Export** menu: **JSON**, **Copy to clipboard**, **View / Edit**, and **SQL**.

7.10.1 Editing the cohort as JSON

View / Edit opens the **Cohort JSON** dialog: *"Edit the Atlas expression directly. Applying replaces the cohort logic — the name and description are kept."* The expression is shown in an editable text area; **Load file** replaces it with a JSON file from disk, and **Copy** and **Download** export what is in the box. **Apply to builder** validates the JSON with the same strict checks as import and, if it passes, replaces the cohort's logic in the builder: *"Cohort JSON applied — review the builder, then save"*. Nothing is written to the server until you click **Save**, so **Cancel** (or discarding the unsaved changes) backs out. **Apply to builder** is disabled while you preview an earlier version.

This is the quickest way to paste a cohort shared as JSON into an existing definition, or to make a bulk edit (renaming many inclusion rules, say) that would be tedious by clicking.

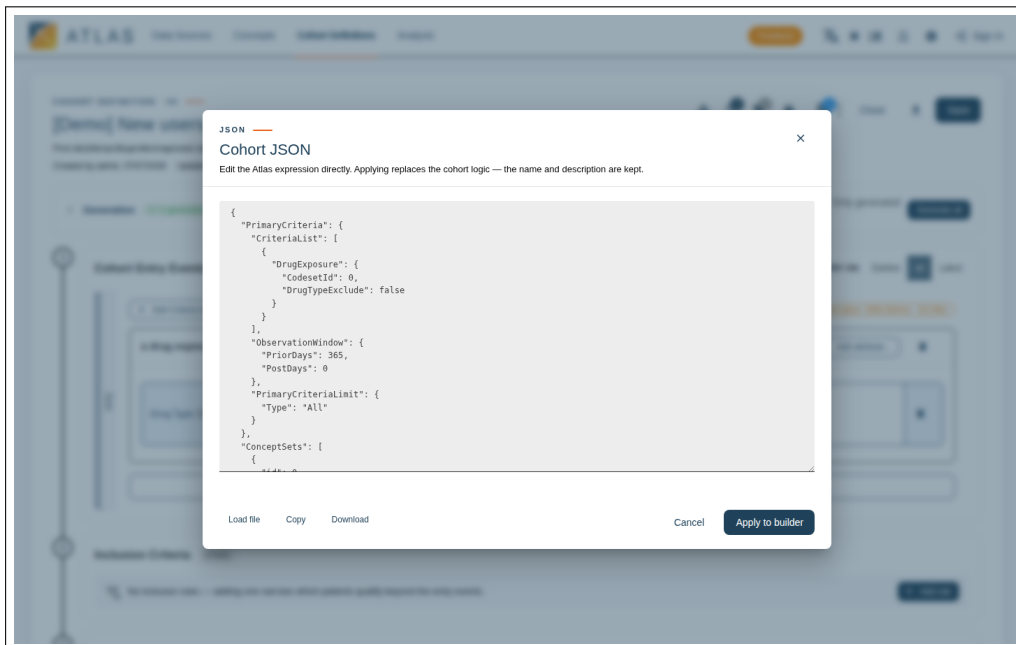


Figure 7.10. The **Cohort JSON** dialog. Edit the expression in place, or use **Load file**, **Copy**, and **Download**; **Apply to builder** replaces the cohort logic but keeps its name and description.

7.10.2 Viewing the generated SQL

SQL opens the **Cohort SQL** dialog with the query this cohort generates. By default it shows the *template* SQL (OHDSI SqlRender dialect) with its @parameters — @cdm_database_schema, @vocabulary_database_schema, @target_cohort_id and so on — so it runs anywhere once translated. Pick a database from the **Dialect** select (SQL Server, PostgreSQL, Oracle, Amazon Redshift, Google BigQuery, Snowflake, Spark, Azure Synapse, Impala, IBM Netezza, Microsoft PDW, or Hive) to see it rendered for that platform. **Copy** puts the SQL on the clipboard; **Download** saves it as a .sql file.

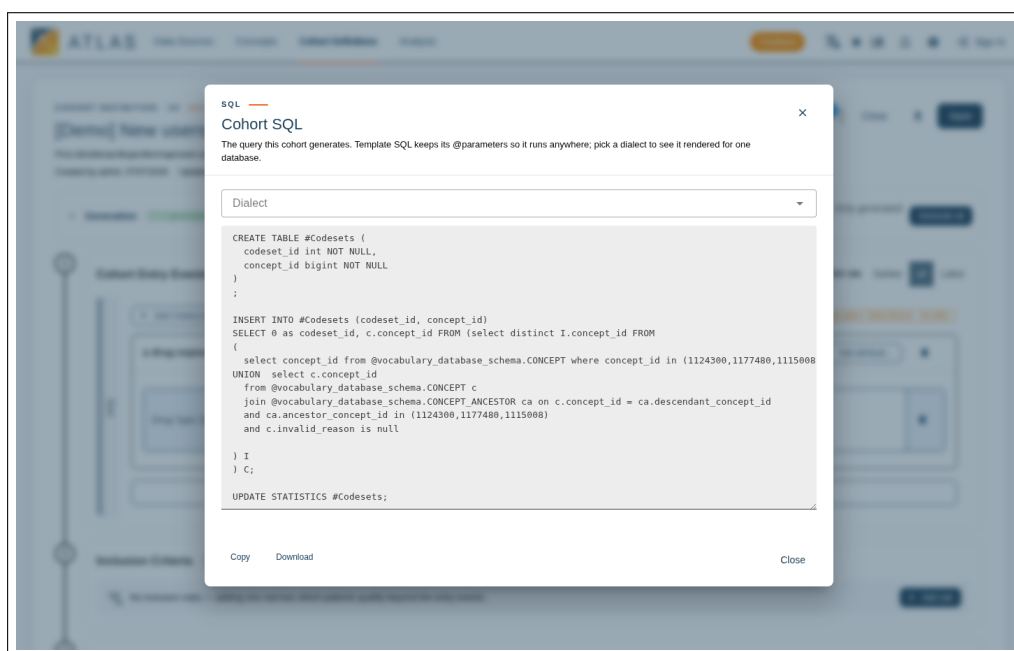


Figure 7.11. The **Cohort SQL** dialog, showing the template SQL. The **Dialect** select renders it for a specific database; **Copy** and **Download** export it.

Reach for the SQL export when a DBA asks what a cohort will run, when you need to execute the cohort outside ATLAS (for instance in a HADES pipeline without WebAPI access), or when debugging why a criterion behaves unexpectedly — the SQL makes every join and date comparison explicit.

7.10.3 Tagging a cohort

The cohort toolbar carries a tag icon (tooltip **Tag Management**) with a count badge of the current tags. Click it to open the **Manage Tags** dialog, which lists every tag defined in the deployment grouped by tag group; tick the ones that apply.

Tags are deployment-wide artefacts. The **Manage Tags** dialog has a **Create New Tag** form for adding one on the spot; tag groups and the curation of existing tags live in the configuration panel's **Tags** tab (Chapter 15). A useful convention is one tag per project area (e.g. cardiology, oncology), one per study phase (exploratory, published, negative-control), and a small set of reusable qualifiers (phenotype, drug-class, outcome). Tags appear on cohort cards and in the **Tags** column of the list page; filter by them with the list's **Filters** menu (the **Tags** column itself is not sortable).

The same tag mechanism applies to concept sets (Chapter 6), characterizations (Chapter 9), pathways (Chapter 10), and incidence rates (Chapter 11); the icon, dialog, and behaviour are identical across editors.

7.10.4 Sharing a cohort

On a secured deployment a cohort is writable only by its author and administrators unless access is granted explicitly. To let a colleague or a project role edit it, click **Configure access** (lock icon) and grant the role **Read** or **Write** access; write implies read. The dialog is the same in every editor and is described in Section 15.4. The button is disabled until the cohort has been saved.

7.10.5 Auto-save and draft recovery

While you edit a *new* cohort definition that has never been saved, ATLAS writes a draft of it to the browser's session storage every 30 seconds. If the page is refreshed, or you sign back in after the session expired, returning to the new-cohort editor restores that draft. Once a cohort has been saved, drafts are not read back: edits since the last **Save** are lost on refresh or tab close.

The draft is per tab and is not synced to other browsers or devices; it is cleared when you save or when the browser session ends. Treat it as a safety net for the first minutes of a new definition, and save early (a name is all **Save** needs).

PART IV

Analyses

The Analysis Hub

The four analyses in this part of the manual — characterizations, cohort pathways, incidence rates, and feature analyses — no longer live behind four separate navigation entries. ATLAS v3.0 gathers them into a single *Analysis Hub*: one page with a tab per analysis type. This chapter explains how to reach each analysis and how the hub is laid out, so the chapters that follow can concentrate on the analyses themselves.

8.1 Where the analyses live

The top navigation bar carries four destinations: `Data Sources`, `Concepts`, `Cohort Definitions`, and `Analysis`. Opening `Analysis` lands you in the hub, which presents its four analyses as tabs across the top of the workspace:

Feature Analyses. Reusable covariate definitions — the building blocks that characterizations draw on (Chapter 13).

Characterizations. Descriptive summaries of one or more cohorts (Chapter 9).

Cohort Pathways. Sequential treatment- and event-pattern diagrams (Chapter 10).

Incidence Rates. Outcome rates within a target population over a time-at-risk window (Chapter 11).

The hub remembers the tab you used last and reopens it the next time you return; the first time you arrive it defaults to **Feature Analyses**. The active tab is drawn bolder, with a thicker underline. Deployments with plugins can add further tabs after the four built-in ones; a plugin tab lives at `/analysis/x/{pluginId}/{itemId}` (Chapter 16).

Each tab shows its own list of saved analyses. The lists share the conventions described in Section 2.4:

- **Columns.** Every list leads with an **Id** column and ends with **Created By**, **Created**, **Updated**, and **Actions**. The dates are shown as relative times ("3 days ago"); hover one for the full date.
- **Sorting.** Click a column header to sort. The default is **Updated**, newest first, so what you touched last is at the top. An analysis that has never been modified sorts by its creation date.
- **Search.** The search box matches every word you type, in any order, against the name (on Cohort Pathways and Incidence Rates also the description).

- **New and Import.** The Characterizations, Cohort Pathways, and Incidence Rates tabs carry a **New** ... button and an **Import** button. **Import** takes a .json file exported from ATLAS, creates the analysis, and opens it in the editor.
- **Feature Analyses is different.** Its list has a **Filter results** box and a **Filters** facet menu instead of a plain search, its **New feature analysis** button is a drop-down that asks which type to create, and it has no **Import** (Chapter 13).

Note

Old bookmarks still work. The former standalone URLs — /characterizations, /feature-analyses, /pathways, and /incidence-rates — redirect into the matching hub tab, so links you saved against an earlier build land in the right place. ATLAS 2.x links such as #/cc/characterizations/12 or #/iranalysis/3 are translated as well (Section 2.7).

8.2 Lists, editors, and results

The hub is the home for the *lists* of saved analyses. Building or reviewing an individual analysis takes you out of the tabbed hub and into a full-screen editor or result page — creating a new characterization, opening a pathway’s sunburst, or drilling into an incidence-rate workbench each fills the whole window rather than sitting inside a tab. Closing or navigating back from one of these full-screen views returns you to the hub tab you started from.

This split keeps the hub itself uncluttered: it is a launcher and an index, while the detailed work happens on a dedicated page with room for the design rail, canvas, and insight panels described in the chapters ahead.

8.3 A note on Profiles

Person *profiles* (Chapter 12) are an analysis in spirit, but they are deliberately *not* part of the hub and do not appear in the top navigation. A profile is always about one specific person, so you reach it by opening a profile URL directly, or — inside a cohort definition — by clicking a person ID in a cohort sample, which shows the profile in the cohort’s report drawer, rather than by browsing a list. Chapter 12 covers those entry points.

ID	NAME	ANALYSIS TYPE:	DOMAIN:	STAT TYPE	CREATED BY	CREATED	UPDATED ↓	ACTIONS
2	Condition Group Era Start Long Term	PRESET	CONDITION	PREVALENCE	---	Unknown	Unknown	
3	Drug Group Era Start Medium Term	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
5	Drug Group Era Long Term	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
6	Condition Group Era Overlapping	PRESET	CONDITION	PREVALENCE	---	Unknown	Unknown	
7	Drug Group Era Short Term	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
8	Drug Group Era Medium Term	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
11	Condition Group Era Start Medium Term	PRESET	CONDITION	PREVALENCE	---	Unknown	Unknown	
15	Drug Group Era Any Time Prior	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
19	Drug Group Era Start Short Term	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
20	Condition Group Era Short Term	PRESET	CONDITION	PREVALENCE	---	Unknown	Unknown	
24	Drug Group Era Start Long Term	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	
25	Drug Group Era Overlapping	PRESET	DRUG	PREVALENCE	---	Unknown	Unknown	

Figure 8.1. The Analysis Hub on its **Feature Analyses** tab. The four tabs — Feature Analyses, Characterizations, Cohort Pathways, Incidence Rates — sit above the list; this tab has a **Filter results** box, a **Filters** menu, and the **New feature analysis** drop-down, with **Id**, **Created**, and **Updated** columns in the table.

Characterizations

A *characterization* summarises the clinical features of one or more cohorts. Where a cohort report (Chapter 7) describes one cohort on one source, a characterization is broader: it can put several target cohorts side by side, attach outcomes, and apply a deliberate, reusable set of feature analyses.

9.1 What you can answer with a characterization

Typical questions:

- Among new users of drug A versus drug B, what does the patient mix look like at index — age, comorbidities, prior medications?
- In a population with disease X, how does the burden of related conditions evolve in the year after diagnosis?
- Across five different data sources, how comparable are the baseline characteristics of the same target cohort?

A well-designed characterization is what you publish alongside a study to demonstrate that “the patients in my cohort look like what you would expect of patients with this condition”.

Note

For comparative-effectiveness work, you typically run a characterization with two target cohorts (the *target* and the *comparator* of an active-comparator new-user design [42]) before propensity adjustment. The study design and methodology are covered in *The Book of OHDSI* [20], chapters 7 and 12.

Note

The ATLAS characterization presets give four standard time-window durations relative to the cohort entry (*index*) date: *Any time prior*, *Long-term* (365 days), *Medium-term* (180 days), and *Short-term* (30 days). A typical baseline bundle covers all four so reviewers can see how prevalence changes as the look-back window narrows. The underlying distinction between baseline, post-index, and time-at-risk windows is methodology covered in *The Book of OHDSI* [20], chapter 11.

9.2 Anatomy of a characterization

Open **Analysis** and switch to the **Characterizations** tab of the Analysis Hub (Chapter 8), then click **New characterization** (or **Import** to load a characterization JSON exported from another ATLAS). The editor header carries the name and description, the authorship line (*Created by ... · Updated by ...*), and a toolbar with the Concept Sets, validation-messages, Versions, and **Configure access** (lock) icons plus **Cancel**, **Import**, **Export**, **Duplicate**, **Delete**, and **Save**. The toolbar behaviour is shared with the other editors (Section 2.4); access sharing is covered in Section 15.4.

Show Analysis Design opens the design rail on the left. The design has three sections, in order:

Cohorts. One or more cohorts whose members you want to describe. **Add** opens the **Choose a Cohort** dialog, which has an **Id** column and a search box that matches name words in any order or an id (exactly or by prefix); tick the cohorts you want one by one — there is no select-all checkbox. Each linked cohort shows its id as a subtitle (*Id 4*). There is no separate outcome-cohort slot — outcome-stratified characterisations are expressed by adding the outcome cohort here and a matching subgroup below.

Feature analyses. The list of covariates to compute. Each feature analysis is itself a reusable artefact (Chapter 13). A typical default set includes demographics, conditions in the year before index, drug exposures in the year before index, prior visits, and Charlson comorbidity components. **Add** opens the **Choose a Feature Analyses...** picker: a **Filter results** box plus a **Filters** menu with **Type**, **Domain**, **Created**, **Updated**, **Author**, and **Designs** facets (each value shows its count; active filters appear as chips with **Clear All Selections**), above a Name / Type / Description table.

Subgroup analyses (optional). Overlapping selections of the population built like miniature inclusion rules: concept set, attribute filters, cardinality, and a temporal window relative to index. Each subgroup becomes an extra column in the result alongside the main cohort. Use them to ask questions like "what does the cohort look like *among those with prior MI?*" without having to redefine and regenerate the cohort itself. A **Calculate subgroup analyses only** toggle is available when you want subgroup columns without the full characterisation.

The data sources to run on are picked at generation time, not on the design form.

9.2.1 Building a subgroup

New subgroup adds a subgroup card with a **Subgroup name** field, a chip summarising its criteria, and an **Edit criteria** button. **Edit criteria** opens a drawer from the right (headed **Subgroup analyses**) holding the same criteria-group editor as a cohort inclusion rule: add criteria, pick or build their concept sets (the concept-set picker of Section 6.12), set attribute filters, cardinality, and a temporal window relative to index. **Close** returns

to the design. The subgroup membership is computed at characterization time and yields a column whose values are restricted to persons satisfying the criteria.

COHORT CHARACTERIZATIONS

[Demo] Characterization of NSAID new users
 Demographics, Charlson, condition & drug group eras for celecoxib vs non-selective NSAID new users.
 Created by admin, 07/07/2026 · Updated by admin, 07/07/2026

[Show Analysis Design](#)

Source Name	Status	Last run	Duration	
OHDSI Eunomia Demo Database EUNOMIA	COMPLETED	8/28/26, 2:09 AM	5s	Rerun View results History 5
SynPUF 5% (CMS DE-SynPUF) SYNPUFSPCT	COMPLETED	8/28/26, 2:09 AM	37s	Rerun View results History 3

Per-analysis Baseline Overview **#119 SYNPUFSPCT · 25,386 results** Threshold 0 [Configure](#) [Export all](#)

NEW SOURCE	STARTED	FINISHED	DURATION	RESULTS
SYNPUFSPCT	Aug 28, 2026, 2:09 AM	Aug 28, 2026, 2:09 AM	37s	25,386

Domain Analysis Cohort Search

DEMOGRAPHICS INDEX YEAR

Prevalence (2)

Covariate	Concept Id	[Demo] New users of celecoxib - Count	[Demo] New users of celecoxib - Pct	[Demo] New users of non-selective NSAIDs - Count	[Demo] New users of non-selective NSAIDs - Pct	Std Diff
2009	—	861	57.82%	17,648	66.17%	-0.1725
2010	—	628	42.18%	9,019	33.81%	0.1729

Figure 9.1. Characterization editor with the design hidden. The header shows the authorship line and the toolbar (including the **Configure** access lock); below it the data-source run table has a **Rerun** button, the eye **View results** icon, and the history icon with its run count per source, followed by the Per-analysis / Baseline / Overview toggle and the Domain / Analysis / Cohort / Search filter strip.

9.3 Generating and viewing results

The canvas shows a *data-source run table* at the top with one row per connected source. Each row carries the source name, its current execution status (**COMPLETED**, **RUNNING**, **FAILED**, **CANCELED**, or pending / starting / stopping while in flight), the last-run timestamp, the duration, and a single primary action button on the right whose label cycles through **Generate** (never run), **Cancel** (while running) and **Rerun** (after completion). Run buttons stay disabled until the design has been saved. Next to it an eye icon, **View results**, appears when the source's latest run has completed and loads that run into the canvas; it is drawn filled for the run currently on screen. A history icon (MDI-HISTORY) follows, with a small count when more than one run exists. WebAPI executes one characterization run per (cohort × source) pair; for each cohort it computes every feature analysis.

Click the history icon on a source's row to open the **Previous runs** dialog scoped to that source — it lists every historical run with its **Date**, **Status**, and **Duration**. Every completed run's row has an enabled **View results** action, so you can reopen an earlier run and compare it with the latest; runs that did not complete show *"Results are available only for completed runs"*. Cohort pathways use the same dialog; incidence rates do not have one (Chapter 11).

9.3.1 When a run fails or has no results

The canvas says plainly why there is nothing to show:

Save the design to enable runs the characterization is new or unsaved; runs need a saved design.

No runs yet the design has never been generated.

Run in progress a run is pending or executing; the canvas shows *Polling for completion...* and fills in when it finishes.

Run failed the selected run failed. On opening, ATLAS selects the newest completed run across all sources, or the newest failed one when none has completed; a failed run prints its exit message in red, typically a WebAPI stack trace — the first line is usually the useful one.

Run finished, but its results could not be loaded the run completed but fetching its results returned an error; the server's message is shown.

No data results exist, but the filter strip hides all of them.

When a run completes you see two output styles:

Prevalence-style features (binary covariates such as "had diabetes in the prior year"): a percentage with a count column, sorted by prevalence within each target cohort.

Distribution-style features (continuous covariates such as age): mean, standard deviation, median, and quartiles.

9.3.2 Result view modes

A toggle on the canvas toolbar switches the result between three view modes:

Per-analysis the default view — the prevalence and distribution tables described above, one block per feature analysis.

Baseline a computed "Table 1" summary that lays the cohorts out as columns and the covariates as rows, in the style expected of a published baseline-characteristics table.

Overview a charts dashboard rather than tables: box plots for every distribution analysis (age, Charlson index, ...), demographic bar charts, a prevalence-comparison chart, a standardized-mean-difference (SMD) chart for spotting covariates that separate the cohorts, and a top-covariates bar chart of the most prevalent features.

A filter strip above the result table carries two multi-selects — **Domain** and **Analysis** — a single-select **Cohort** dropdown, and a **Search** box (placeholder *Filter results by name*) that matches covariate or concept names, every word in any order. Combine them to narrow a busy result to the rows relevant to the question at hand; the **Search** box works in the Per-analysis and Baseline views. On the toolbar a **Threshold** $\geq$ prevalence filter hides every covariate whose prevalence falls below the value you set, so you can strip a long report down to its common features in one step.

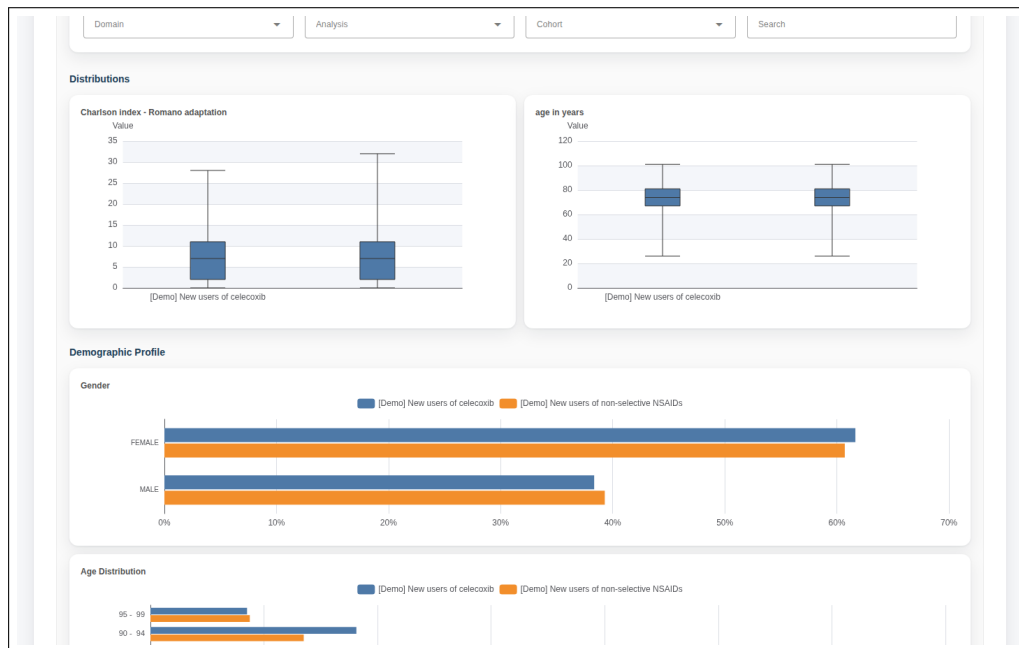


Figure 9.2. The characterization result in Overview mode — a Distributions row of box plots (Charlson index, age) and a Demographic Profile of grouped bar charts — reached from the Per-analysis / Baseline / Overview toggle on the canvas toolbar.

Each row has an **Explore** action that opens a single-covariate detail dialog. The dialog leads with the covariate’s metadata — its analysis, domain, and concept id; clicking the concept id opens the concept detail drawer — and then shows a per-cohort comparison table with an inline bar indicator against each cohort’s value, so you can read one covariate’s prevalence across all the target cohorts side by side. It is a focused look at a single row, not a drill-down into the concepts that contribute to it.

9.4 Export

Export all on the canvas toolbar writes the result table to CSV for use in a spreadsheet, a manuscript table, or further analysis in R. The export honours the current filters — clear the filter strip first if you want every row. The downloaded file is named `characterization-{executionId}.csv` (or `characterization-results.csv` when no specific run is selected) so you can match it back to the run that produced it.

9.5 Common pitfalls

- **Empty rows on a source.** If the target cohort generates zero persons there, every covariate is necessarily blank. Check the cohort generation log first.
- **A run that fails straight away.** A **Run failed** state with a short duration usually means a feature analysis or cohort the design references is broken on the server side; read the first line of the red exit message before re-running.

Cohorts

+ Add



[Demo] New users of cele...
Id 3

×



[Demo] New users of non-...
Id 4

×

Feature analyses

+ Add



Demograph...
PRESET ...

☐ Temporal

×



Demograph...
PRESET ...

☐ Temporal

×



Charlson In...
PRESET ...

☐ Temporal

×



Condition Group Era Start...
PRESET — One...

×



Demograph...
PRESET ...

☐ Temporal

×



Drug Group Era Long Term
PRESET — One...

×



Demograph...
PRESET ...

☐ Temporal

×



Demograph...
PRESET ...

☐ Temporal

×

Subgroup analyses

+ New subgroup

No subgroups defined

Figure 9.3. The design rail in detail: a **Cohorts** section in which each linked cohort shows its id, and a **Feature analyses** section listing each linked feature with its type, a **Temporal** checkbox where applicable, and a remove action. Each section has its own **Add** button; the Subgroup analyses section follows further down.

- **Picking too many features.** Each feature analysis costs time and produces a row in the report. A scoped, reproducible set is more useful than an unfiltered dump.
- **Window mismatch.** Feature analyses define their own windows relative to index. If your cohort exit is shorter than a covariate's window, the covariate may pull data from outside the person's cohort membership.

9.6 Reproducing a characterization in R

Once a characterization is saved, the same definition can be re-executed in R against a CDM the WebAPI may not be connected to. The HADES FeatureExtraction [15] package consumes the same JSON the ATLAS v3.0 editor produces; build a `covariateSettings` object via `createCovariateSettings(...)` or, for the aggregated form ATLAS computes, `createDefaultCovariateSettings(...)` with `aggregated = TRUE`. The R workflow is the right choice when the result needs to feed a downstream HADES analysis or when you want to add custom covariates not expressible in the ATLAS UI.

Cohort pathways

Cohort pathways answer the question: in what order do clinical events happen for the people in my cohort? You feed in a target cohort plus a small set of *event cohorts* (each representing a treatment or a clinical milestone), and ATLAS v3.0 produces a sunburst-style diagram that visualises the most common sequences. The technique was used at network scale to characterize treatment patterns across the OHDSI collaborators in Hripcsak et al. [5].

10.1 When this analysis is the right tool

Pathways shine when the comparison of interest is sequential rather than cross-sectional. Examples:

- Among newly diagnosed type 2 diabetes patients, what is the usual order in which oral antidiabetic drug classes are tried?
- For patients with depression, how often is the second prescribed medication a different class from the first?
- In oncology, what does the line of therapy distribution look like?

10.2 Setting up a pathway

Open Analysis and switch to the **Cohort Pathways** tab of the Analysis Hub (Chapter 8), click **New pathway** (or **Import** to load a pathway JSON), open the design with **Show Analysis Design**, and fill in:

Target cohorts. One or more cohorts whose pathways you want to compare. The result page lays out one sunburst per target.

Event cohorts. Each event cohort represents one node type in the pathway — typically one drug class or one milestone per cohort. The shorter and more orthogonal this list, the more readable the resulting diagram.

Collapse window (days). Two event cohorts that overlap inside this window are treated as a combination at the same step rather than as a sequence. Pickable values: 1, 3, 5, 7, 10, 14, 30.

Minimum cell count. Sequences with fewer than this many persons are suppressed (selectable from 1 to 10). Pair this with whatever small-cell suppression policy your project or IRB requires so the diagram never reveals near-unique patient trajectories.

Maximum path length. How many steps the analysis follows past the index event, from 1 to 7. Longer paths yield more detail but produce sunbursts that fragment into increasingly small wedges and rapidly cross the minimum cell count threshold.

Allow repeats. When on, the same event cohort may appear more than once on a single person's pathway (e.g. a drug re-initiation after a gap). When off, the second occurrence is collapsed into the first.

Targets are typically *treatment-initiator* cohorts (e.g. "new users of any first-line anti-hypertensive"), and event cohorts are *one per intervention* (one per drug class, one per procedure type) [5]. Keeping the event cohorts orthogonal and small in number is the single biggest determinant of whether the resulting diagram is readable.

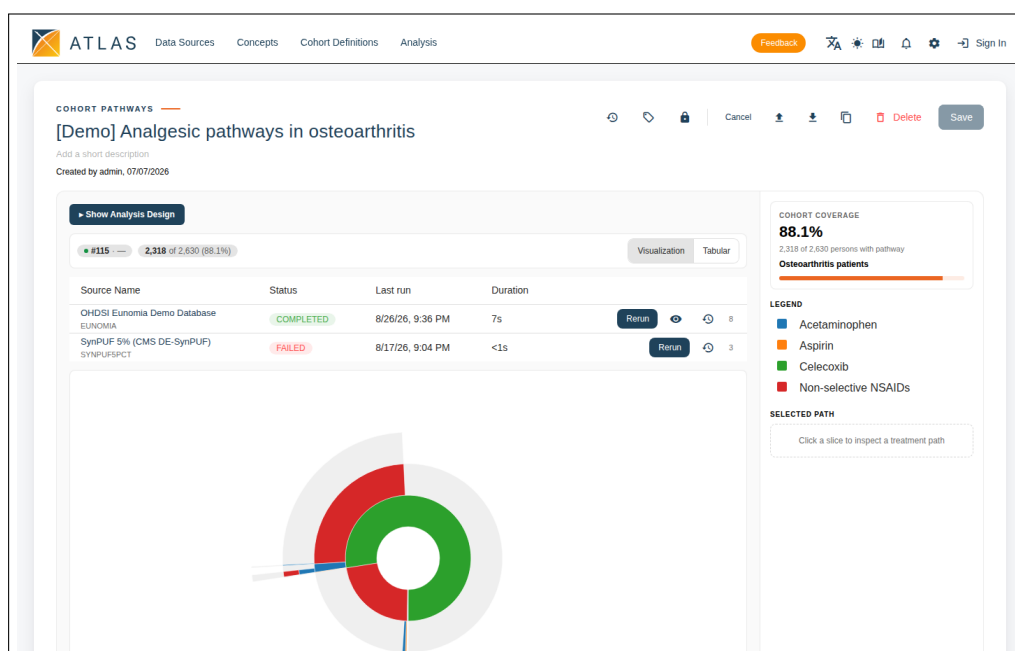


Figure 10.1. Cohort-pathway editor with the design hidden. The header carries the authorship line and the toolbar (Versions, Tags, **Configure access**, Cancel, Import, Export, Duplicate, Delete, Save). The canvas shows the run chip and Visualization / Tabular toggle, the data-source run table with **Rerun**, the eye **View results** icon, and the history icon per source, and the sunburst; cohort coverage, legend, and selected path live in the right rail. **Show Analysis Design** opens the design rail with Target Cohorts, Event Cohorts, and the analysis settings.

10.3 Reading the result

The sunburst shows the index event at the centre and each subsequent step as a ring expanding outwards. The wedges in each ring are colour-coded by event cohort, drawn from a 20-colour palette — the legend table beside the chart maps each colour to the event cohort it represents, and the same colour appears at every step that event takes part in. The angular extent of every wedge is proportional to the number of persons whose pathway took that sequence.

Hovering a wedge shows the count at that step. Clicking a wedge opens the *path details* table beside the diagram, which lists every person-pathway containing that node, with

columns for **Event**, **Remain**, **%**, **Diff**, and **%** — useful when you want to read off a specific path verbatim rather than guess from the geometry. A **Visualization / Tabular** toggle on the canvas toolbar switches between the sunburst and a table-only view of the same pathways.

Generations are dispatched per source from the data-source run table at the top of the canvas: each row's primary action button cycles **Generate** → **Cancel** → **Rerun** as the run progresses. Beside it, the eye icon **View results** loads that source's latest run into the canvas (it is only shown when that latest run has completed), and a history icon (with a count once multiple runs exist) opens the **Previous runs** dialog scoped to that source — the same dialog characterizations use (Chapter 9). Any completed run in that list can be reopened with its **View results** action.

Note

The pathway count CSV counts each person once per event cohort, so the percentages in it cannot exceed 100%. Exports taken from older ATLAS builds could double-count people who entered the same event cohort more than once.

A diagram that looks visually busy but where the largest sequences only account for a few percent of the cohort suggests the event cohorts are too granular. Consolidate related medications into class-level event cohorts and re-run.

Incidence rates

The *incidence rate* module estimates how often new outcomes occur inside a target population per unit of person-time. For each (target cohort $\times$ outcome cohort $\times$ source) combination, ATLAS v3.0 returns persons-at-risk, time-at-risk, outcome cases, and the rate expressed per a chosen multiplier (commonly 1,000 person-years).

Note

The formal definitions of *incidence proportion* (per 1,000 persons) and *incidence rate* (per 1,000 person-years), and the "first event only, prior cases excluded" semantics ATLAS uses, are covered in *The Book of OHDSI*, chapter 11 [20].

11.1 Setting up an incidence rate analysis

Open **Analysis** and switch to the **Incidence Rates** tab of the Analysis Hub (Chapter 8), then click **New incidence rate** (or **Import** to load an incidence-rate JSON) and open the design with **Show Analysis Design**. Inputs:

Target cohorts. The populations whose outcome incidence you want to estimate.

Outcome cohorts. One per outcome you are studying.

Time at risk. Set start and end as anchor + offset pairs: each side picks an anchor date (**Cohort start** or **Cohort end**) and a number of days to offset. So an intent-to-treat 365-day window is *Start: Cohort start + 0 days, End: Cohort start + 365 days*; a per-protocol window pegs to **Cohort end**. Time-at-risk is automatically clipped at end of observation and at the first outcome.

Study window (optional). Restrict the analysis to a calendar date range with explicit **Start date** and **End date** fields. Use it for cohorts that should be analysed within a specific reporting period.

Stratify rules (optional). User-defined strata, each one a **Rule name**, an optional description, and a criteria group edited with the same editor as a cohort inclusion rule (concept set, attribute filters, cardinality, temporal window; concept sets come from the picker described in Section 6.12). Add a rule per age band, per gender, per calendar year, or per custom partition; the result table breaks the rate down by every rule that evaluates to true for a person.

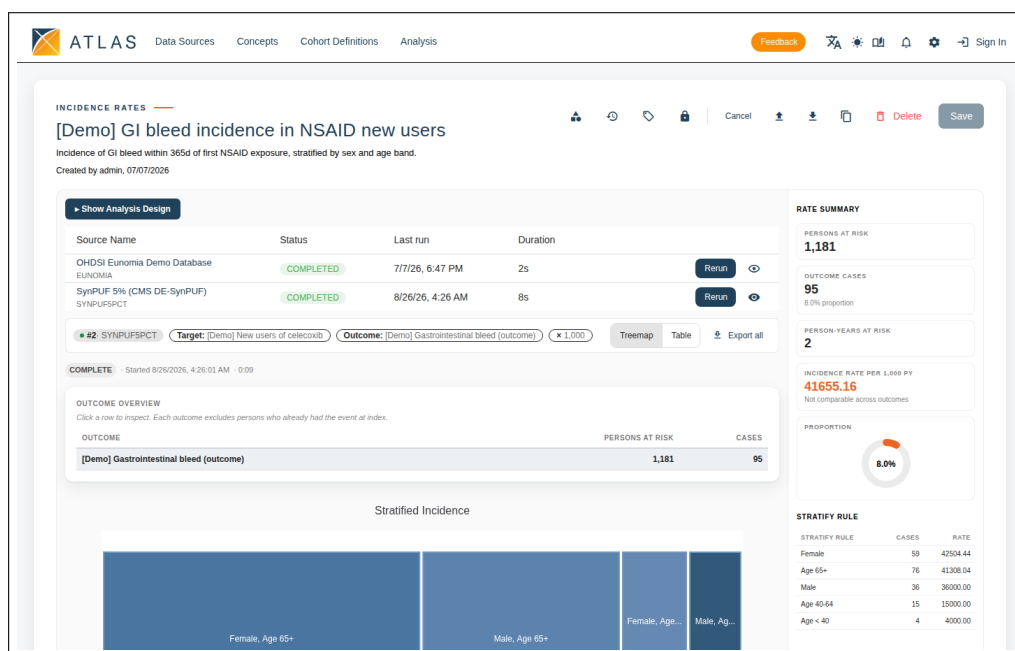


Figure 11.1. Incidence-rate editor with the design hidden. The data-source run table has a **Rerun** button and the eye **View results** icon per source — there is no history icon. Below it sit the Target / Outcome / multiplier chips, the Treemap / Table toggle, and **Export all**; the **Rate summary** rail is on the right. **Show Analysis Design** opens the design rail with Target cohorts, Outcome cohorts, Time at risk, Study window, and Stratify rules.

The time-at-risk window is the most consequential setting in this editor. Different choices (intent-to-treat vs per-protocol, etc.) can swing the rate by a factor of two or more on the same cohort. Report the TAR start and end with your rate and run a sensitivity check; the trade-offs are covered in *The Book of OHDSI* [20].

11.2 Reading the result

The result opens as a three-pane workbench: a collapsible design rail on the left (the same inputs you set when building the analysis), the centre canvas, and a right **Rate summary** rail.

The canvas stacks its elements top to bottom — the data-source run table, a toolbar, a one-line run-meta summary of the selected run, an **Outcome Overview** table, and finally the rates view itself. The **Outcome Overview** is a clickable **Outcome** / **Persons at risk** / **Cases** table: selecting a row switches the rest of the canvas and the **Rate summary** rail to that outcome, which is how you move between outcomes when the analysis carries more than one. A **Treemap** / **Table** toggle on the toolbar switches the rates view between the treemap and the stratified table.

The toolbar also carries **Target** and **Outcome** chips, a multiplier chip (e.g. $\times 1,000$) that opens the multiplier menu, and an **Export all** menu (CSV export is implemented; SVG and PNG are listed as options).

The rates table columns are:

- **Stratify Rule — Summary Statistics:** for the headline row, then one row per stratify rule.
- **Persons** — persons at risk in that stratum.
- **Cases** — outcome occurrences inside the time-at-risk window.
- **(years)** — total time-at-risk in person-years.
- **Rate / N** — the incidence rate, expressed per the multiplier N picked from the toolbar (the menu offers $\times 1$, $\times 10$, $\times 100$, $\times 1,000$, $\times 10,000$ and $\times 100,000$).

The table does not carry a confidence-interval column.

The treemap view shows nested rectangles whose area is proportional to the persons-at-risk in each stratum and whose fill follows the incidence-rate colour scale — the fastest way to spot strata where the rate is unusually high or low without scanning the table row by row.

11.2.1 The Rate summary rail

The right **Rate summary** rail summarises the selected outcome at a glance. It leads with a row of KPI cards — **Persons at risk**, **Outcome cases** (with the case proportion as a percentage), **Person-years at risk**, and **Incidence rate per N PY** for the chosen multiplier — followed by a **Proportion** donut and a stratification mini-table with **Stratify Rule**, **Cases**, and **Rate** columns.

The canvas shows a data-source run table at the top with one row per connected source. Each row carries status, last-run timestamp, duration, and a single primary action button that cycles **Generate** → **Cancel** → **Rerun** as the run progresses, plus the eye icon **View results**, which loads that source's latest completed run into the canvas. Unlike characterizations and pathways, incidence rates have no history icon or **Previous runs** dialog: each source shows only its latest run, and a rerun replaces what you see.

A high rate computed from a tiny denominator looks impressive in isolation. Always look at *persons at risk* alongside the rate; suppress or flag rows where it is below a sensible cell-count threshold.

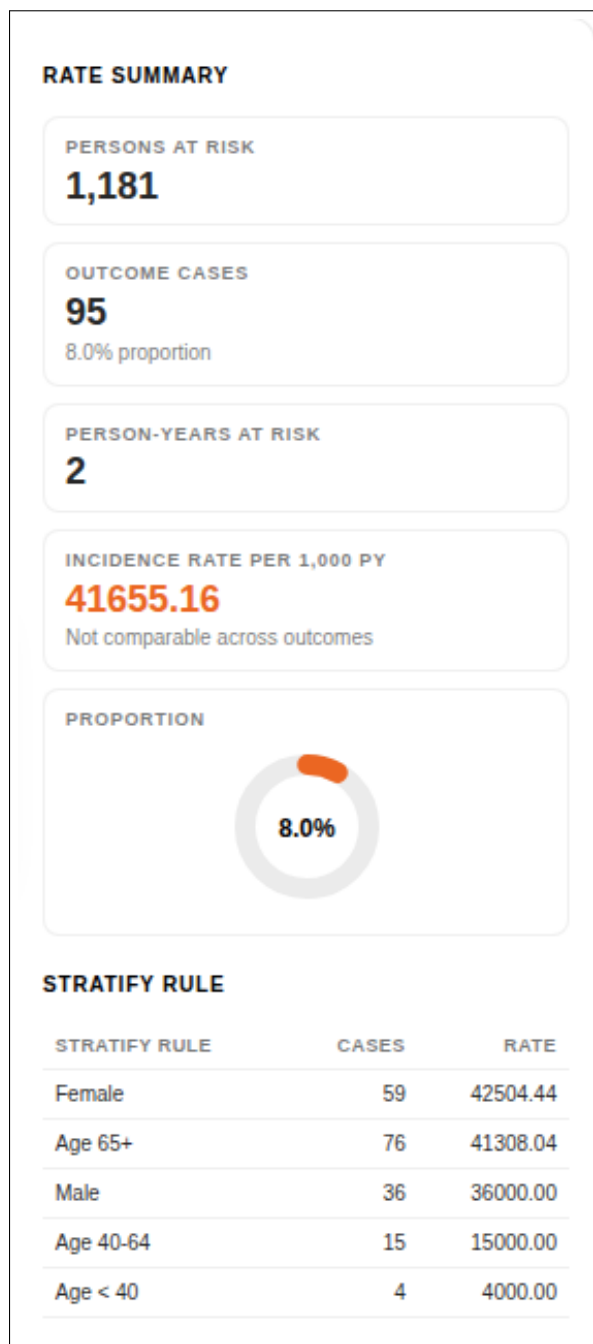


Figure 11.2. The incidence-rate **Rate summary** rail — KPI cards for Persons at risk, Outcome cases with case proportion, Person-years at risk, and Incidence rate per *N* PY; the Proportion donut; and a Stratify Rule / Cases / Rate mini-table.

Profiles

The **Profiles** area lets you look at one specific person's longitudinal record on one specific data source. It is useful for sanity-checking a cohort definition (does my cohort really catch the right people?) and for clinical review of individual cases when the data agreement permits.

12.1 Finding a person

1. Profiles is *not* in the top navigation. Open a profile URL directly (`/profiles`, optionally with a source and person ID as below); inside a cohort definition, clicking a person ID in a cohort sample shows that person's profile in the cohort's report drawer instead (Chapter 8 §8.3).
2. Pick a data source from the dropdown in the page actions, or pass it in the URL as `/profiles/{sourceKey}`.
3. Set the person ID in the inline-editable hero title at the top of the page: edit it in place and commit with Enter or by clicking away (blur). The reload icon next to the **Data source** dropdown refetches the record. The URL becomes `/profiles/{sourceKey}/{personId}` and the timeline loads.
4. Optionally append a cohort ID as a fourth path segment, `.../{personId}/{cohortId}` — the timeline will then highlight that cohort's membership window for the person.

12.2 The timeline view

The main view is a horizontal time axis spanning the person's observation period, with one swim-lane per OMOP domain in which the person has records — drug, condition, visit, procedure, observation, measurement, device, specimen, condition era, drug era, dose era, and death, in that order, omitting domains the person has no records in. The lanes are fixed for the person, so hiding a domain or narrowing the range does not make rows jump. The x-axis is labelled **Day (0 = cohort entry)**: every record is plotted at its day index, with day 0 — cohort entry — marked by a dashed *Cohort entry* markline. The axis is still a day index, not a calendar date.

Each lane is a sequence of marks at the day of each record in that domain; hovering a mark reveals the concept name and the day number. Records that fall on the same day are spread slightly up and down within their lane so they do not hide one another;

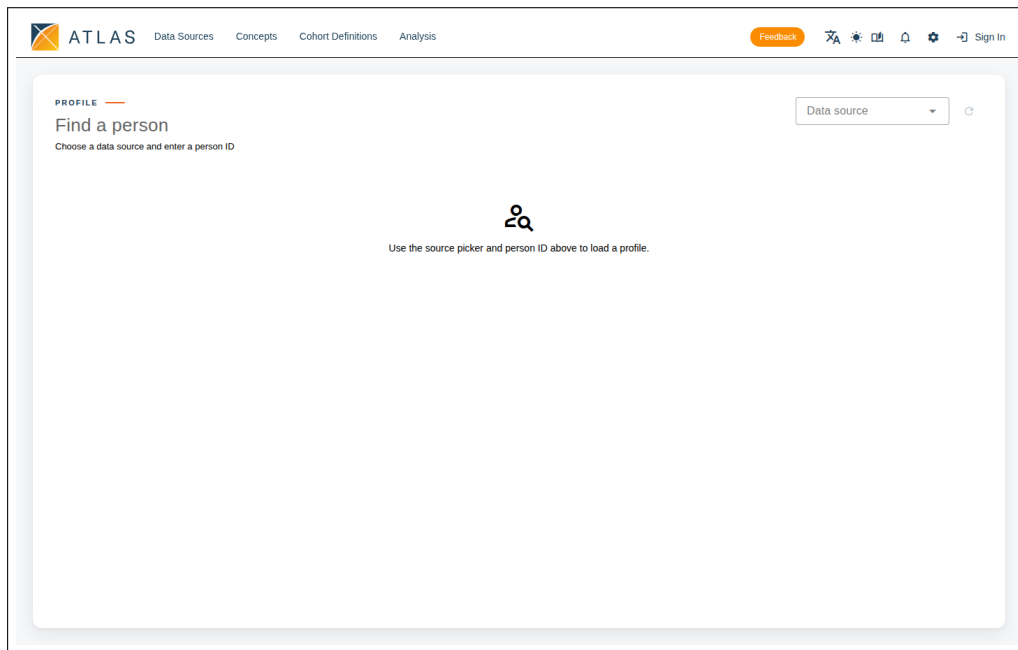


Figure 12.1. Profiles landing page. Pick a source from the dropdown top-right, then type a person ID (or arrive via a deep-linked URL) to load the timeline.

the offset is stable, so a mark stays put when you filter. A row of filter chips above the chart lets you hide domains.

To zoom in, pick the horizontal range-select tool in the chart's toolbox and drag across the period you care about: the axis re-scales to exactly that day range, and the events table below narrows to records that overlap it (a record that starts before the range but is still ongoing inside it is kept). The active range appears as a closable chip ("Day *a* → Day *b*"); close the chip to zoom back out to the full observation period.

A separate events table below lists every record in detail, with **Concept Id**, **Concept Name**, **Domain**, **Start Day**, and **End Day** columns — day numbers rather than source codes or calendar dates — plus a **Search events** box to filter it.

For phenotype validation, set the cohort context before opening the profile (e.g. via the URL form `/profiles/{sourceKey}/{personId}/{cohortId}`). The cohort identifier appears as a closable chip in the page header so you know which cohort the records are being read against.

12.3 The rest of the profile page

12.3.1 The Demographics strip

A **Demographics** strip sits at the top of the profile, summarising the person in four fields: **Gender**, **Birth Year**, **Age** (its tooltip names the reference date — the cohort index date when a cohort is in the URL), and an **Events** count of the records plotted on the timeline.

GENDER ♀ Female	BIRTH YEAR 1963	AGE 1	EVENTS 90
--------------------	--------------------	----------	--------------

Figure 12.2. The Demographics strip — Gender, Birth Year, Age, and Events count — across the top of the profile page.

12.3.2 The Observation periods band

Below the demographics is the **Observation periods** band: a thin, full-width strip with one rectangle per observation period, so a person with a gap in coverage shows two or more separated rectangles at a glance.

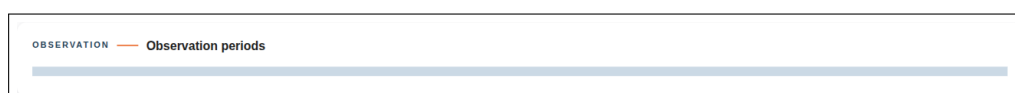


Figure 12.3. The **Observation periods** band for a person with a single, continuous observation period, which fills the strip; a gap in coverage would show as two or more separate rectangles.

12.3.3 The Highlights panel

The **Highlights** panel is a right-hand aside with two tabs, **Concepts** and **Sets**.

The **Concepts** tab lets you search the person's own concepts (every word you type must match, in any order) and assign each match one of six highlight colours; the matching marks recolour on the timeline so a chosen concept stands out against the rest. A **Clear all highlights** action resets every assignment.

The **Sets** tab is disabled unless the URL carries a cohort ID. With a cohort in context it lists that cohort's concept sets, but their colour dots are inactive (tooltip "*Concept-set highlighting coming soon*"), so concept-set highlighting does not work yet.

12.4 Validating a phenotype using Profiles

Profiles is the lightweight half of phenotype validation. The typical workflow is:

1. Generate the cohort against the source you want to validate on (Chapter 7).
2. Pick a small set of person IDs from the cohort — either by querying the cohort table directly, exporting the cohort result, or asking your administrator for a representative sample. Around a hundred persons is enough to find gross errors.
3. For each sampled person, open their profile with the cohort context set (`/profiles/{sourceKey}/{personId}/{cohortId}`). Read the records in the baseline window: do they look like patients with the condition? Read the records around the index date: is the entry event clinically plausible?
4. Tally what you see. Any obvious mismatches (a "type 2 diabetes" cohort that includes people with no glucose-lowering therapy and no diagnostic codes outside the index event) is a signal the rule is too loose; any common false negatives (clear diabetic patients excluded by an over-tight rule) is a signal it is too strict.

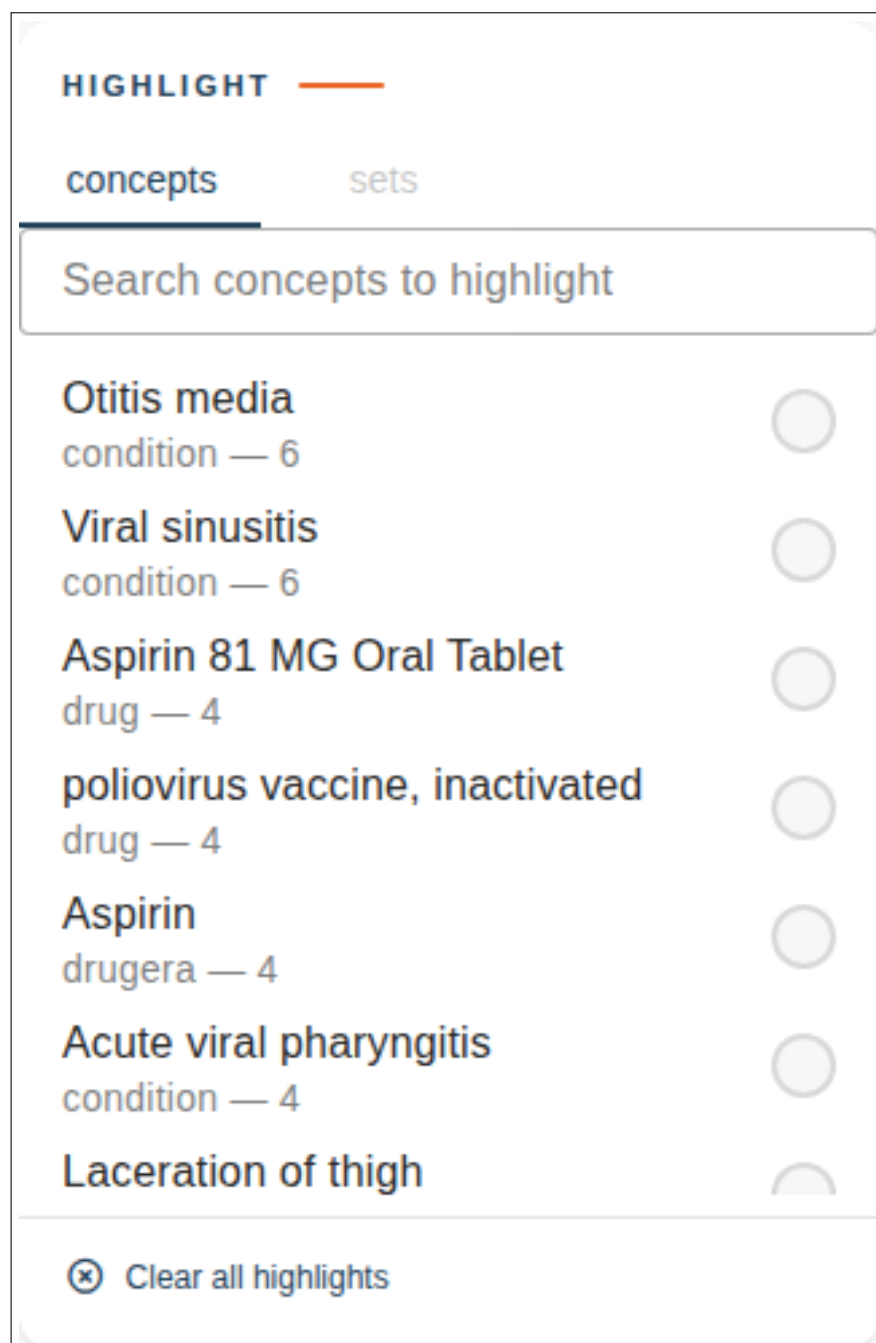


Figure 12.4. The Highlights panel with its Concepts and Sets tabs; the Concepts tab shows a search box, colour swatches, and a Clear all highlights action.

5. Adjust the cohort definition, save a new version (Chapter 14), regenerate, and resample.

Note

A sample of 100 charts is enough to find gross errors in a phenotype but not enough to estimate sensitivity, specificity, or positive predictive value with useful precision. For that, see the PheValuator section below.

12.5 Going further: PheValuator

ATLAS v3.0 itself does not formally evaluate a phenotype. For quantitative sensitivity, specificity, and PPV against a probabilistic gold standard, OHDSI ships *PheValuator* [18, 37, 38] as part of the HADES R suite [23]. The workflow is broadly: build the candidate phenotype and a tightly-specified "xSpec" cohort in ATLAS, export both as JSON, and run PheValuator in R against your CDM database — it fits a predictive model that yields per-source operating characteristics with confidence intervals.

The methodology, including how to construct the xSpec cohort, how to size the evaluation cohort, what target metrics make a phenotype publishable, and how PheValuator complements the CohortDiagnostics [28, 30] package, is covered in the package vignette and in *The Book of OHDSI*, chapter 16 "Clinical Validity" [22]. This manual documents what ATLAS provides; the evaluation step lives in R and outside the scope of the UI.

Feature analyses

A *feature analysis* is a reusable definition of a covariate or summary statistic. Characterizations (Chapter 9) pull their content from feature analyses, and once a feature is defined once it can be reused across studies — which is what makes characterizations comparable to each other.

13.1 Built-in versus custom features

ATLAS v3.0 surfaces the feature analyses defined in WebAPI — including the standard library that corresponds to the OHDSI `FeatureExtraction` [15] R package (part of the HADES suite [23]): demographic features, condition occurrence in fixed windows, drug exposure in fixed windows, visit counts, Charlson index components, and so on. Most users start by combining these in characterizations rather than authoring new ones.

These built-in analyses are *presets*. They are read-only: in the list a preset's name is plain text rather than a link, its edit action is absent, and its copy and delete actions are disabled. A preset is simply the name of a `FeatureExtraction` covariate setting, so there is nothing to edit; link it to a characterization and WebAPI computes it. Presets cannot be created from the UI.

13.2 Finding a feature analysis

Open `Analysis` and switch to the **Feature Analyses** tab of the Analysis Hub — the hub's default first tab (Chapter 8). The list has columns **Id**, **Name**, **Analysis type**, **Domain**, **Stat Type**, **Created By**, **Created**, **Updated**, and **Actions**, and is sorted by **Updated**, newest first.

Instead of a plain search box the list has a facet filter:

- a **Filter results** box that matches every word you type, in any order, against the name, description, type and domain;
- a **Filters** menu with facets **Type** (*Criteria set*, *Custom*, *Preset*), **Domain**, **Created** and **Updated** (*This Week*, *Last Week*, *2+ Weeks Ago*), **Author**, and **Designs** (*My designs* / *Other designs*). Every value shows how many analyses carry it, e.g. *Criteria set* (12).

The same facet filter appears in the characterization editor's **Choose a Feature Analyses...** picker (Chapter 9), so a search you learn here works there too.

13.3 Creating a feature analysis

New feature analysis is a drop-down. Pick the kind of analysis you want to build:

Prevalence Criteria one or more named criteria; each produces a binary covariate per person (present / absent), reported as a count and percentage.

Distribution Criteria one or more named criteria whose matching events are aggregated into a continuous covariate, reported as mean, standard deviation, median, and quartiles.

Custom SQL a SQL query you write yourself, for features that cannot be expressed as criteria.

The choice is fixed at creation: the editor shows it as a read-only chip at the top of the form (*Prevalence Criteria*, *Distribution Criteria*, *Custom*, or *Preset* for a built-in analysis) and there is no way to switch type afterwards. Every type has **Name**, **Description**, and **Domain** (default `CONDITION`) fields.

13.3.1 Prevalence Criteria

Add Criteria feature appends a criteria feature; until you add one the form reads *No criteria features yet*. Each feature is a **Criteria name** field plus a criteria group edited exactly like a cohort inclusion rule (Section 7.5): **Add criteria to group...** adds a criterion, **Add Nested Group** adds a sub-group, the vertical label on the left switches between *all*, *any*, *at least*, and *at most*, and each criterion carries its concept set, attributes, cardinality, and a temporal window relative to index. A person gets the covariate when the group is satisfied.

13.3.2 Distribution Criteria

Here **Add Criteria feature** opens a menu of criterion types (each with its domain as a subtitle, or *All domains*) plus **Demographic Criteria** — *"Demographic criteria are not windowed"*. Each feature row has a **Criteria name**, an aggregate drop-down that decides what number is summarised per person, a delete action, and the editor for that one criterion. The aggregate defaults to **Events count**; the other choices depend on the criterion type — for a drug exposure, for example, **Distinct start dates** and the plain, max, min and average of **Duration**, **Refills**, **Quantity** and **Days supply**. The criterion's temporal-window chip opens the same popover as in the cohort builder, with **Quick Presets** and **Add Temporal Window**; flag chips below it toggle *restrict to the same visit occurrence* and *ignore observation period*.

13.3.3 Custom SQL

The **Analysis SQL:** block frames your query in a fixed wrapper:

```
SELECT covariate_id, covariate_name, concept_id,
       sum_value, average_value FROM (
  -- your SQL here
)
```

The screenshot shows the 'New' form for a 'Prevalence Criteria' feature analysis. The form is titled 'New' and has a toolbar with icons for Concept Sets, Configuration, Cancel, Duplicate, Delete, and Save. The 'Prevalence Criteria' section includes a 'Name' field, a 'Description' field, and a 'Domain' dropdown menu set to 'CONDITION'. Below these fields is a '+ Add Criteria feature' button. The 'Criteria name' field is empty. A criteria group is shown with a blue chip labeled 'No events in group. Add events to build criteria logic.' and buttons for '+ Add criteria to group...' and '+ Add Nested Group'.

Figure 13.1. A new **Prevalence Criteria** feature analysis. The read-only type chip sits above **Name**, **Description**, and **Domain**; below, **Add Criteria feature** has added one feature with a **Criteria name** field and an empty criteria group with **Add criteria to group...** and **Add Nested Group**. The toolbar carries the Concept Sets and **Configure access** icons, **Cancel**, Duplicate, **Delete**, and **Save**.

The screenshot shows the 'New' form for a 'Distribution Criteria' feature analysis. The form is titled 'New' and has a toolbar with icons for Concept Sets, Configuration, Cancel, Duplicate, Delete, and Save. The 'Distribution Criteria' section includes a 'Name' field, a 'Description' field, and a 'Domain' dropdown menu set to 'CONDITION'. Below these fields is a '+ Add Criteria feature' button. The 'Criteria name' field is empty. An aggregate dropdown menu is set to 'Events count'. A criterion card is shown with a blue chip labeled 'a drug exposure of', a '+ Select Concept Set' button, and a '+ Add attribute...' button. Below the criterion card are two flag chips: 'event starts between all days Before and all days After index start' and 'restrict to the same visit occurrence'.

Figure 13.2. A new **Distribution Criteria** feature analysis with one drug-exposure feature: the **Criteria name** field, the aggregate drop-down (here **Events count**), the criterion card with **Select Concept Set**, the temporal-window chip, and the two flag chips.

Populate Sample SQL fills the box with a working example and **Copy Analysis SQL Template To Clipboard** copies the template. Under the box, **Available variables:** lists the parameters you may reference: @cdm_database_schema, @cohort_table, @cohort_id, @analysis_id, and all variables specified in the characterization's parameters.

13.3.4 Concept sets and the toolbar

The criteria types keep their own concept sets. The **Concept Sets** icon on the toolbar (with a count badge) opens the list of the analysis's concept sets, with **New Concept Set** and per-row edit and delete; editing opens the concept-set editor as an embedded drawer (Section 6.13). Inside a criterion, **Select Concept Set** opens the concept-set picker (Section 6.12).

The rest of the toolbar follows the other editors (Section 2.4): **Configure access** to share the analysis (Section 15.4), **Cancel**, Duplicate, **Delete**, and **Save**. **Save** is disabled until the analysis has a name, and hovering it explains why. Leaving with unsaved changes asks for confirmation.

Note

A typical baseline bundle combines the two stat types: demographics (age as distribution, gender as prevalence), condition / drug / procedure occurrences in the four standard time windows (prevalence), visit and observation counts (distribution), and Charlson comorbidity index components. This default is what most published OHDSI studies use unless the question specifically calls for something narrower — and all of it is available as presets, so reach for a custom analysis only for what the presets do not cover.

13.4 Reuse

Once a feature analysis is saved, every characterization that references it picks up the latest definition automatically. Unlike cohorts and concept sets, feature analyses do not carry their own version history or tags in the UI. Treat each save as the canonical state; if a study depends on the exact definition, export the characterization (which embeds its feature analyses) alongside the results.

PART V

Operations

Versions

Cohort definitions, concept sets, pathways and incidence rates are versioned. Every save creates a new version; older versions remain readable, and you can preview them, annotate them, or copy one into a new artefact. Feature analyses do not carry version history; their definitions are referenced directly by the characterizations that use them.

Note

Characterizations are versioned by WebAPI, but the characterization editor does not show that history yet: its Versions icon opens a placeholder dialog. Export the characterization JSON if you need to keep a particular state.

14.1 Where versions live

Each editable artefact carries a history icon (MDI-HISTORY) in its toolbar that opens a **Versions** dialog. The dialog lists versions in reverse chronological order; each row shows the version number, author, save timestamp, and a short comment if one was supplied, plus three actions: **Preview**, **Add** (or **Edit comment** when a comment is already set), and **Create a copy**. A small badge on the toolbar icon shows the current version count at a glance.

14.2 Previewing a version

Click a row in the Versions dialog to enter *preview mode*. The URL changes to e.g. `/cohortdefinition/{id}/version/{N}` and the editor renders the historical state. For cohorts, pathways and incidence rates preview mode is read-only — you cannot accidentally overwrite the past: **Save** is disabled, and hovering it explains “*You are previewing an earlier version. Return to the current version to save.*” To leave preview mode, use the cohort editor’s **Back to current** banner button, or the **Back to current version** toolbar icon in the pathway and incidence-rate editors, or navigate to `.../version/current`.

Concept sets behave differently. A previewed concept-set version loads into the normal editor with no preview banner, and **Save** stays enabled: saving writes the previewed content back as a new version on top of the current one. That is a quick way to restore an old concept set — and an easy way to do it by accident. Close the editor without saving if you only meant to look.

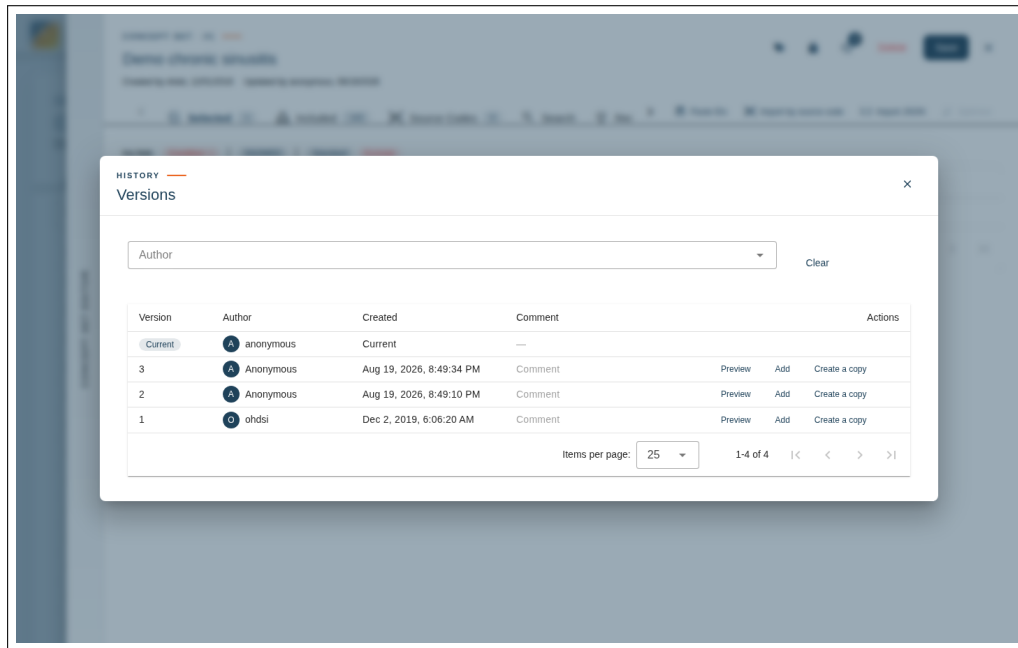


Figure 14.1. Versions dialog. Each historical version carries an author, timestamp, and optional comment, with **Preview**, **Add**, and **Create a copy** actions.

14.3 Restoring a version

Apart from the concept-set behaviour above, there is no one-click “make this the current version” action. To get an older state back, use **Create a copy** on its row (in the Versions dialog or while previewing it): ATLAS creates a new, separate artefact from that version and opens it. The original artefact and its history are untouched.

When the artefact itself must carry the older content — because other analyses reference its ID — carry the change forward by hand. For a cohort, the quickest route is to preview the old version, copy its JSON from the **Export** menu, return to the current version, and paste it into **Export** → **View / Edit** (Chapter 7); saving then appends a new version with the older logic.

Note

Either way, nothing is deleted. The version you started from stays in the list, so a restore you regret can be undone the same way.

14.4 Pinning a published study

When a cohort or concept set is referenced in a published paper or a regulatory submission, pin the exact version in the citation: cohort definition ID and version number, not just “the latest version of cohort 482”. This is the cheapest way to guarantee a reader on a different ATLAS instance — or on the same one a year later — can recreate the exact analytical artefact. The version-preview URLs (`/cohortdefinition/{id}/version/{N}`) are stable; copy them straight into the paper’s supplementary materials.

Note

When the cohort itself was reused from the OHDSI Phenotype Library, pin *two* version numbers: the local cohort version and the Phenotype Library release DOI it was imported from. Definitions accepted into a Library release are immutable for that release, but the same definition can be marked deprecated [D] or in error [E] in a later release; quoting the release DOI insulates your study from those later changes [26].

Administration

This chapter is for users with administrative permissions. Most researchers can skip it; come back when you need to grant a colleague write access, manage data sources and tags, or check on long-running jobs. The one section every researcher should read is Section 15.4: sharing your own cohorts and analyses with colleagues does not need an administrator.

15.1 The configuration panel

Administrative controls live in a side panel that opens from the **cog icon** in the top right of the navigation bar. The cog itself is gated: it is shown only to users who hold at least one of `admin:cache`, `admin:source`, `admin:tags`, or `admin:security`, or when a plugin contributes an admin tab. A researcher account usually does not see it at all. Once open, the panel shows one tab per area the user may see:

Cache (`admin:cache`). The configuration cache and the per-source patient caches that live counts rely on. See "Caches and TrexSQL" below.

Data Sources (read access to at least one source — `read:source`, `write:source`, `admin:source`, `source:*:access`, or a per-source grant). The CDM databases/WebAPI is connected to; add, edit, test and share them here. See "Managing data sources" below. Note that source access alone does not show the cog: a user who holds only `read:source` or `write:source` has no way into the panel.

Tags (`admin:tags`). Create, rename and merge the tags used across cohorts, concept sets and analyses.

Permissions (`admin:security`). The entry point for role and permission management. Selecting a role here opens the dedicated role page (covered below).

Plugin tabs. Plugins can add their own admin tabs after the built-in ones (Chapter 16), each with its own permission requirement.

Note

The configuration panel is a side overlay rather than a separate page, so you can quickly check cache or source state without leaving the page you were on. Long-running jobs have their own drawer, opened from a separate navbar icon (see "Jobs and long-running tasks" below).

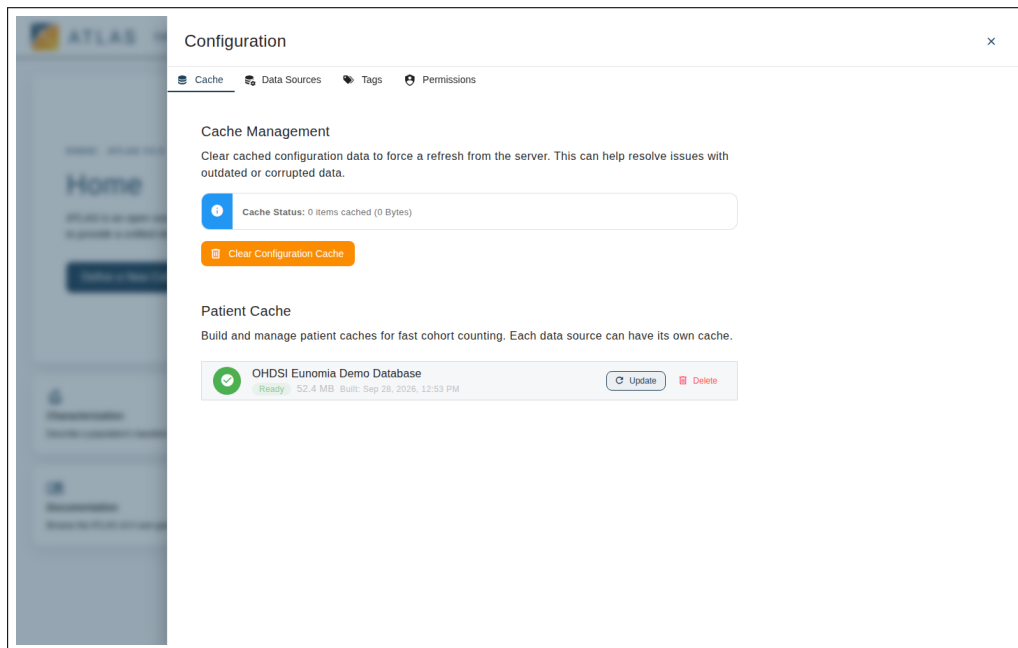


Figure 15.1. Configuration panel on the **Cache** tab (as an administrator sees it). The **Cache Management** card is at the top; the **Patient Cache** card below has a per-source row with the cache state — here *Ready*, with its size and build time, after a first build — and the **Update** and **Delete** actions (**Build Cache** before the first build).

15.2 Caches and TrexSQL

Several ATLAS pages back queries that are expensive to recompute on every visit — vocabulary searches, source-level descriptive reports, cohort report rollups. The configuration panel’s **Cache** tab has two cards. **Cache Management** shows the size of the browser-side configuration cache (*Cache Status: N items cached*) with a **Clear Configuration Cache** button. **Patient Cache** lists one row per source with a status chip (**Ready**, **Building**, **Stale**, **Error**, **Not Built**), the patient count and size, the build date, and any error text. **Build Cache** starts the first build; once built the button reads **Update** (**Building...** while running), and **Delete** removes the cache after a **Delete cache?** confirmation. A cache whose data source has been deleted is flagged **No dataset** and can be deleted to reclaim disk space.

Behind these cards WebAPI keeps two layers of cache:

Patient cache. A per-source pre-aggregation that descriptive reports (Chapter 4, cohort **Reports** tab in Chapter 7) read from, and that the cohort builder uses to show *live patient counts* as a researcher edits a definition — saving a full **Generate** round-trip on every change. The cache sits in the source’s results schema. Initial build is a long-running job; once built, it is reused until the source data is refreshed.

TrexSQL cache. A query-level cache used by WebAPI’s SQL execution layer. Repeated vocabulary queries and concept-set lookups hit the cache rather than the database, so the second time a user opens the same concept set the response is essentially

instant. The cache lives on disk on the WebAPI host (path set by the WebAPI environment variable `TREXSQL_CACHE_PATH`) and is shared across users.

Rebuild a cache after:

- the underlying CDM source has been refreshed (so the patient cache reflects the new data),
- the OHDSI vocabularies on the source have been updated (so the TrexSQL cache does not return stale concept information), or
- the cache was built against an older WebAPI version whose query shape has changed.

Cache builds run as jobs and show in the Jobs drawer; they can take from minutes to hours depending on the size of the source.

15.3 Managing data sources

The **Data Sources** tab lists every CDM database WebAPI is connected to. It is no longer read-only: users with the right permissions add, edit, test, and delete sources here without touching WebAPI's tables. Everyone who can open the configuration panel and read at least one source sees the tab; controls a user may not use are disabled, and hovering one explains *"You do not have permission for this action"*.

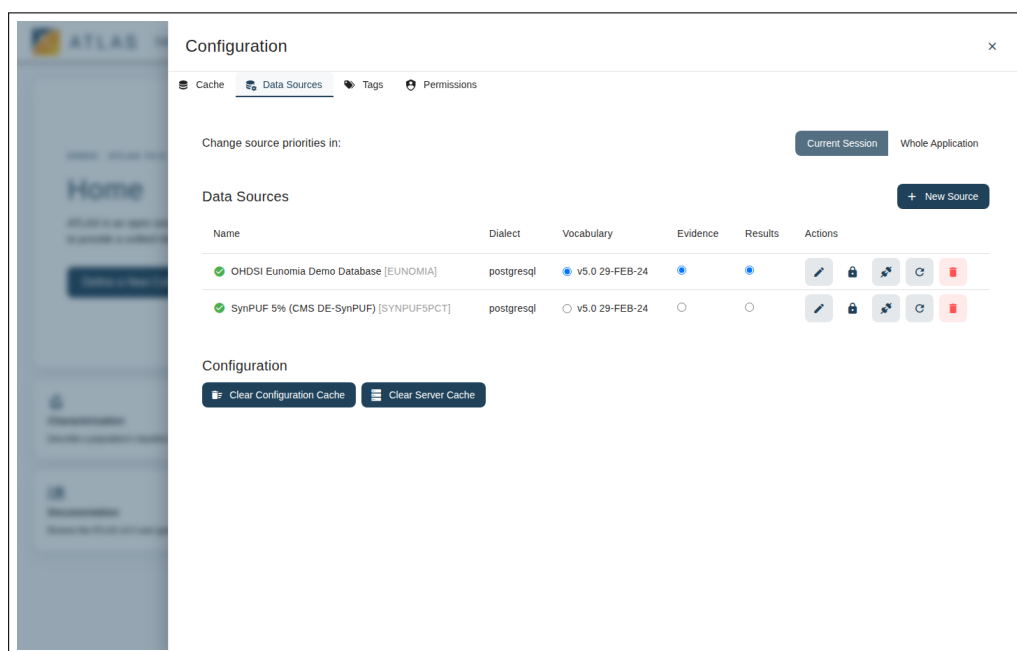


Figure 15.2. The **Data Sources** tab of the configuration panel. The **Change source priorities in:** toggle sits above the table, which lists each source with its dialect and the Vocabulary / Evidence / Results priority radios; each row carries Edit, **Configure access**, **Check connection**, **Refresh cache**, and Delete; the Configuration card below holds **Clear Configuration Cache** and **Clear Server Cache**.

Source priorities. The **Vocabulary**, **Evidence**, and **Results** radio columns choose which source ATLAS uses by default for each purpose (for example, which vocabulary concept searches run against). The choices are stored in this browser only. The **Change source priorities in:** toggle (**Current Session** / **Whole Application**) currently has no effect.

New Source / Edit. The source dialog (titled **New Source** or **Edit**) asks for **Name**, **Source Key:** (letters, numbers, hyphens, underscores; it cannot be changed after creation), **Source dialect:**, **Connection string:**, **Username:**, and **Password:**, plus Kerberos options or a BigQuery service account key file where the dialect needs them. The **Source daemons** table (**Enabled** / **Type** / **Schema**) enables the CDM, Vocabulary, Results, and other daemons and sets each schema. A **Check connection** checkbox tests the connection when you save. **New Source** needs `write:source` or `admin:source`.

Check connection asks WebAPI to connect to the source and reports success or failure.

Refresh cache clears and rebuilds WebAPI's cached descriptive results (the Data Sources reports) for that source — use it after the source's Achilles results were regenerated.

Configure access shares the source with roles, READ or WRITE, exactly like any other asset (Section 15.4).

Clear Configuration Cache drops the source selections this browser has cached, so they are re-read from the server. It no longer signs you out.

Clear Server Cache (after a confirmation) clears WebAPI's server-side caches for everyone.

15.4 Sharing an asset: Configure access

Every saved cohort, concept set, characterization, feature analysis, pathway, and incidence rate — and every data source — has a **Configure access** button (a lock icon) in its editor toolbar or table row. It lets the owner, or anyone with write access, grant other roles access to that one asset, without an administrator changing global permissions. The button only works on a saved asset: it is hidden until the first save on pathways, incidence rates, characterizations and concept sets, and disabled until then on cohorts and feature analyses (a cohort's is also disabled while you preview an older version). If the dialog is opened without an asset it warns *"Save the asset before assigning permissions."*

The dialog opens with *"Grant READ or WRITE access to this asset by assigning roles. WRITE implies READ."* and has two sections, **Read** and **Write**. Each has:

- an **Add READ access to role** / **Add WRITE access to role** field with a **Search roles...** box;
- an **Add** button, enabled once a role is picked;
- a table of the roles that already hold that access, each with **Revoke**.

Changes apply immediately. The people in a role pick up the new access on their next page change — ATLAS re-reads the user's permissions on every navigation, so nobody has to sign out and in again. To share with one person, grant access to their personal role (WebAPI creates one per user, named after the login).

15.5 Roles and permissions

ATLAS v3.0 uses role-based access control. A *role* is a named bundle of *permissions*; a user is a member of zero or more roles. Open the configuration panel and pick **Permissions**, or navigate directly to `/config/roles` to see the role list.

The **Permissions** tab depends only on `admin:security`. The `enablePermissionManagement` key in `config-local.json` is accepted but has no effect in this build.

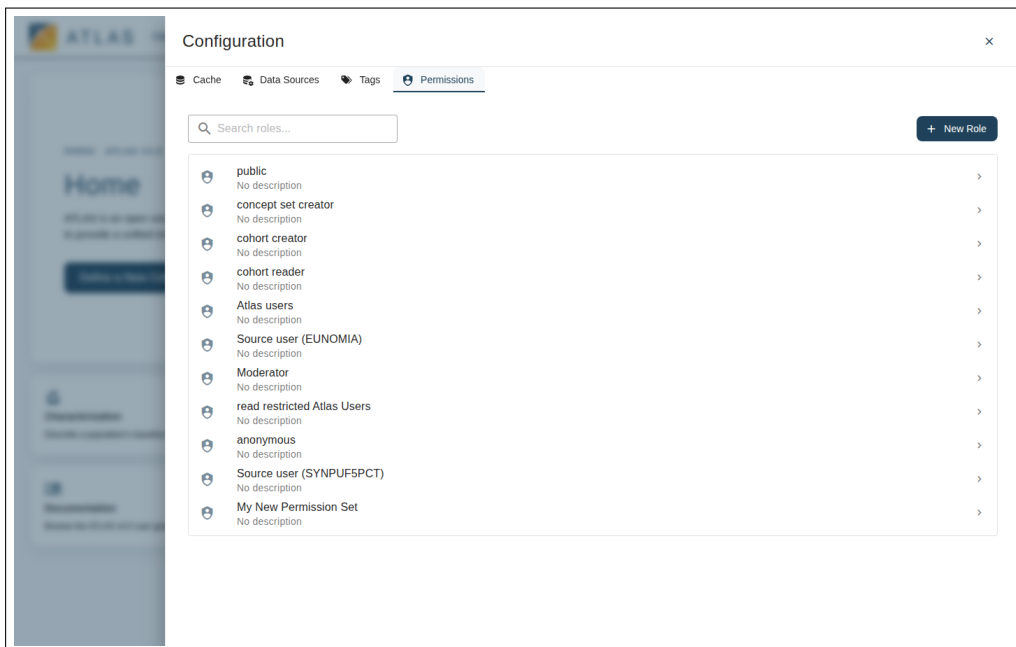


Figure 15.3. Roles list inside the **Permissions** tab of the configuration panel. Click any role to open the role-detail page (**Users** / **Permissions** / **Utilities** tabs); **New Role** starts a fresh empty role.

15.5.1 Built-in roles

A typical deployment ships a few built-in roles that you should not delete: `public` (default for unauthenticated visitors where allowed), `user` (read access to most artefacts), and an admin-style role with full write access. Your organisation may have added project-specific roles such as a per-study analyst role.

15.5.2 Editing a role

Click a role to open its detail page at `/config/roles/{id}`. The page lets you edit the role's name and description inline at the top, then exposes three tabs in this order (**Users** is the default):

Users The users currently in the role. Add or remove members here. Changes take effect at the user's next request, usually immediately.

Permissions The flags that decide what the role's members can do — view cohorts, write cohorts, generate against specific sources, manage roles, and so on. Most permissions are per-source, which is how you give a contractor access to one database without exposing the rest.

Utilities Import, export and clone operations on the role itself — useful when you want to copy a vetted role bundle to a sibling deployment.

15.5.3 Creating, importing and deleting a role

The role list carries three companion dialogs:

New role opens a small dialog with a name and description; the new role starts empty and you flesh it out from the detail page above. Pick a clear, hyphenated name (e.g. cardiology-analyst). Role names are visible to all users, so avoid embarrassing or project-internal language.

Import reads a role JSON file produced by another ATLAS instance's **Utilities** tab. It is the right way to replicate a vetted role bundle from one deployment to another.

Delete removes the role after a confirmation dialog. Built-in roles are typically protected; only delete a role you own.

Removing a permission from a role takes effect immediately for every member of that role. If a long-running cohort generation is in flight and the role member loses generation permission mid-flight, the run will fail. Coordinate sensitive role changes with your users.

15.5.4 Permission reference

The permission strings ATLAS checks fall into four families. The table below lists the ones the frontend reads when deciding what to render.

Resource verbs `read:<resource>`, `write:<resource>`, `create:<resource>`, where *resource* is one of `cohort-definition`, `conceptset`, `cohort-characterization`, `feature-analysis`, `pathway`, or `incidence`. Read gates the list and editor; write gates **Save** and **Delete**; create gates the **New** ... button on the list page.

Per-entity grants alongside the global verbs, WebAPI returns a per-entity record for each artefact the user has been granted access to individually: an `accessTypes` list (containing `READ` and/or `WRITE`) plus a separate `isOwner` boolean. Ownership implies both read and write; an explicit `WRITE` entry overrides a missing global `write:` permission. This is how you give a researcher full control over their own cohorts without granting blanket `write:cohort-definition`.

Admin `admin:cache` (Cache tab and patient-cache rebuilds), `admin:source` (full control of the Data Sources tab), `admin:tags` (Tags tab and tag-management actions on entities), `admin:security` (Permissions tab, role CRUD; also acts as a global read+write override on every artefact). Holding any of these makes the cog icon visible in the navbar. Source access stays gated by `admin:source` / `source::access`, not by `admin:security`.

Source access `source::access` gates which CDM sources a user can run cohort generations and reports against. Per-source variants (`source:<key>:access`) can grant access to a single source without the wildcard. `read:source` and `write:source` decide who sees and who edits the Data Sources tab; per-source READ / WRITE grants made through **Configure access** work on top of them.

Jobs `job:execution:get` shows the Jobs icon in the navbar.

Note

A common starter-role bundle for an analyst is `read:*`, `write:*` on the artefact resources plus `create:*`, `source::access`, and `job:execution:get` — everything except `admin:*`. A read-only viewer role drops the `write:*` and `create:*` entries. Per-entity grants then layer on top to share specific cohorts with collaborators outside the analyst pool (Section 15.4).

15.6 Jobs and long-running tasks

Users with `job:execution:get` see a Jobs icon in the navbar, just left of the cog. It opens the **Jobs** drawer, the status view for every long-running task in the system: cohort generations, characterization runs, incidence-rate computations, and pathway analyses (patient-cache builds are tracked in the **Cache** tab instead). The table's columns are **Execution Id**, **Type**, **Job Name**, **Status** (**RUNNING**, **COMPLETED**, **FAILED**, ...), **Author**, start and end times, and duration.

Refresh Jobs re-fetches the list from WebAPI — useful when you are watching a long generation finish.

Filtering by status lets you focus on running jobs or recently failed ones.

The job name links to the asset that owns the job (the cohort, characterization, and so on). The drawer has no log view; SQL errors are in the WebAPI server log.

The Jobs drawer is the right place to diagnose "why is my generation taking so long" and "did my characterization actually finish". For administrators, scanning the failed-jobs filter periodically catches WebAPI-side problems (connection timeouts, out-of-memory errors) before users notice them.

15.7 Audit information

ATLAS v3.0 stores the author and last modifier on every artefact. Each editor header shows them (*Created by ..., date · Updated by ..., date*), and the list views carry **Id**,

Created, and **Updated** columns (plus the author). Together with the version history (Chapter 14) this is the basic audit trail. Deeper audit logs — of generation runs and login events — live in the WebAPI database and are typically inspected directly there rather than through the UI.

Plugins

ATLAS v3.0 supports plugins — independently developed front-end modules that appear inside the application as additional navigation entries and pages. From a user's perspective a plugin behaves like any other area of ATLAS v3.0; from an administrator's perspective it is a separately released JavaScript bundle that the host loads at runtime.

16.1 What plugins look like

When your deployment has plugins installed, they can appear in several places:

- as entries in the top navigation, alongside the built-in ones;
- as extra reports in the **Data Sources** sidebar, grouped under the plugin's own heading (Chapter 4);
- as extra tabs after the four built-in tabs of the **Analysis** hub;
- as extra tabs in the configuration panel;
- as extra entries in the user menu;
- in the single floating-button slot the host provides (`pythia.fab`; see `fabMounts` below).

The content area is replaced with the plugin's own UI when you select it. Plugins share the active session, so you do not log in again, and they receive the currently selected data source automatically.

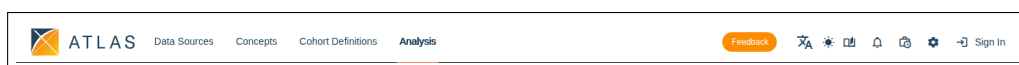


Figure 16.1. Top navigation on a deployment without navigation plugins; plugin entries would appear among the built-in entries (**Data Sources**, **Concepts**, **Cohort Definitions**, **Analysis**), at the position the manifest asks for.

16.2 Common plugin categories

A given deployment may ship none, some, or all of these:

- **Custom analytics.** Statistical workflows specific to a research network, exposed as a plugin so they live alongside ATLAS v3.0's built-in analyses without forking the application.
- **Connectors.** Bridges to external systems — a redcap export, a SAS dataset writer, a registry submission tool.

- **Visualisations.** Dashboards or chart libraries that consume cohort or characterization output.

16.3 Trust and isolation

Each plugin runs in its own sandbox with a scoped message bus to the host. Plugins receive only the auth context and the API surface the host explicitly exposes; they cannot reach into stores they have not been granted. From a research integrity standpoint this matters, because a plugin that produced study results becomes part of the chain of custody — knowing exactly what version of which plugin produced a chart is something your audit process should care about.

The message bus is also how a plugin asks the host to do things on its behalf. Two requests work today: `navigation(request, navigation:back)` and `conceptSet:choose`, which opens ATLAS's own concept-set chooser. The bus also defines `notification:show` and `data:request`, but in this build the first only writes to the debug log and the second always answers with an empty `{success: true, data: {}}`. The host runs the chooser dialog and answers with the user's pick as `{conceptSetId, name}`, or null if the user dismissed it; an optional title in the request labels the dialog.

Note

There is no in-application admin UI for installed plugins; the plugin manifest is a static `src/config/plugins.json` on the host, set at build or deploy time. If you are reviewing a publication that used ATLAS v3.0, asking the deployment operator for that file — and the build versions of the plugins it references — is a fair request.

16.4 The `plugins.json` manifest

The host loads its plugin manifest from `/config/plugins.json` at startup; the file is validated with a Zod schema (`PluginManifestSchema` in `src/models/PluginModels.ts`). A minimal manifest looks like this:

```
{
  "version": "1.0",
  "plugins": [
    {
      "id": "hello-world-plugin",
      "name": "Hello World",
      "version": "1.0.0",
      "entryPoint": "hello-world-plugin/index.system.js",
      "menuItems": [
        {
          "id": "main",
          "name": "Hello World",
```

```

        "route": "/plugins/hello-world-plugin/main",
        "icon": "mdi-hand-wave",
        "order": 0
      }
    ],
    "metadata": {
      "author": "Atlas Team",
      "description": "Demo plugin",
      "homepage": "https://github.com/ohdsi/atlas"
    }
  }
]
}

```

The relevant fields:

version manifest format version (currently "1.0").

plugins[] one entry per plugin. Each plugin specifies an `id` (unique within the manifest), a human-readable name, the plugin's version, and an `entryPoint`. A relative `entryPoint` is resolved against a hard-coded `<BASE_URL>/plugins/` — the deployment's base path followed by `plugins/`. An `entryPoint` that is already absolute (starts with `/`) or a full http URL is used verbatim, which is how a plugin bundle hosted on a separate origin is referenced. The plugin's `menuItems[].route` **must** start with `/plugins/<id>/` — the host rejects the manifest otherwise. A menu item can also carry `insertBefore` or `insertAfter` naming a built-in entry (`datasources`, `concepts`, `cohorts`, `analysis`) to control where it lands.

mountPoints optional. Places a plugin inside an existing area of the host instead of the top navigation. Each mount point needs an `id`, a `surface` and a `name`; optional fields are `icon`, `path`, `group`, `hint`, `order` (default 999), `insertBefore` / `insertAfter`, `requiredPermissions` (the entry is shown if the user holds *any one* of them), and `visible`. The surfaces are:

datasource-sidebar a report entry in the **Data Sources** sidebar, grouped under `group` (default "Plugins"); the plugin is told the selected source key.

analysis-tabs a tab in the **Analysis** hub, served at `/analysis/x/<pluginId>/<itemId>`.

admin-tabs a tab in the configuration panel. A plugin admin tab alone is enough to make the configuration cog appear.

account-menu an entry in the user menu, linking to `/plugins/<id>/<path>`. `path` must be relative, with no `..` and no URL scheme, or the manifest is rejected.

`main-nav` is not accepted as a surface — use `menuItems` for top-navigation entries.

fabMounts optional. A plugin can mount a *floating action button*. Each entry carries an `id`, a `label`, an `icon`, an optional `color`, and an optional `position` (`bottom-right`, `bottom-left`, `top-right` or `top-left`; the default is `bottom-right`). The host currently provides one button slot, `pythia.fab`, shown only when `enablePythia` is `true` in `config-local.json` and the user is signed in; entries with any other `id` are ignored.

metadata optional descriptive fields; surfaced nowhere user-facing but useful for audit queries. `metadata.homepage` is validated as a URL and `metadata.icon` as a string; an invalid URL rejects the whole manifest.

The manifest also has an optional `settings` block that tunes the host without touching code:

enableHotReload reload plugins on bundle changes (development only; default `true`).

loadTimeout milliseconds to wait for a plugin bundle before failing the load (default 30000).

pluginsPath parsed and validated into the manifest, but **not currently honoured** by the plugin loader: relative `entryPoints` are always resolved against the hard-coded `<BASE_URL>/plugins/` described above, regardless of this setting. Treat it as reserved for now.

showLoadingIndicators whether the host shows a spinner while a plugin's bundle is fetching.

navigation.enabledCoreItems / **disabledCoreItems**: explicit lists used to hide built-in nav entries on a deployment that does not need them. `disabledCoreItems` takes precedence.

Colour and logo keys `theme.primaryColor`, `theme.accentColor`, `theme.logoUrl`, `theme.logoNavigateTo` and `theme.landingLogoUrl` override colours and logos. `accentColor` drives the accent used for highlights; `landingLogoUrl` replaces the hero image on the home page. Colours are regex-validated as hex codes (e.g. `#1f425a`); an unrecognised value rejects the manifest.

theme.chartColors, **theme.treemapGradient** arrays of hex colours replacing the categorical palette used by multi-series charts and the light-to-dark gradient used by treemaps.

theme.defaultMode, **theme.enableDarkMode** the theme a first-time visitor starts in (`light`, `dark` or `system`; default `light`), and whether the theme selector is shown at all (default `true`; see Section 2.5).

`header.show...` toggles for the navbar slots: `showNavBar`, `showFeedbackButton`, `showLanguageSelector`, `showConfigButton`, `showUserMenu`; plus `header.feedbackUrl` for the external feedback link target.

A common deployment hygiene check is "does the manifest only list plugins we expect to ship?" — since the manifest is checked into deployment config, a code review on `plugins.json` catches the same supply-chain risks as a code review on the application itself.

16.5 Errors and troubleshooting

If a plugin fails to load, the host shows a **Plugin Failed to Load** card in place of the plugin's UI. A **Show Details** / **Hide Details** toggle reveals the error, and a **Retry** button appears when the error is recoverable. The most common causes are network errors fetching the bundle, version skew between plugin and host, and configuration errors in `plugins.json`. Reload the page after the operator has redeployed; if the problem persists, open the browser developer console and forward the single-spa lifecycle errors to your administrator.

Troubleshooting

The most common problems users hit on ATLAS v3.0, and how to recognise them. Items are grouped by symptom rather than by chapter; if you know what is wrong already, jump to the relevant reference chapter for the long form.

17.1 I cannot log in

Symptom: the credentials I usually use are rejected. Check the WebAPI clock. ATLAS uses JWT tokens signed by WebAPI; if the WebAPI host's clock has drifted more than a minute or two relative to the browser, every freshly issued token looks already-expired and the login appears to fail (Chapter 2).

Symptom: my OAuth/SAML/OIDC button is missing. The login providers are configured on the WebAPI side. Ask your administrator whether `SECURITY_AUTH_*` for that provider is set to `true`; on a fresh deployment only DB-auth is enabled by default (Appendix A).

Symptom: I expected anonymous access but get a login prompt. At startup ATLAS calls `user/me`. Anonymous browsing works only when the WebAPI is actually configured to allow it — then that call returns 200 with an anonymous permission set. If it returns 401 instead (anonymous access not enabled on WebAPI), the frontend's `userAuthenticationEnabled` flag is on, and the page you opened requires sign-in, the router opens the login modal. Ask your administrator to enable anonymous access on WebAPI, or expect to sign in.

Symptom: an OIDC/SAML redirect returns but I am still not logged in. The provider bounced you back to the callback route (`/openid/callback`, `/saml/callback` or `/oauth/callback`) but ATLAS could not obtain a session token. All three routes behave the same way: if the URL carries a one-time `?code=`, ATLAS redeems it through WebAPI's `/user/login/otc` and shows WebAPI's error if that fails; otherwise it looks for a token in local storage or a cookie, and when there is none it reports *"Authentication failed - no token received"*. Either way this is a server-side handshake problem, not a browser one. Check the provider and WebAPI `SECURITY_AUTH_*` configuration (redirect URIs, client secret, callback URLs) with your administrator. Old integrations that pass the token itself in the URL (`?token=...`) are no longer accepted and must be switched to the code exchange.

Symptom: I am repeatedly logged out mid-task. The token lifetime is shorter than your session. ATLAS shows an extend-session dialog before the token expires; if the page is not active the token quietly expires. Auto-saved cohort drafts are restored from session storage on next login, but unsaved analysis runs are lost.

17.2 I cannot see any data

Symptom: the Data Sources page lists my source but every report tab is empty. The OHDSI Achilles routine has not been run against that source's results schema. Atlas reads from the Achilles result tables; if they are missing, every tab renders blank even when the source itself is connected. This is the single most common "set up but nothing shows" cause. See Appendix A for how to run Achilles.

Symptom: the Data Sources page is empty. WebAPI has no sources registered. Check `<host>/WebAPI/source/sources` — if it returns an empty JSON array, no source rows exist. Add them via the WebAPI database tables or the atlasdb seed scripts (Appendix A).

Symptom: per-source record-count columns or live patient counts stay blank. Live patient counts in the cohort builder come from the per-source patient cache. If the cache has not been built (or is stale after a CDM refresh), they remain dashes. An administrator can rebuild the cache from the configuration panel's **Cache** tab; build progress is shown on the source's row in the **Patient Cache** card (Chapter 15), not in the jobs drawer.

Symptom: record counts in vocabulary search are blank or all zero. Search-result record counts come from the source picked in **View record count for:** above the results table (Chapter 5). Only sources with a Results daimon are listed, and a source whose Achilles results are missing reports zero for every concept. Pick a source with Achilles results; if loading fails, ATLAS says *"Failed to load record counts for the selected source."*

17.3 Cohort generation does not produce what I expect

Symptom: the cohort generates zero rows. First check the **Patients** column on the source's row in the **Generation** section — that is the cohort's actual record count. Then click **Inclusion report**. If the report says *"This generation recorded no inclusion statistics for this mode, so there is no attrition to report"*, the run simply stored no attrition data; it does not mean the cohort is empty, so trust the **Patients** count. Otherwise, the first step of the attrition funnel is the entry-event count; if it is already zero, your concept set has no records on this source. Either the concept set resolves to nothing (open the set's **Included** tab to see what it expands to, and check record counts in vocabulary search or the concept's details), or the entry-event window or attribute filters exclude every candidate. Run the cohort against a source you know has the relevant data first.

Symptom: the cohort produces vastly more rows than expected. Almost always one of: (i) **Cohort Entry On** is set to **all** when you wanted **Earliest**, multiplying the row count; (ii) a concept set has **Descendants** ticked on a parent that pulls in unintended children (Chapter 6).

Symptom: an inclusion rule drops nobody. The rule is either trivially satisfied (cardinality too lax, temporal window too wide) or redundant with an earlier rule. Tighten or remove. The inclusion-rule debugger guidance in Chapter 7 walks through this.

Symptom: the same JSON produces wildly different counts on two sources. Almost always an ETL or vocabulary-mapping difference, not a real clinical signal. Run the same cohort on a third source (Chapter 7); if its attrition pattern matches one of the two but not the other, the outlier is the ETL-suspect source.

Symptom: generation takes much longer than expected. Open the jobs drawer (the jobs icon in the top navigation) and confirm the job is still **RUNNING**. The drawer has no log view: SQL warnings and database-side errors are in the WebAPI server log, which an administrator can read. Common causes: a concept set with thousands of resolved members hammering `CONDITION_OCCURRENCE`, or a missing index on the results schema (an administrator concern).

17.4 Concept set issues

Symptom: the search returns no results for a term I know is in the data. Check the result filters first. A leftover term in the **Filter results** box, or a facet ticked in the **Filters** menu (the badge on the button counts active filters), hides rows without touching the search itself; **Clear All Selections** in the menu resets them. Then check the Vocabulary facet — if it excludes the vocabulary your data uses you will see nothing. ICD codes live in the ICD vocabulary; SNOMED standards live in SNOMED; non-US drugs live in RxNorm Extension rather than RxNorm.

Symptom: a concept set with Descendants ticked covers thousands of unrelated terms. Either the parent concept is too high in the hierarchy — **Descendants** pulls down everything — or the parent concept is itself non-specific. Move down the hierarchy or narrow the parent. The **Included** tab shows exactly what the set expands to, and the concept's **Descendant Record Count** in vocabulary search or the hierarchy dialog (Section 5.3.5) tells you how much data the parent drags in.

Symptom: my concept set looks right but the cohort matches nobody. The source's `*_concept_id` columns may be populated from a different vocabulary than the one you selected (e.g. the ETL maps to SNOMED while you searched on ICD). Tick **Mapped** on the rows in the **Selected** tab so the standard concepts your source codes map to also count, then check on the **Included** and **Source Codes** tabs that the expansion now covers the codes your data uses before re-generating.

Symptom: *"Recommendations are not available."* The **Recommend** tab needs the PHOEBE 2.0 vocabulary tables, which are not part of a standard vocabulary download. The banner links to **How to install the PHOEBE 2.0 tables**; ask your administrator to load them into the vocabulary schema.

Symptom: concept-set rows read *"Not in the selected vocabulary"*, or Optimize is disabled. The set references concept IDs that the current vocabulary source does not contain, usually because the set was built against another vocabulary release. The IDs are kept and still count in cohort SQL run against a source that has them; only their details cannot be shown here, and the optimizer refuses to run because it would silently drop them. Switch the vocabulary source in the configuration panel to one that contains the concepts (Section 6.4.1).

17.5 Permissions: I cannot see or do something

Symptom: the cog icon for the configuration panel is not visible. The cog is hidden from users who hold none of the admin permissions (`admin:cache`, `admin:source`, `admin:tags`, `admin:security`) and for whom no plugin contributes an admin tab. Job permissions do not count: jobs have their own icon in the top navigation. Ask your administrator to add you to a role that carries the relevant permission (Chapter 15).

Symptom: the configuration panel opens but only some tabs are present. Each admin tab is gated by its own permission, so a role with only `admin:cache` sees only the **Cache** tab. The other tabs are not disabled; they are not rendered at all. This is expected. The exception is **Data Sources**: it is shown to anyone with read access to a source, but controls you cannot use are disabled with the tooltip *"You do not have permission for this action"*.

Symptom: Save or Delete is disabled on something I am editing. Hover the disabled **Save**: the tooltip names the reason (Section 2.4). The common ones are:

- *"Give the ... a name before saving."* — the most frequent cause. A cohort needs nothing else: it can be saved as a draft with no entry event.
- *"You do not own this ... and have not been granted write access to it."* — you are not the author and your role lacks the global `write:...` permission. The owner or an administrator can grant you `WRITE` on this one asset with **Configure access** (Section 15.4).
- *"You are previewing an earlier version."* — return to the current version first (Chapter 14).

Symptom: Generate all or Run is disabled on a cohort. Generation runs the saved definition, so it is blocked while the editor has unsaved changes (tooltip *"Save changes before generate"*) or the design fails validation (*"Design is not valid"*). Save, fix any

messages under the validation icon in the toolbar, and try again. A row whose source you cannot write to stays disabled regardless (Chapter 7).

17.6 Imports and analysis runs

Symptom: importing a cohort JSON is refused. The cohort importer, and the **View / Edit** JSON dialog, validate the expression strictly: a field that circle does not define is refused, with its path and, where possible, a suggestion, e.g. *"InclusionRules[0].title: unrecognized field (did you mean "name"?)"*. Nothing is saved. Fix or delete the named field — usually a hand edit or an export from a tool that adds its own keys — and import again (Chapter 7).

Symptom: a characterization shows "Run failed". The run's status is FAILED; the exit message WebAPI recorded is shown in red under the run summary. It is usually a SQL or feature-extraction error on that source — read the first line of the message, fix the design (for example, remove a feature analysis the source cannot compute), and **Rerun**.

Symptom: "Run finished, but its results could not be loaded". The run completed, but fetching its results from WebAPI failed; the server error is shown below the message. The results usually still exist — reload, or pick the run again from **Previous runs**; if the error persists, check the WebAPI log (Chapter 9).

17.7 Errors that show up in the browser console

CORS errors. The frontend is on a different origin from WebAPI, and WebAPI's SECURITY_CORS_ALLOWED_ORIGINS does not list the ATLAS origin. Add the origin or put both behind the same reverse proxy (Appendix A).

401 Unauthorized. The bearer token has expired or was rejected. Reload the page; ATLAS will prompt for a fresh login.

404 Not Found on a /WebAPI route. The WebAPI base the frontend calls points at the wrong place. ATLAS reads it at runtime from `api.url` in `config-local.json`; if that file sets no `api.url`, the built-in default `/WebAPI` (same origin) is used. On the compose stack the same-origin `/WebAPI` path is served by Caddy's `/WebAPI reverse_proxy` handler, so a 404 there means either `api.url` is wrong or Caddy is not forwarding to the WebAPI service. The appendix's verify-step is to call `<host>/WebAPI/info` and check the JSON response (Appendix A).

500 Internal Server Error on a /WebAPI route. A WebAPI-side problem. The response body usually carries a human-readable error; if not, check the WebAPI log.

17.8 Plugin problems

Symptom: a plugin entry in the navigation shows an error card instead of its UI.

The plugin bundle failed to load. Common causes: a network error fetching it, version skew between plugin and host, or a configuration error in `plugins.json` (Chapter 16). The browser developer console usually carries a single-spa lifecycle error that points at the issue.

17.9 When the answer is not in this manual

- Live WebAPI reference: `<host>/WebAPI/swagger-ui/` (covered in Appendix A).
- Atlas3 source / issue tracker: github.com/OHDSI/Atlas3.
- OHDSI community forum: forums.ohdsi.org.
- The Book of OHDSI [20] for the methodology behind any feature.

Setup guide: WebAPI and ATLAS v3.0

This appendix walks through a working installation of WebAPI and the ATLAS v3.0 frontend. The goal is to get a developer or evaluator from zero to a running web application that talks to a real PostgreSQL backing store. For production deployments, follow your organisation's infrastructure standards rather than the recipes here — this chapter is intentionally minimal.

ATLAS v3.0 requires **WebAPI v3.0**. The 2.x line of WebAPI that the production ATLAS uses — and that ships with the OHDSI Broadsea distribution — is *not* compatible. The v3.0 backend introduces schema and endpoint changes the frontend relies on, and the frontend assumes the v3.0 contract from its first request. The development docker compose in the Atlas3 repository builds its WebAPI image *on top of* the matching base image `ghcr.io/ohdsi/webapi:3.0-dev` for exactly this reason; if you build WebAPI from source instead, check out the `webapi-3.0` branch explicitly.

A.1 The pieces

A working ATLAS v3.0 deployment has three independent components:

A relational database holding the OHDSI WebAPI schema, the OMOP Common Data Model schemas you want to analyse, and any result schemas. PostgreSQL is the simplest choice and the one the development `docker-compose.yml` ships with; Microsoft SQL Server, Oracle, Amazon Redshift, and Snowflake are all supported by WebAPI.

The WebAPI service a Java/Spring application that exposes the REST endpoints ATLAS talks to, executes cohort SQL against the configured CDM sources, and runs Flyway migrations against its own schema on first boot.

The ATLAS v3.0 frontend a Vue 3 single-page application, served either by the Vite development server or as a static bundle behind a reverse proxy.

In a Docker Compose setup these are three services. In a production deployment they may be one database cluster, one Java application server, and a static-asset host.

A.2 Prerequisites

- Docker and Docker Compose v2 (the fastest path).
- *or* for native installs: PostgreSQL 12+, JDK 21 (the `webapi-3.0` branch compiles with Java 21), Maven 3.9+, Node.js 20.19+ or 22.12+ (required by Vite 7) and npm.
- Git.
- Roughly 4 GB of free disk for container images and the WebAPI Flyway baseline.
- A WebAPI build from the v3.0 line — container image `ghcr.io/ohdsi/webapi:3.0-dev` or a local build of the matching branch from the [WebAPI repository](#).

A.3 The Docker Compose path (recommended for evaluation)

The ATLAS v3.0 repository ships a `docker-compose.yml` that brings up Postgres, WebAPI, a database initialiser, and the frontend behind Caddy with a self-signed certificate. From a fresh checkout:

```
git clone https://github.com/OHDSI/Atlas3.git
cd Atlas3
touch .env
docker compose up -d
```

The `atlas3-webapi` service reads a `.env` file, and Compose refuses to start (*"env file .../.env not found"*) if it is missing. A fresh clone ships only `.env.example`; both of its variables are optional and only switch on the Pythia AI assistant (`VITE_BAO_AGENT_ENABLED`, `AWS_BEARER_TOKEN_BEDROCK`), so an empty `.env` is enough for an evaluation stack. Copy `.env.example` to `.env` and fill it in only if you want Pythia.

The stack does not pull a prebuilt WebAPI image verbatim. The `atlas3-webapi` service *builds* a local image (`docker/webapi/Dockerfile`) that starts FROM `ghcr.io/ohdsi/webapi:3.0-dev`, pinned to a specific image digest, and overrides the bundled TrexSQL plugin jar with a pinned release (`TREXSQL_VERSION=0.3.3`, downloaded from [OHDSI/trex](#)). The database is the `ohdsi/broadsea-atlasdb:2.3.0` image (service `atlas3-postgres`, network hostname `broadsea-atlasdb`), and a separate `atlas3-db-init` step runs the `atlasdb/` seed scripts once WebAPI reports healthy. ATLAS v3.0 will not work against the WebAPI 2.x line, which is why the image is pinned to the v3.0 base. The first up spends a few minutes building the WebAPI and frontend images; after that WebAPI typically reports healthy within a minute, once its Flyway migrations have run into the `webapi` schema (baseline 3.0.0), and the init step then seeds the default users and one data source, the *OHDSI Eunomia Demo Database* (source key `EUNOMIA`).

When the stack is healthy:

- ATLAS v3.0 is at <https://localhost/atlas/> — Caddy redirects the bare / to /atlas/ and mints a self-signed certificate, which you must accept. Routing is *hash-based*, so deep links and OAuth/OIDC/SAML callbacks carry the in-app path after a #, e.g. <https://localhost/atlas/#/cohorts>. Links in the ATLAS 2.x form ([#/cohortdefinitions](#)) are redirected (Section 2.7).
- WebAPI is at <http://localhost:8080/WebAPI/info>; that endpoint returns a JSON blob with the version once the service is up. Caddy also reverse-proxies it at <https://localhost/WebAPI/> on the same origin as ATLAS.
- PostgreSQL is *not* published to the host. It is reachable only on the compose network, as `broadsea-atlasdb:5432` (equivalently `atlas3-postgres:5432`). To reach it from the host, exec `psql` inside the container or add an explicit ports mapping to the service yourself.

Note

The Caddy front end reads four environment variables (defaults in parentheses): `CADDY_HOSTNAME` (`localhost`), the site address it answers on; `CADDY_TLS_DIRECTIVE` (`tls internal`), the TLS directive, whose default auto-mints a local certificate; and `WEBAPI_HOST` / `WEBAPI_PORT` (`atlas3-webapi` / `8080`), where Caddy forwards `/WebAPI`. Set them in a `.env` file to serve on a real hostname with a real certificate. The `WEBAPI_URL` variable in `docker-compose.yml` is not read by anything; to point the frontend at a different WebAPI, override `/srv/atlas/config-local.json` (below).

The stack has database login switched on. The seed script `atlasdb/050_auth_user.sql` creates two users, `admin` / `admin` (with the administrator role) and `ohdsi` / `ohdsi`; *change both passwords before any non-local use*.

The default `POSTGRES_PASSWORD` in the compose file is `mypass`. It is a placeholder for local development. Set `POSTGRES_PASSWORD` in a `.env` file (or your secrets manager) before running anywhere a hostile network can reach.

Note

The compose file already enables the TrexSQL query cache (`TREXSQL_ENABLED=true`, `TREXSQL_CACHE_PATH=/data/cache`, `HOME=/data`) and persists it on the `atlas3-webapi-data` named volume. Live patient counts in the cohort builder and the inclusion-rule rail only appear once TrexSQL is enabled *and* a per-source patient cache has been built — trigger the first build from Configuration → Cache after login. See Chapter 15, "Caches and TrexSQL".

A.4 Native install

If you want each component on the host without Docker, the steps below mirror the compose recipe.

A.4.1 1. Database

1. Install PostgreSQL 12 or newer.
2. Create a database (`ohdsi` is a common choice) and a `webapi` schema.
3. Create a role with `CREATE` privileges on that schema — WebAPI needs to run Flyway migrations against it on startup.
4. Optionally, restore one or more OMOP CDM databases as additional schemas. These are the data sources the application will offer in **Data Sources**.

A.4.2 2. WebAPI v3.0

1. `git clone --branch webapi-3.0 https://github.com/OHDSI/WebAPI.git`.
The default branch of the WebAPI repository tracks the production 2.x line, which ATLAS v3.0 cannot talk to; explicitly check out the `webapi-3.0` branch.
2. Create a Maven settings profile or `application.properties` override that maps the database connection details to the property keys WebAPI expects. The compose file lists the relevant variables verbatim — see `atlas3-webapi.environment` in `docker-compose.yml` for the canonical names (`DATASOURCE_URL`, `DATASOURCE_USERNAME`, `DATASOURCE_PASSWORD`, `DATASOURCE_OHDSI_SCHEMA`, the `SPRING_FLYWAY_*` keys, and the `SECURITY_AUTH_DB_DATASOURCE_*` keys for database-backed authentication). PostgreSQL is the default datasource dialect; no extra Maven profile is required for it. For Oracle, MSSQL, Redshift, Snowflake, BigQuery, etc. activate the matching `webapi-*` profile from `pom.xml` to pull in the JDBC driver.
3. Build the Spring Boot fat-jar:
`mvn package -Dpackaging.type=jar`
The result is `target/WebAPI.jar`. (Without the `-Dpackaging.type=jar` flag, Maven produces a WAR; WebAPI v3.0 is designed to run as the fat-jar, not in an external servlet container.)
4. Run it with `java -jar target/WebAPI.jar`, passing the `DATASOURCE_*` and `SPRING_FLYWAY_*` settings as environment variables or via `--spring.config.location`. WebAPI will run Flyway migrations against the configured schema on first boot.
5. Confirm <http://localhost:8080/WebAPI/info> returns the version JSON.

A.4.3 2a. Enable the TrexSQL query cache

Live patient counts in the cohort builder, the inclusion-rule rail, and several descriptive views all read from *TrexSQL*, a query-result cache that **is not part of the WebAPI fat-jar** and must be added separately. The WebAPI image used by the Docker Compose

stack already bakes it in — in fact the stack rebuilds that image to pin a specific TrexSQL release (see *The Docker Compose path* above) — so this step matters only for native installs that built `WebAPI.jar` themselves.

Only **one artefact** is required: the DuckDB-backed `trexsql.jar`, a self-contained uberjar published on the [OHDSI/trex](#) GitHub releases page. It embeds its native DuckDB library inside the jar, so the separate per-platform shared library (`libtrexsql.so` / `.dylib` / `.dll`) and the JNA platform-directory layout that older instructions described are **no longer needed**. Match the jar version to the WebAPI build you assembled — the `ARG TREXSQL_VERSION` line in the `docker/webapi/Dockerfile` of the Atlas3 repo pins the combination used for the official image and is the safest reference. Drop the jar into a plugins directory the launcher will scan:

```
/opt/webapi/plugins/trexsql.jar
```

Then set three environment variables and one JVM flag on the WebAPI process:

`TREXSQL_ENABLED=true` turns the cache on. With this unset or false the `/WebAPI/trexsql/*` endpoints respond 404, the cohort builder hides the patient-count bar entirely, and the Configuration → Cache page omits the TrexSQL section.

`TREXSQL_CACHE_PATH` absolute filesystem path where the cache files live, e.g. `/var/lib/webapi/cache`. WebAPI creates it on first boot and writes per-source subdirectories underneath. Place it on a volume that survives restarts — losing the directory forces a full rebuild for every source.

`HOME` writable directory the cache uses for per-user working files. The compose stack sets `HOME=/data` alongside `TREXSQL_CACHE_PATH=/data/cache`; mirror that pattern by pointing `HOME` at a writable location owned by the WebAPI process user. Without a writable `HOME`, cache initialisation fails on startup.

`-Dloader.path=/opt/webapi/plugins` JVM flag that tells Spring Boot's properties launcher to scan the plugins directory for additional jars. Without it, the `trexsql.jar` on disk is never loaded and the `TREXSQL_ENABLED` flag is a no-op.

Putting it together, the launch command becomes:

```
java
  -Dloader.path=/opt/webapi/plugins
  -jar target/WebAPI.jar
```

with `TREXSQL_ENABLED`, `TREXSQL_CACHE_PATH`, `HOME` and the usual `DATASOURCE_*` variables exported in the environment. After restart, confirm the cache endpoints are mounted by hitting one of them for a known source key (substitute `<sourceKey>` for a key returned by `/WebAPI/source/sources`):

```
WEBAPI=http://localhost:8080/WebAPI
curl -s $WEBAPI/trexsql/<sourceKey>/cache/status
```


At startup WebAPI may log warnings such as *"Failed to download trex extension 'circe'"*; they concern optional add-ons and do not affect the patient cache. The status endpoint should then return a JSON object rather than a 404: before the first build it reads `"cacheExists":false`; after a successful build `cacheExists` and `cacheAttached` are `true`, with the cache file's path and size, and `lastJob` records the build (`"status":"COMPLETE"`). Then sign in to ATLAS v3.0 and open Configuration → Cache; each registered source appears in the **Patient Cache** card with a **Build Cache** button (**Update** once a cache exists). The first build runs as a long job (Chapter 15); only after it finishes does the cohort builder start showing live patient counts for that source.

Note

Per-user access to TrexSQL is gated by the `trexsql:*` permission group (Chapter 15). The built-in admin role has the full set; for non-admin users, add the relevant flags to their role before they can use live patient counts or trigger a rebuild.

A.4.4 3. Add data sources to WebAPI

WebAPI reads its source list from a database table populated by the `atlasdb` initialisation scripts (or by the [Broadsea](#) stack). Each row points WebAPI at one CDM database with a JDBC URL, schema names for CDM and results, and a friendly source key. Until at least one source exists, the ATLAS v3.0 **Data Sources** page is empty.

A.4.5 4. ATLAS v3.0 frontend

1. `git clone https://github.com/OHDSI/Atlas3.git && cd Atlas3.`
2. `npm install.`
3. Point the frontend at WebAPI and configure authentication. In ATLAS v3.0 this is done at *runtime*, not at build time: on startup the app fetches `config-local.json` from the same directory as `index.html`, Zod-validates it, and merges it over the built-in defaults (see *Externalized runtime configuration* below). The old build-time `VITE_AUTH_ENABLED`, `VITE_AUTH_SKIP_LOGIN` and `VITE_AUTH_PROVIDERS` variables are no longer read. For local development, drop a `config-local.json` into the served root (e.g. `public/`) with at least `api.url`, `userAuthenticationEnabled`, and your `authProviders`.
4. For development, run `npm run dev`; the dev server listens on `http://localhost:5173`, opens it in your browser, and proxies `/WebAPI` requests to `http://localhost:8080` by default. To point the dev server at a WebAPI on a different host or port (a remote shared instance, a native build on 8081, etc.), set the `WEBAPI_URL` environment variable when starting it, e.g. `WEBAPI_URL=https://atlas-preview.ohdsi.org npm run dev`. The proxy rewrites the request origin to the target, so WebAPI's CORS filter accepts it.

The bundle's own WebAPI base comes from `api.url` in `config-local.json` (default `/WebAPI`), which does not affect the dev proxy target.

5. For production, run `npm run build` and serve the contents of `dist/` from a static host. A reverse proxy must forward `/WebAPI` to the Java service so the same origin serves both. The repository's Caddyfile shows a working configuration. Production builds emit source maps, so stack traces in the browser console point at the original source files. Setting `VITE_ENABLE_PROD_DEVTOOLS=true` at build time additionally enables the Vue devtools browser extension against the production bundle — useful for diagnosing a deployed instance, not for general use.

Note

`npm run dev` is the right entry point for trying things out or for plugin development; it runs the Vite dev server with hot module reload. The compose stack instead builds the static bundle into the Caddy image, which is what a production deployment would do.

A.4.6 Externalized runtime configuration

Auth and feature configuration is loaded at *runtime*, so the same static bundle can be re-pointed without a rebuild. On startup the app fetches `config-local.json` from the directory that serves `index.html`, validates it against a Zod schema, and merges the recognised keys over the built-in defaults (a shallow merge; arrays replace entirely). A missing or malformed file falls back to defaults with a console warning. In the Docker image the file is baked in at `/srv/atlas/config-local.json` (copied from `docker/config-local.json`).

The keys the loader recognises:

api.url WebAPI base URL (default `/WebAPI`, i.e. same-origin behind the reverse proxy).

userAuthenticationEnabled whether a session is required. The shipped image sets it `true`; left `false` (the default) the app runs against WebAPI's anonymous access (Chapter 2).

enableSkipLogin when authentication is enabled, opens the login dialog automatically on startup for users who are not signed in.

refreshTokenThreshold how long before the session token expires ATLAS refreshes it (default 900000 ms, i.e. 15 minutes).

enableIAPSession keeps the session alive behind Google Identity-Aware Proxy (default `false`).

enablePythia shows the Pythia assistant button for signed-in users (default `false`; Chapter 16).

authProviders the list of login providers (name, login URL, icon, whether it uses the credentials form) rendered in the login modal.

Unused keys `enablePermissionManagement`, `enableTermsAndConditions`, `enablePersonCount`, `enableTaggingSection`, `defaultLocale` and `pollInterval` are accepted by the schema but not used by this build.

A.5 First-time login and creating a user

The default DB-auth path expects a row in the `webapi.auth_user` table whose password column is a bcrypt hash of the chosen plaintext. The compose seed scripts in the `atlasdb/` folder create an initial administrator. To add more users, either insert rows into `webapi.auth_user` directly or, if your deployment exposes LDAP/OAuth/SAML/OIDC, configure the corresponding `SECURITY_AUTH_*` group of properties on the WebAPI side and let users log in via that flow instead.

After signing in, an administrator should:

- visit Configuration → Permissions and set up the role bundles their organisation needs (Chapter 15);
- confirm the **Data Sources** page lists each connected CDM and that its dashboard renders without errors (Chapter 4);
- run a small smoke-test cohort against each source to validate end-to-end execution (Chapter 7).

A.6 Common installation issues

The ATLAS page loads but every list is empty. WebAPI is reachable but no CDM sources are configured. Add rows to the WebAPI source tables (or run the `atlasdb` init scripts).

CORS errors in the browser console. The frontend and WebAPI are on different origins and WebAPI's `SECURITY_CORS_ALLOWED_ORIGINS` does not include the ATLAS v3.0 origin. Either add the origin to the allow-list or put both behind the same reverse proxy.

Flyway error on WebAPI startup. The configured database user lacks privileges on the `webapi` schema, or a previous WebAPI version left a migration in an inconsistent state. Inspect the WebAPI log; Flyway prints the failing migration explicitly.

Login fails with valid credentials. Check the WebAPI clock against the host running the browser. A skew larger than the JWT clock-tolerance window makes every freshly issued token look already-expired.

Data Sources page is empty for every tab. The most common case: the source is configured but *Achilles* [9] has not been run against it. ATLAS v3.0's descriptive reports read from the Achilles results tables in the source's results schema; if those tables are missing, every tab renders empty even though the source itself is reachable. See the next section.

A.7 Verifying a clean setup

After the stack is up, run these three checks before declaring victory:

1. `curl -sk https://localhost/WebAPI/info` should return JSON containing the WebAPI version and build date.
2. `curl -sk https://localhost/WebAPI/source/sources` should return a non-empty JSON array with one entry per configured CDM. An empty array means no sources are registered, and every ATLAS v3.0 list page that depends on a source will appear empty.
3. In the browser, open the developer console (Ctrl+Shift+I in Chromium) and watch the Network tab while loading **Data Sources**. You should see 200 responses for the `/source/sources` call and for each per-source report. Repeated 401 responses point at an authentication problem, 404 at routing, 500 at WebAPI itself — the response body usually has a clear error.

A.8 Achilles: the missing prerequisite for descriptive reports

OHDSI Achilles [9] is a separate R package that analyses a CDM database and writes summary tables into the same source's results schema. ATLAS v3.0's **Data Sources** reports *read from those Achilles tables*; if Achilles has never run, the source is connected but the dashboards are blank. Most evaluators hit this once and remember it forever.

For each source you want to use:

1. Install Achilles in an R environment that can reach the CDM, from CRAN or from GitHub:

```
install.packages("Achilles")  
remotes::install_github("OHDSI/Achilles")
```
2. Run `Achilles::achilles(connectionDetails, ...)` with the CDM and results schema names. On a multi-million person database this can take from minutes to hours.
3. Re-run Achilles after every CDM refresh; the results schema is rebuilt from scratch each time.

Two adjacent OHDSI tools you may want to install at the same time: the Data Quality Dashboard [14] for systematic data-quality checks against the CDM (Chapter 4), and PheValuator [18, 37] for formal phenotype evaluation (Chapter 12).

A.9 Related: Broadsea

If the recipe in this appendix feels too low-level, the OHDSI Broadsea [13] project bundles WebAPI, ATLAS, Achilles, the Data Quality Dashboard, and a sample CDM into a single Docker stack with a guided setup. It is the fastest way to demo the whole OHDSI toolchain on a laptop.

The current Broadsea release ships ATLAS 2.x and the matching WebAPI 2.x backend, neither of which works with ATLAS v3.0. Broadsea support for ATLAS v3.0 / WebAPI v3.0 is on the OHDSI roadmap and expected in an upcoming release; until that lands, swapping in the v3.0 components manually is required and non-trivial (the database seed scripts and the WebAPI image both need to change in lockstep). For a v3.0 stack today, use the `docker-compose.yml` that ships in the Atlas3 repository — it is the supported path and what the rest of this appendix documents.

A.10 Further reading and API reference

This manual documents what users do in the ATLAS v3.0 interface; it is not a reference for the WebAPI backend that the UI talks to. For power users who want to call WebAPI directly — to script cohort generation, batch concept-set imports, or build a plugin against the same endpoints the UI uses — the canonical references live upstream:

ATLAS v3.0 frontend repository github.com/OHDSI/Atlas3 — source, issue tracker, build instructions.

WebAPI repository github.com/OHDSI/WebAPI — the v3.0 backend; check out the `webapi-3.0` branch for the version this manual documents.

Live WebAPI reference `<host>/WebAPI/swagger-ui/` — the deployed instance's own swagger UI lists every endpoint with request and response shapes. On a local Docker Compose stack the URL is <http://localhost:8080/WebAPI/swagger-ui/>.

Book of OHDSI ohdsi.github.io/TheBookOfOhdsi — the conceptual reference for OMOP / ATLAS / the OHDSI toolchain. Cited throughout this manual.

Scripts should authenticate with a personal API key rather than a password. Keys are created from the user menu's **API Keys** page (Chapter 2, Section 2.6) and managed through WebAPI's `/user/apikeys` endpoints; the swagger UI of your WebAPI build documents how a request presents the key.

The bibliography section at the back of the manual lists the papers and packages cited from individual chapters, including HADES [23], FeatureExtraction [15], Achilles [9], and PheValuator [37].

Glossary

CDM (Common Data Model) The OMOP Common Data Model is a person-centric relational schema that standardises observational health data so the same analytic code runs against any compliant dataset.

Cohort A set of persons who satisfy one or more inclusion criteria for a duration of time. In ATLAS v3.0 a cohort is defined by an entry event, optional inclusion rules, and an exit rule.

Concept A single entry in the OMOP standardised vocabularies, such as a SNOMED condition, an RxNorm ingredient, or a LOINC measurement.

Concept set A reusable, named collection of concepts, with options to include descendants, include mapped source codes, and exclude specific concepts.

Standard concept A concept the OHDSI vocabulary marks as canonical for its domain. Source codes (ICD-10, NDC, etc.) usually map up to a standard concept. Cohort logic should reference standard concepts so it is portable across data sources.

Era A continuous period of clinical state derived from underlying events — for example, a drug era is built from contiguous drug exposures with a configurable persistence gap.

Index date The cohort entry date for a person; most criteria are expressed as windows relative to the index date.

WebAPI The REST backend that ATLAS v3.0 talks to. WebAPI executes cohort generation, vocabulary search, and reporting against the underlying CDM databases.

Source / data source A configured CDM database. ATLAS v3.0 lists every source that WebAPI is connected to; reports and cohort generation always run against a chosen source.

Characterization An analysis that summarises clinical features of a target cohort, optionally compared to outcomes, using a set of feature analyses.

Feature analysis A reusable definition of a covariate or summary statistic (e.g. "drug exposure in the 365 days before index"). Used by characterizations.

Cohort pathway A visualisation of the order in which event cohorts occur for the persons in a target cohort.

Incidence rate The number of new outcome events per person-time-at-risk inside a target cohort.

Inclusion rule A criterion applied after the entry event that a person must satisfy to remain in the cohort.

Censoring event A clinical event that ends a person's cohort membership before the configured exit window would otherwise close it.

PHOEBE A vocabulary-recommendation engine packaged with WebAPI that suggests additional concepts likely to belong in a concept set, drawn from co-occurrence patterns across an OMOP network. ATLAS v3.0 surfaces it as the **Recommend** tab inside the concept-set editor.

TrexSQL A query-result cache shared across users on a WebAPI host, used to short-circuit repeated vocabulary and concept-set lookups. Distinct from the patient cache below.

Patient cache The per-source, pre-aggregated dataset that descriptive reports and the live patient-count bar in the cohort builder read from. Built on demand from the configuration panel's **Cache** tab; rebuilt when the underlying CDM or vocabulary refreshes.

Previous runs The historical generations of an analysis, listed in the **Previous runs** dialog. The dialog is scoped to a single data source and opened from the history icon (MDI-HISTORY, with a count badge once multiple runs exist) on each row of the data-source run table at the top of the Characterization, Incidence Rate, and Pathway editors.

Subgroup analysis An overlapping selection of the population inside a characterization, defined like a small inclusion rule. Each subgroup adds an extra column to the result. (Internally referred to as *strata*; the UI label is "Subgroup analyses".)

Sunburst The radial diagram used to visualise cohort pathways: the index event sits at the centre, each subsequent step is a ring expanding outwards, wedge area is proportional to person count.

Achilles The OHDSI R routine that computes the descriptive statistics ATLAS reads from for the **Data Sources** reports. A source is "Achilles-processed" when those routines have been run against its results schema.

Athena OHDSI's vocabulary repository (<https://athena.ohdsi.org>). The vocabulary tables Athena publishes are loaded into WebAPI; Atlas's vocabulary search runs against that data, so two ATLAS instances on the same release but with different Athena snapshots can return slightly different concept IDs.

HADES The OHDSI R-package suite (*Health Analytics Data to Evidence Suite*). HADES packages such as `FeatureExtraction`, `CohortDiagnostics` and `PheValuator` consume the JSON ATLAS exports for analyses ATLAS itself does not run.

Bibliography

- [1] Peter C. Austin. Balance diagnostics for comparing the distribution of baseline co-variates between treatment groups in propensity-score matched samples. *Statistics in Medicine*, 28(25):3083–3107, 2009.
- [2] EHDEN. Phenotype definition, characterisation and evaluation. EHDEN Academy course, https://www.ehden.eu/course_phenotypedefinitioncharacterisationevaluation/, 2024.
- [3] Health Data Research UK. HDR UK phenotype library. <https://phenotypes.healthdatagateway.org/>, 2025.
- [4] George Hripcsak, Jon D. Duke, Nigam H. Shah, Christian G. Reich, Vojtech Huser, Martijn J. Schuemie, Marc A. Suchard, Rae Woong Park, Ian Chi Kei Wong, Peter R. Rijnbeek, Johan van der Lei, Nicole Pratt, G. Niklas Norén, Yu-Chuan Li, Paul E. Stang, David Madigan, and Patrick B. Ryan. Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers. *Studies in Health Technology and Informatics*, 216:574–578, 2015.
- [5] George Hripcsak, Patrick B. Ryan, Jon D. Duke, Nigam H. Shah, Rae Woong Park, Vojtech Huser, Marc A. Suchard, Martijn J. Schuemie, Frank J. DeFalco, Adler Perotte, Juan M. Banda, Christian G. Reich, Lisa M. Schilling, Michael E. Matheny, Daniella Meeker, Nicole Pratt, and David Madigan. Characterizing treatment pathways at scale using the OHDSI network. *Proceedings of the National Academy of Sciences*, 113(27):7329–7336, 2016.
- [6] George Hripcsak, Matthew E. Levine, Ning Shang, and Patrick B. Ryan. Effect of vocabulary mapping for conditions on phenotype cohorts. *Journal of the American Medical Informatics Association*, 25(12):1618–1625, 2018.
- [7] Michael G. Kahn, Tiffany J. Callahan, Juliana Barnard, Alan E. Bauck, Jeff Brown, Bruce N. Davidson, Hossein Estiri, Carsten Goerg, Erin Holve, Steven G. Johnson, Siaw-Teng Liaw, Marianne Hamilton-Lopez, Daniella Meeker, Toan C. Ong, Patrick Ryan, Ning Shang, Nicole G. Weiskopf, Chunhua Weng, Meredith N. Zozus, and Lisa Schilling. A harmonized data quality assessment terminology and framework for the secondary use of electronic health record data. *eGEMs (Generating Evidence & Methods to improve patient outcomes)*, 4(1):18, 2016.
- [8] John Leider and contributors. Vuetify: Material design component framework. <https://vuetifyjs.com/>.
- [9] OHDSI. OHDSI achilles. <https://github.com/OHDSI/Achilles>, .
- [10] OHDSI. Athena OHDSI vocabulary repository. <https://athena.ohdsi.org/>, .

- [11] OHDSI. Atlas v3.0 source repository. <https://github.com/OHDSI/Atlas3>, .
- [12] OHDSI. Ohdsi atlas (production, atlas 2.x). <https://github.com/OHDSI/Atlas>, .
- [13] OHDSI. OHDSI broadsea. <https://github.com/OHDSI/Broadsea>, .
- [14] OHDSI. OHDSI data quality dashboard (DQD). <https://github.com/OHDSI/DataQualityDashboard>, .
- [15] OHDSI. OHDSI FeatureExtraction r package. <https://ohdsi.github.io/FeatureExtraction/>, .
- [16] OHDSI. OMOP common data model specification. <https://ohdsi.github.io/CommonDataModel/>, .
- [17] OHDSI. OHDSI phenotype library. <https://github.com/OHDSI/PhenotypeLibrary>, .
- [18] OHDSI. PheValuator r package. <https://ohdsi.github.io/PheValuator/>, .
- [19] OHDSI. Ohdsi webapi. <https://github.com/OHDSI/WebAPI>, .
- [20] OHDSI. *The Book of OHDSI: Observational Health Data Sciences and Informatics*. OHDSI Collaborators, 2021. URL <https://ohdsi.github.io/TheBookOfOhdsi/>. Open-access community handbook.
- [21] OHDSI. Ohdsi atlas wiki. <https://github.com/OHDSI/Atlas/wiki>, 2024.
- [22] OHDSI. The book of OHDSI, chapter 16: Clinical validity. <https://ohdsi.github.io/TheBookOfOhdsi/ClinicalValidity.html>, 2024.
- [23] OHDSI. Ohdsi hades (health analytics data-to-evidence suite). <https://ohdsi.github.io/Hades/>, 2024. R packages for cohort generation, characterization, population-level estimation, and patient-level prediction.
- [24] OHDSI. Phenotype Phebruary. Annual community phenotyping sprint, <https://www.ohdsi.org/phenotype-phebruary-2024/>, 2024.
- [25] OHDSI. OHDSI themis: Conventions for the omop common data model. <https://github.com/OHDSI/Themis>, 2024.
- [26] OHDSI. Cohort definition submission requirements — OHDSI phenotype library. <https://ohdsi.github.io/PhenotypeLibrary/articles/CohortDefinitionSubmissionRequirements.html>, 2025.
- [27] OHDSI. Evaluating phenotype algorithms (PheValuator vignette). <https://ohdsi.github.io/PheValuator/articles/EvaluatingPhenotypeAlgorithms.html>, 2025.
- [28] OHDSI HADES. CohortDiagnostics: An r package for performing diagnostics on cohort definitions. <https://ohdsi.github.io/CohortDiagnostics/>, 2025.
- [29] J. Marc Overhage, Patrick B. Ryan, Christian G. Reich, Abraham G. Hartzema, and Paul E. Stang. Validation of a common data model for active safety surveillance

- research. *Journal of the American Medical Informatics Association*, 19(1):54–60, 2012.
- [30] Gowtham A. Rao, Martijn J. Schuemie, Azza Shoaibi, et al. CohortDiagnostics: Phenotype evaluation across a network of observational data sources using population-level characterization. *PLOS ONE*, 2025. doi: 10.1371/journal.pone.0310634.
- [31] William A. Ray. Evaluating medication effects outside of clinical trials: New-user designs. *American Journal of Epidemiology*, 158(9):915–920, 2003.
- [32] Regenstrief Institute. LOINC: Logical observation identifiers names and codes. <https://loinc.org/>.
- [33] Jenna M. Reys, Martijn J. Schuemie, Marc A. Suchard, Patrick B. Ryan, and Peter R. Rijnbeek. Design and implementation of a standardized framework to generate and evaluate patient-level prediction models using observational healthcare data. *Journal of the American Medical Informatics Association*, 25(8):969–975, 2018.
- [34] Martijn J. Schuemie, George Hripcsak, Patrick B. Ryan, David Madigan, and Marc A. Suchard. Empirical confidence interval calibration for population-level effect estimation studies in observational healthcare data. *Proceedings of the National Academy of Sciences*, 115(11):2571–2577, 2018.
- [35] SNOMED International. SNOMED CT. <https://www.snomed.org/>.
- [36] Samy Suissa. Immortal time bias in pharmaco-epidemiology. *American Journal of Epidemiology*, 167(4):492–499, 2008.
- [37] Jill N. Swerdel, George Hripcsak, and Patrick B. Ryan. PheValuator: Development and evaluation of a phenotype algorithm evaluator. *Journal of Biomedical Informatics*, 97:103258, 2019. doi: 10.1016/j.jbi.2019.103258.
- [38] Joel N. Swerdel, Martijn Schuemie, Gayle Murray, and Patrick B. Ryan. PheValuator 2.0: Methodological improvements for the PheValuator approach to semi-automated phenotype algorithm evaluation. *Journal of Biomedical Informatics*, 135: 104177, 2022. doi: 10.1016/j.jbi.2022.104177.
- [39] Joel N. Swerdel, Martijn Schuemie, Gayle Murray, and Patrick B. Ryan. Comparing broad and narrow phenotype algorithms: differences in performance characteristics and immortal time incurred. *JAMIA Open*, 2024. doi: 10.1093/jamiaopen/ooae002.
- [40] U.S. National Library of Medicine. RxNorm. <https://www.nlm.nih.gov/research/umls/rxnorm/>.
- [41] Erica A. Voss, Rupa Makadia, Amy Matcho, Qianli Ma, Christopher Knoll, Martijn Schuemie, Frank J. DeFalco, Ajit Londhe, Vivienne Zhu, and Patrick B. Ryan. Feasibility and utility of applications of the common data model to multiple, dis-

- parate observational health databases. *Journal of the American Medical Informatics Association*, 22(3):553–564, 2015.
- [42] Kazuki Yoshida, Daniel H. Solomon, and Seoyoung C. Kim. The active comparator, new user study design in pharmacoepidemiology: established but no longer the standard? *Current Epidemiology Reports*, 2:221–228, 2015.
- [43] Evan You and contributors. Vue.js: The progressive javascript framework. <https://vuejs.org/>.